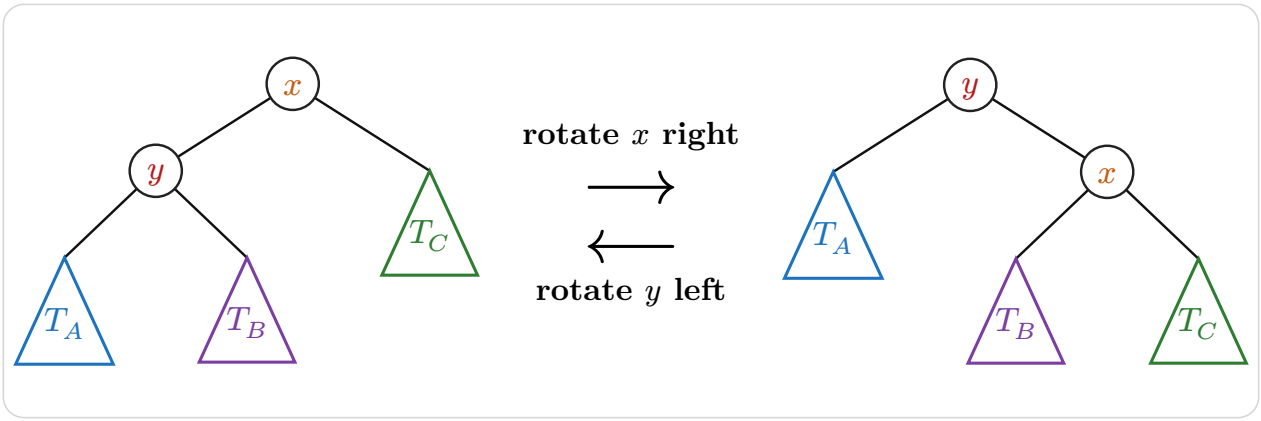
typed-dsa

Declarative data-structure diagrams for Typst

User Guide



Version 0.6.0

Contents

1 Introduction . . . 3
2 Getting started . . . 4
2.1 Installation and import . . . 4
2.2 Your first diagram . . . 5
3 Structures . . . 6
3.1 `bst()` and `avl()` . . . 6
3.2 `min-heap()` and `max-heap()` . . . 13
3.3 `linked-list()` . . . 18
3.4 `doubly-linked-list()` . . . 21
3.5 `stack()` . . . 25
3.6 `queue()` . . . 28
3.7 `skip-list()` . . . 33
3.8 `array-view()` and `matrix()` . . . 37
3.9 `hash-table()` . . . 41
3.10 Sorting algorithms . . . 43
3.11 `graph()` . . . 57
3.12 `labels` and `positions` . . . 72
3.13 `edge-customizations` . . . 73
3.14 Graph algorithm traces . . . 74
4 Hand-composed trees . . . 77
5 Operation transitions . . . 87
5.1 Tree operations: `tree-insert()`, `tree-delete()`, `tree-search()` . . . 92
5.2 Heap operations: `heap-insert()` and `heap-extract` . . . 95
6 Objects and operations . . . 96
6.1 The object model . . . 96
6.2 Chaining operations . . . 97
6.3 Wrapped operation sequences . . . 99
6.4 Custom arrows with `op-arrow()` . . . 102
7 Localization . . . 103
7.1 The three resolution layers . . . 103
7.2 `language:` . . . 103
7.3 `messages:` and the `messages(...)` helper . . . 104
7.4 `step-label:` has the last word . . . 106
7.5 Structural default labels . . . 106
7.6 Message-key reference . . . 106
8 Styling . . . 109
8.1 Named themes and typography roles . . . 109
8.2 Autocomplete-friendly style builders . . . 110
8.3 Where keys are documented . . . 111
8.4 Diff-highlight styling . . . 112
9 Worked example: Last Stone Weight . . . 114
10 Limitations . . . 115
11 Quick reference . . . 116
11.1 Structure builders . . . 116
11.2 Graph algorithms . . . 116
11.3 Sorting algorithms . . . 117
11.4 Hand-composed trees . . . 117
11.5 Transitions and operations . . . 117

11.6 Localization 118
11.7 Styling keys 118
11.8 Styling helpers 119

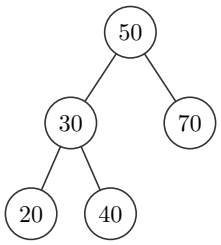
1 Introduction

typed-dsa renders data-structure diagrams from a declarative description inside Typst documents. It is built on top of CeTZ. The package targets CS coursework: lecture notes, problem sets, exam solutions, where you need to show how an **operation** changes a structure, not just draw it once.

This guide covers every builder, argument, and operation, each with a runnable example: source on the left, rendered output on the right.

```
1  #bst(50, 30, 70, 20, 40).diagram
```

Typst



2 Getting started

2.1 Installation and import

Import the package from the Typst preview namespace. A wildcard import gives you every public symbol:

```
#import "@preview/typed-dsa:0.6.0": *
```
Typst

Or import only what you need:

```
#import "@preview/typed-dsa:0.6.0": bst, avl, min-heap, max-heap
```
Typst

> **Note.** A wildcard import shadows Typst's built-in `stack` function. Use `std.stack` for layout, or import `typed-dsa` selectively to avoid the name clash.

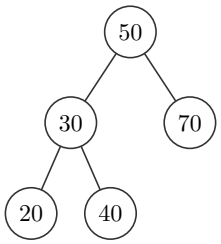
The package exports these symbols:

Symbol	Purpose
`bst` , `avl`	Binary search tree / AVL tree builders.
`min-heap` , `max-heap`	Binary heap builders.
`linked-list` , `doubly-linked-list` , `stack` , `queue`	Linear-structure builders.
`array-view` , `matrix`	Static array and matrix/grid builders.
`merge-sort` , `merge-operation` , `partition-step` , `quick-sort` , `bubble-sort` , `insertion-sort` , `selection-sort`	Automatic sorting traces.
`graph`	Graph builder, from an adjacency dict.
`tree` , `node` , `subtree`	Hand-composed abstract trees.
`transition`	One-shot before → after operation diagrams for trees and heaps.
`tree-insert` , `tree-delete` , `tree-search`	Tree operation constructors, for `transition` .
`heap-insert` , `heap-extract`	Heap operation constructors, for `transition` .
`sequence`	Wrapped layout for operation-step sequences.
`op-arrow`	Reusable labeled arrow, for custom operation layouts.
`theme` , `resolve`	The default style dictionary and its merge helper.

2.2 Your first diagram

Every builder takes keys and returns an object; `.diagram` is its drawing.

```typst
#bst(50, 30, 70, 20, 40).diagram
```



3 Structures

Every structure builder returns a **unified object**: a dictionary that is both the drawing (`.diagram`) and the thing you operate on. Calling an operation field returns a **step**; Section 6 covers that model in full.

This section documents each builder's own arguments and methods. Section 6 covers the step shape they share.

3.1 `bst()` and `avl()`

Full signatures:

```
#bst(..keys, style: (:), edge-customizations: (), node-customizations: (), node-labels: (:), language: "en", messages: (:))
#avl(..keys, style: (:), edge-customizations: (), node-customizations: (), node-labels: (:), language: "en", messages: (:))
```

Typst

Both build a binary tree by inserting `keys` one at a time, in the order given. `bst` is a plain binary search tree; `avl` additionally rebalances after every insertion and deletion, so the tree stays height-balanced.

Argument	Type	Default	Description
`..keys`	`int` / `content`	(required)	Keys inserted into the tree in order, one BST/AVL insert per key. Duplicate keys are ignored.
`style`	`dictionary`	`(:)`	Per-call style override merged over the defaults. See Section 8.
`edge-customizations`	`array`	`()`	`(parent, child, options)` tuples restyling individual tree edges by node value. See Section 3.13.
`node-customizations`	`array`	`()`	`(node, options)` tuples restyling individual tree nodes by value.
`node-labels`	`array` / `dictionary`	`(:)`	Labels drawn outside individual tree nodes.

Nested keys for `style`

Path	Type	Default	Description
`style.node-radius`	`float`	`0.34`	Circle radius, or base size for other node shapes, in canvas units.
`style.node-shape`	`str`	`"circle"`	`"circle"` , `"square"` , `"rounded"` , `"capsule"` , `"diamond"` , or `"hexagon"` .
`style.x-gap`	`float`	`1.05`	Horizontal spacing between in-order columns.
`style.y-gap`	`float`	`1.2`	Vertical spacing between depths.
`style.tree-layout`	`str`	`"columns"`	`"columns"` gives every leaf its own column. `"tidy"` packs sibling subtrees against their contours, keeping parents centered and subtree triangles clear. See Section 3.1.1.

style.node-stroke	stroke	0.6pt + rgb("#333333")	Node outline.
style.node-fill	color	white	Default node fill.
style.edge-stroke	stroke	0.6pt + rgb("#333333")	Parent-child edge stroke.
style.edge-arrow	none / bool / str	none	Edge arrowheads: none / false , "end" or true , "start" , or "both" .
style.edge-pattern	str	"normal"	"normal" , "dashed" , "dotted" , or "wavy" .
style.edge-wave-amplitude	float	0.07	Wave height in canvas units.
style.edge-wave-step	float	0.14	Approximate wavelength in canvas units.
style.node-text	dictionary	(size: 9pt)	Node text dictionary. See nested keys below.
style.value-text	dictionary	inherits node-text	Value text overrides.
style.label-text	dictionary	(fill: rgb("#555555"))	Annotation text dictionary. See nested keys below.
style.edge-label-text	dictionary	inherits label-text	Edge-label typography.
style.operation-text	dictionary	(size: 8pt)	Operation-arrow caption typography.
style.scale	float	1.0	Uniform scale on .diagram , .before , and .after .
style.diff-colors	bool	true	false keeps operation marks but uses normal fills.

Nested keys for `style.node-text`

Path	Type	Default	Description
style.node-text.size	length	inherits	Text size.
style.node-text.color	color	inherits	Text color.
style.node-text.rotation	angle	0deg	Text rotation applied by the diagram renderer.
style.node-text.font	str / array	inherits	Typst text font selection.
style.node-text.weight	str / int	inherits	Typst text weight.

Nested keys for `style.label-text`

Path	Type	Default	Description
style.label-text.size	length	85% of style.node-text.size	Label text size.
style.label-text.color	color	rgb("#555555")	Label text color.
style.label-text.rotation	angle / str	0deg	Label rotation. Tree and graph edge labels also accept "edge" to follow the edge angle.

style.label-text.font	str / array	inherits	Typst text font selection.
style.label-text.weight	str / int	inherits	Typst text weight.

Nested keys for diff-highlight dictionaries

Path	Type	Default	Description
style.new-style	color / dictionary	rgb("#C8E6C9")	Node/cell added by an operation. A color is shorthand for (fill: color) .
style.path-style	color / dictionary	rgb("#FFE9A8")	Traversal or sift path.
style.remove-style	color / dictionary	rgb("#FFCDD2")	Node/cell removed by an operation.
style.rotate-style	color / dictionary	rgb("#BBDEFB")	AVL rotation nodes and visible subtree roots.
style.*-style.fill	color	mark default	Fill for the marked node/cell.
style.*-style.shape	str	style.node-shape	Marked tree/heap node shape: "circle" , "square" , "rounded" , "capsule" , "diamond" , or "hexagon" .
style.*-style.stroke	stroke	normal stroke	Marked node/cell outline.
style.*-style.node-radius	float	style.node-radius	Marked tree/heap node radius.
style.*-style.text	dictionary	style.node-text	Marked node/cell text overrides. Accepts size , color , font , and weight .

Nested keys for edge-customizations

Path	Type	Default	Description
edge-customizations[]	tuple	n/a	One (from, to, options) tuple. For trees, from / to are parent and child values; for graphs, node labels.
edge-customizations[].from	content	required	Source node identity.
edge-customizations[].to	content	required	Target node identity.
edge-customizations[].options	dictionary	required	Per-edge options.
edge-customizations[].options.stroke	stroke	none	Full stroke override. Wins over color if both are set.
edge-customizations[].options.color	color	none	Paint only, at the default thickness.
edge-customizations[].options.pattern	str	"normal"	"normal" , "dashed" , "dotted" , or "wavy" .
edge-customizations[].options.arrow	none / bool / str	none	Per-edge arrowheads: none , "end" / true , "start" , or "both" .

edge-customizations[].options.bend	str / bool	false	"left" or "right" curves the edge relative to its travel direction. false keeps it straight.
edge-customizations[].options.angle	angle	25deg	How sharply a bent edge curves. Ignored when bend is false .
edge-customizations[].options.label	content / dictionary	none	Tree edge label content, or graph edge-label text overrides. A dictionary can include content for trees.

Nested keys for edge-customizations[].options.label

Path	Type	Default	Description
edge-customizations[].options.label.size	length	style.label-text.size	Edge-label size.
edge-customizations[].options.label.content	content	tree only	Tree edge label content. Graph edge labels come from adjacency entries such as ("B", [4]) .
edge-customizations[].options.label.color	color	style.label-text.color	Edge-label text color.
edge-customizations[].options.label.rotation	angle / str	style.label-text.rotation	An angle, or "edge" to follow the tree/graph edge angle.
edge-customizations[].options.label.font	str / array	style.label-text.font	Typst text font selection.
edge-customizations[].options.label.weight	str / int	style.label-text.weight	Typst text weight.

Nested keys for node-customizations

Path	Type	Default	Description
node-customizations[]	tuple	n/a	One (node, options) tuple.
node-customizations[].node	content	required	For bst / avl , the inserted key.
node-customizations[].options	dictionary	required	Per-node drawing options.
node-customizations[].options.fill	color	normal fill	Node fill.
node-customizations[].options.stroke	stroke	normal stroke	Node outline.
node-customizations[].options.shape	str	style.node-shape	Trees/graphs only: "circle" , "square" , "rounded" , "capsule" , "diamond" , or "hexagon" .
node-customizations[].options.node-radius	float	style.node-radius	Trees/graphs only. Shape size.
node-customizations[].options.text	dictionary	style.node-text	Text style merged into the node text.

Nested keys for node-labels

Path	Type	Default	Description
node-labels[]	tuple	n/a	One (node, label) tuple.
node-labels[].node	content	required	For bst / avl , the inserted key.

`node-labels[].label`	`content` / `dictionary`	required	Bare content for quick labels, or a dictionary with `content` plus style and placement keys.
`node-labels[].label.content`	`content`	required for dict labels	Label body. `body` is also accepted.
`node-labels[].label.position`	`str` / `angle`	`"right"`	`"right"` , `"left"` , `"top"` , `"bottom"` , or an angle.
`node-labels[].label.offset`	`point`	`(0, 0)`	Extra `(dx, dy)` shift after the position is applied.
`node-labels[].label.size`	`length`	`style.label-text.size`	Node-label size.
`node-labels[].label.color`	`color`	`style.label-text.color`	Node-label text color.
`node-labels[].label.rotation`	`angle`	`style.label-text.rotation`	Rotation of the label text itself.
`node-labels[].label.font`	`str` / `array`	`style.label-text.font`	Typst text font selection.
`node-labels[].label.weight`	`str` / `int`	`style.label-text.weight`	Typst text weight.

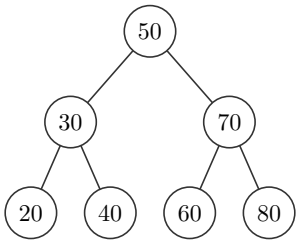
Nested keys for `style.node-labels`

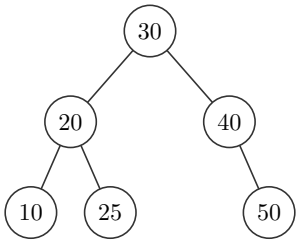
Path	Type	Default	Description
`style.node-labels.position`	`str` / `angle`	`"right"`	Default position for all node labels in this diagram.
`style.node-labels.offset`	`point`	`(0, 0)`	Default extra `(dx, dy)` shift for all node labels.
`style.node-labels.gap`	`float`	`0.22`	Default distance from the node boundary in canvas units.
`style.node-labels.size`	`length`	`style.label-text.size`	Default node-label text size.
`style.node-labels.color`	`color`	`style.label-text.color`	Default node-label text color.
`style.node-labels.font`	`str` / `array`	inherits	Typst text font selection.
`style.node-labels.weight`	`str` / `int`	inherits	Typst text weight.
`style.node-labels.rotation`	`angle`	`style.label-text.rotation`	Default rotation of the label text itself.

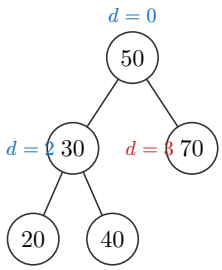
Field	Call	Effect
`.diagram`	n/a	The rendered tree.
`.insert`	`(obj.insert)(key)`	Insert `key` . Returns a step (Section 6).
`.delete`	`(obj.delete)(key)`	Delete `key` . AVL deletes rebalance when removal makes a subtree too heavy. Returns a step.
`.search`	`(obj.search)(key)`	Highlight the traversal path to `key` . Returns a step.

```typst
#bst(50, 30, 70, 20, 40, 60, 80).diagram
#avl(10, 20, 30, 40, 50, 25).diagram

#bst(
  50, 30, 70, 20, 40,
  node-labels: (
    (50, (content: [$d=0$], position: "top")),
    (30, [$d=2$]),
    (70, (content: [$d=3$], color: rgb("#C92A2A"))),
  ),
  style: (node-labels: (position: "left", color: rgb("#1971C2"))),
).diagram
```



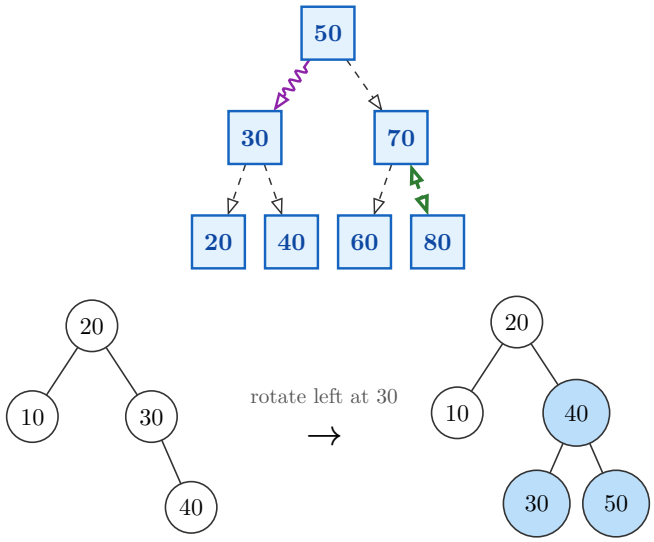




Argument coverage: this example combines tree-level styling, nested `node-text` , edge arrows/patterns, per-edge customizations, and diff-highlight dictionaries.

Typst

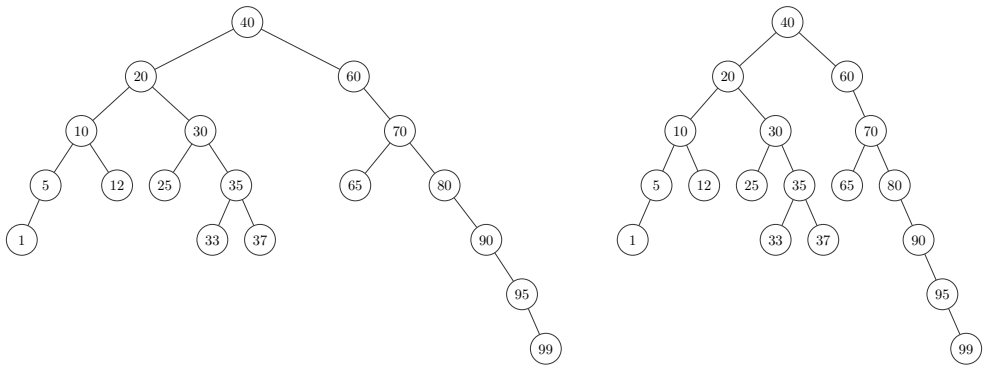
```
#std.stack(spacing: 1em,
  bst(
    50, 30, 70, 20, 40, 60, 80,
    style: (
      node-shape: "square",
      node-radius: 0.38,
      node-fill: rgb("#E3F2FD"),
      node-stroke: 1pt + rgb("#1565C0"),
      node-text: (size: 10pt, weight: "bold", color: rgb("#0D47A1")),
      edge-arrow: "end",
      edge-pattern: "dashed",
      scale: 0.92,
      y-gap : 1.5,
    ),
    edge-customizations: (
      (50, 30, (pattern: "wavy", color: rgb("#8E24AA"))),
      (70, 80, (stroke: 1.4pt + rgb("#2E7D32"), arrow: "both")),
    ),
  ).diagram,
  transition(
    "avl",
    (20, 10, 30, 40),
    tree-insert(50),
    style: (
      new-style: (shape: "square", stroke: 1.5pt + rgb("#2E7D32")),
      path-style: (stroke: 1.2pt + rgb("#F9A825")),
      rotate-style: (node-radius: 0.44),
    ),
  ),
)
```



Tidy tree layout

By default every leaf receives its own column, which keeps trees predictable but can leave wide gaps between uneven subtrees. Set `style: (tree-layout: "tidy")` to pack sibling subtrees against each other instead: each subtree is placed as close to its left neighbour as its outline allows. Binary left/right placement, parent centering, child order, and the space taken by `subtree()` triangles are preserved. The option applies to `bst` , `avl` , `tree` , heaps, and tree transitions, and `"columns"` remains the default.

```typst
#let keys = (40, 20, 10, 5, 30, 60, 70, 65, 80, 1, 12, 25, 35, 33, 37, 90, 95, 99)
#std.stack(dir: ltr, spacing: 2em,
  bst(..keys, style: (scale: 0.6)).diagram,
  bst(..keys, style: (scale: 0.6, tree-layout: "tidy")).diagram,
)
```



3.2 `min-heap()` and `max-heap()`

Full signatures:

```typst
#min-heap(..keys, style: (:), language: "en", messages: (:))
#max-heap(..keys, style: (:), language: "en", messages: (:))
```

A heap is an array underneath. Index `i`'s children live at `2i+1` and `2i+2`. Each key sifts up as it's inserted, and the diagram draws the complete binary tree that array shape forms. `min-heap` keeps the smallest key at the root; `max-heap` keeps the largest.

Argument	Type	Default	Description
`..keys`	`int` / `content`	(required)	Keys inserted into the heap in order, one sift-up per key.
`style`	`dictionary`	`(:)`	Per-call style override merged over the defaults. See Section 8.

Nested keys for `style`

Path	Type	Default	Description
`style.node-radius`	`float`	`0.34`	Circle radius, or base size for other node shapes, in canvas units.
`style.node-shape`	`str`	`"circle"`	`"circle"` , `"square"` , `"rounded"` , `"capsule"` , `"diamond"` , or `"hexagon"` .

`style.x-gap`	`float`	`1.05`	Horizontal spacing between in-order columns.
`style.y-gap`	`float`	`1.2`	Vertical spacing between depths.
`style.tree-layout`	`str`	`"columns"`	`"columns"` gives every leaf its own column. `"tidy"` packs sibling subtrees against their contours, keeping parents centered and subtree triangles clear. See Section 3.1.1.
`style.node-stroke`	`stroke`	`0.6pt + rgb("#333333")`	Node outline.
`style.node-fill`	`color`	`white`	Default node fill.
`style.edge-stroke`	`stroke`	`0.6pt + rgb("#333333")`	Parent-child edge stroke.
`style.edge-arrow`	`none` / `bool` / `str`	`none`	Edge arrowheads: `none` / `false` , `"end"` or `true` , `"start"` , or `"both"` .
`style.edge-pattern`	`str`	`"normal"`	`"normal"` , `"dashed"` , `"dotted"` , or `"wavy"` .
`style.edge-wave-amplitude`	`float`	`0.07`	Wave height in canvas units.
`style.edge-wave-step`	`float`	`0.14`	Approximate wavelength in canvas units.
`style.node-text`	`dictionary`	`(size: 9pt)`	Node text dictionary. See nested keys below.
`style.value-text`	`dictionary`	inherits `node-text`	Value text overrides.
`style.label-text`	`dictionary`	`(fill: rgb("#555555"))`	Annotation text dictionary. See nested keys below.
`style.edge-label-text`	`dictionary`	inherits `label-text`	Edge-label typography.
`style.operation-text`	`dictionary`	`(size: 8pt)`	Operation-arrow caption typography.
`style.scale`	`float`	`1.0`	Uniform scale on `.diagram` , `.before` , and `.after` .
`style.diff-colors`	`bool`	`true`	`false` keeps operation marks but uses normal fills.

Nested keys for `style.node-text`

Path	Type	Default	Description
`style.node-text.size`	`length`	inherits	Text size.
`style.node-text.color`	`color`	inherits	Text color.
`style.node-text.rotation`	`angle`	`0deg`	Text rotation applied by the diagram renderer.
`style.node-text.font`	`str` / `array`	inherits	Typst text font selection.
`style.node-text.weight`	`str` / `int`	inherits	Typst text weight.

Nested keys for `style.label-text`

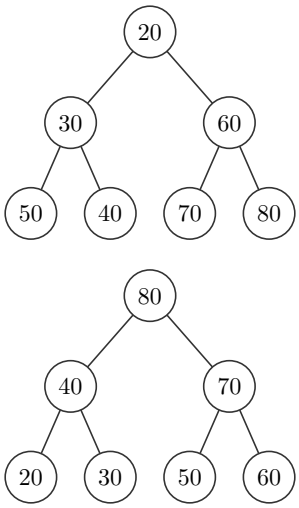
Path	Type	Default	Description
`style.label-text.size`	`length`	85% of `style.node-text.size`	Label text size.
`style.label-text.color`	`color`	`rgb("#555555")`	Label text color.
`style.label-text.rotation`	`angle` / `str`	`0deg`	Label rotation. Tree and graph edge labels also accept `"edge"` to follow the edge angle.
`style.label-text.font`	`str` / `array`	inherits	Typst text font selection.
`style.label-text.weight`	`str` / `int`	inherits	Typst text weight.

Nested keys for diff-highlight dictionaries

Path	Type	Default	Description
`style.new-style`	`color` / `dictionary`	`rgb("#C8E6C9")`	Node/cell added by an operation. A color is shorthand for `(fill: color)` .
`style.path-style`	`color` / `dictionary`	`rgb("#FFE9A8")`	Traversal or sift path.
`style.remove-style`	`color` / `dictionary`	`rgb("#FFCDD2")`	Node/cell removed by an operation.
`style.rotate-style`	`color` / `dictionary`	`rgb("#BBDEFB")`	AVL rotation nodes and visible subtree roots.
`style.*-style.fill`	`color`	mark default	Fill for the marked node/cell.
`style.*-style.shape`	`str`	`style.node-shape`	Marked tree/heap node shape: `"circle"` , `"square"` , `"rounded"` , `"capsule"` , `"diamond"` , or `"hexagon"` .
`style.*-style.stroke`	`stroke`	normal stroke	Marked node/cell outline.
`style.*-style.node-radius`	`float`	`style.node-radius`	Marked tree/heap node radius.
`style.*-style.text`	`dictionary`	`style.node-text`	Marked node/cell text overrides. Accepts `size` , `color` , `font` , and `weight` .

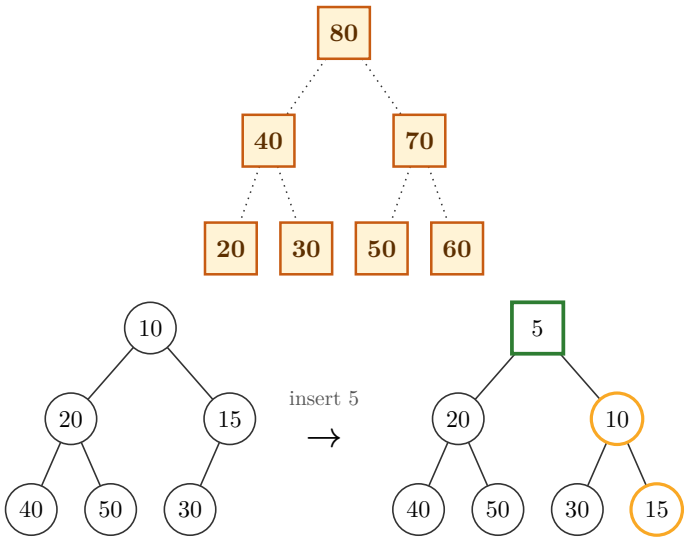
Field	Call	Effect
`.diagram`	n/a	The rendered heap.
`.insert`	`(obj.insert)(key)`	Append `key` and sift it up. Returns a step.
`.extract`	`(obj.extract)()`	Remove the root (no key), move the last leaf up, and sift it down. Returns a step.

```typst
#min-heap(50, 30, 70, 20, 40, 60, 80).diagram
#max-heap(50, 30, 70, 20, 40, 60, 80).diagram
```



Argument coverage: heaps reuse tree styling, and heap operations use the same diff-highlight dictionaries.

```typst
#std.stack(spacing: 1em,
  max-heap(
    50, 30, 70, 20, 40, 60, 80,
    style: (
      node-shape: "square",
      node-radius: 0.36,
      y-gap: 1.5,
      node-fill: rgb("#FFF3D6"),
      node-stroke: 1pt + rgb("#C75B12"),
      node-text: (size: 10pt, color: rgb("#5D2F00"), weight: "bold"),
      edge-pattern: "dotted",
      scale: 0.95,
    ),
  ).diagram,
  transition(
    "min-heap",
    (10, 20, 15, 40, 50, 30),
    heap-insert(5),
    style: (
      diff-colors: false,
      new-style: (shape: "square", stroke: 1.4pt + rgb("#2E7D32")),
      path-style: (stroke: 1.2pt + rgb("#F9A825")),
    ),
  ),
)
```



Note. A binary heap supports inserting and pulling out the root efficiently. It has no key-based `delete` or `search` the way `bst` / `avl` do: that's not a gap in typed-dsa, it's what a heap is.

3.3 `linked-list()`

Full signature:

```typst
#linked-list(
  ..vals,
  style: (:),
  pointer: false,
  addresses: none,
  head: false,
  language: "en",
  messages: (:),
)
```

Argument	Type	Default	Description
`..vals`	`int` / `content`	(required)	Values in the list, head first.
`style`	`dictionary`	`(:)`	Per-call style override merged over the defaults. See Section 8.
`pointer`	`bool`	`false`	Draw each node as a `data | next` cell pair instead of a single value cell.
`addresses`	`none` / `array`	`none`	With `pointer: true` , one address label drawn under each node, in order.
`head`	`bool`	`false`	Draw a “Head” arrow pointing at the first node.

Nested keys for `style`

Path	Type	Default	Description
`style.box-w`	`float`	`0.95`	Cell width.
`style.box-h`	`float`	`0.7`	Cell height.
`style.box-shape`	`str`	`"square"`	`"square"` , `"rounded"` , or `"capsule"` .
`style.box-gap`	`float`	`0.55`	Gap between cells or list nodes.
`style.box-stroke`	`stroke`	`0.6pt + rgb("#333333")`	Cell outline.
`style.box-fill`	`color`	`white`	Default cell fill.
`style.ptr-fill`	`color`	`rgb("#D7ECC9")`	Next-pointer cell fill for `linked-list(pointer: true)` .
`style.prev-ptr-fill`	`color`	`rgb("#D7ECC9")`	Previous-pointer cell fill for `doubly-linked-list(pointer: true)` .
`style.next-ptr-fill`	`color`	`rgb("#D7ECC9")`	Next-pointer cell fill for `doubly-linked-list(pointer: true)` .
`style.node-text`	`dictionary`	`(size: 9pt)`	Cell text dictionary. See nested keys below.
`style.value-text`	`dictionary`	inherits `node-text`	Value text overrides.

`style.label-text`	`dictionary`	`(fill: rgb("#555555"))`	Head, address, front, and rear label text dictionary. See nested keys below.
`style.pointer-text`	`dictionary`	inherits `label-text`	Pointer, head, address, front, and rear typography.
`style.operation-text`	`dictionary`	`(size: 8pt)`	Operation-arrow caption typography.
`style.scale`	`float`	`1.0`	Uniform scale on `.diagram` , `.before` , and `.after` .
`style.diff-colors`	`bool`	`true`	`false` keeps operation marks but uses normal fills.

Nested keys for `style.node-text`

Path	Type	Default	Description
`style.node-text.size`	`length`	inherits	Text size.
`style.node-text.color`	`color`	inherits	Text color.
`style.node-text.rotation`	`angle`	`0deg`	Text rotation applied by the diagram renderer.
`style.node-text.font`	`str / array`	inherits	Typst text font selection.
`style.node-text.weight`	`str / int`	inherits	Typst text weight.

Nested keys for `style.label-text`

Path	Type	Default	Description
`style.label-text.size`	`length`	85% of `style.node-text.size`	Label text size.
`style.label-text.color`	`color`	`rgb("#555555")`	Label text color.
`style.label-text.rotation`	`angle / str`	`0deg`	Label rotation. Tree and graph edge labels also accept `"edge"` to follow the edge angle.
`style.label-text.font`	`str / array`	inherits	Typst text font selection.
`style.label-text.weight`	`str / int`	inherits	Typst text weight.

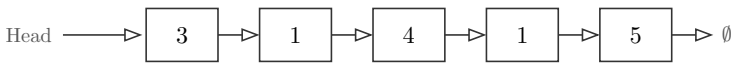
Nested keys for diff-highlight dictionaries

Path	Type	Default	Description
`style.new-style`	`color / dictionary`	`rgb("#C8E6C9")`	Node/cell added by an operation. A color is shorthand for `(fill: color)` .
`style.path-style`	`color / dictionary`	`rgb("#FFE9A8")`	Traversal or sift path.
`style.remove-style`	`color / dictionary`	`rgb("#FFCDD2")`	Node/cell removed by an operation.
`style.rotate-style`	`color / dictionary`	`rgb("#BBDEFB")`	AVL rotation nodes and visible subtree roots.

`style.*-style.fill`	`color`	mark default	Fill for the marked node/cell.
`style.*-style.shape`	`str`	`style.node-shape`	Marked tree/heap node shape: `"circle"` , `"square"` , `"rounded"` , `"capsule"` , `"diamond"` , or `"hexagon"` .
`style.*-style.stroke`	`stroke`	normal stroke	Marked node/cell outline.
`style.*-style.node-radius`	`float`	`style.node-radius`	Marked tree/heap node radius.
`style.*-style.text`	`dictionary`	`style.node-text`	Marked node/cell text overrides. Accepts `size` , `color` , `font` , and `weight` .

Field	Call	Effect
`.diagram`	n/a	The rendered list.
`.prepend`	`(obj.prepend)(v)`	Insert `v` at the head. Returns a step.
`.insert`	`(obj.insert)(v, index: none)`	Append `v` by default, or insert at an index from `0` through the current length.
`.delete`	`(obj.delete)(v)`	Remove the first node equal to `v` . Returns a step.
`.delete-at`	`(obj.delete-at)(index)`	Remove the node at `index` . Returns a step.
`.search`	`(obj.search)(v)`	Highlight the traversed prefix. The step also exposes `.found` and `.index` .

```
1  #linked-list(3, 1, 4, 1, 5, head: true).diagram
```
Typst



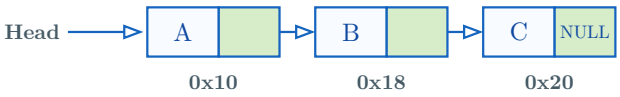

```
1  #linked-list(15, 3, 17, 90, pointer: true, head: true,
2    addresses: ("3200", "3600", "4000", "4400")).diagram
```
Typst



Argument coverage: `pointer` , `addresses` , `head` , label text, pointer-cell fills, scaling, and operation mark styling.

Typst

```
#let xs = linked-list(
  [A], [B], [C],
  pointer: true,
  head: true,
  addresses: ("0x10", "0x18", "0x20"),
  style: (
    box-w: 0.9,
    box-h: 0.62,
    box-gap: 0.45,
    box-fill: rgb("#F8FBFF"),
    box-stroke: 0.8pt + rgb("#1565C0"),
    ptr-fill: rgb("#D7ECC9"),
    node-text: (size: 9pt, color: rgb("#0D47A1")),
    label-text: (size: 7.5pt, color: rgb("#455A64"), weight: "bold"),
    scale: 1.05,
    new-style: (fill: rgb("#C8E6C9"), stroke: 1pt + rgb("#2E7D32")),
    remove-style: (fill: rgb("#FFCDD2"), stroke: 1pt + rgb("#C62828")),
  ),
)
#xs.diagram
```



3.4 `doubly-linked-list()`

Full signature:

Typst

```
#doubly-linked-list(
  ..vals,
  style: (:),
  pointer: false,
  addresses: none,
  head: false,
  language: "en",
  messages: (:),
)
```

Argument	Type	Default	Description
`..vals`	`int` / `content`	(required)	Values in the list, head first.
`style`	`dictionary`	`(:)`	Per-call style override merged over the defaults. See Section 8.

`pointer`	`bool`	`false`	When `false` , draw one value cell per node with a forward arrow above and a backward arrow below. When `true` , draw `prev | data | next` cells.
`addresses`	`none` / `array`	`none`	With `pointer: true` , one address label drawn under each node, in order.
`head`	`bool`	`false`	Draw a “Head” arrow pointing at the first node.

Nested keys for `style`

Path	Type	Default	Description
`style.box-w`	`float`	`0.95`	Cell width.
`style.box-h`	`float`	`0.7`	Cell height.
`style.box-shape`	`str`	`"square"`	`"square"` , `"rounded"` , or `"capsule"` .
`style.box-gap`	`float`	`0.55`	Gap between cells or list nodes.
`style.box-stroke`	`stroke`	`0.6pt + rgb("#333333")`	Cell outline.
`style.box-fill`	`color`	`white`	Default cell fill.
`style.ptr-fill`	`color`	`rgb("#D7ECC9")`	Next-pointer cell fill for `linked-list(pointer: true)` .
`style.prev-ptr-fill`	`color`	`rgb("#D7ECC9")`	Previous-pointer cell fill for `doubly-linked-list(pointer: true)` .
`style.next-ptr-fill`	`color`	`rgb("#D7ECC9")`	Next-pointer cell fill for `doubly-linked-list(pointer: true)` .
`style.node-text`	`dictionary`	`(size: 9pt)`	Cell text dictionary. See nested keys below.
`style.value-text`	`dictionary`	inherits `node-text`	Value text overrides.
`style.label-text`	`dictionary`	`(fill: rgb("#555555"))`	Head, address, front, and rear label text dictionary. See nested keys below.
`style.pointer-text`	`dictionary`	inherits `label-text`	Pointer, head, address, front, and rear typography.
`style.operation-text`	`dictionary`	`(size: 8pt)`	Operation-arrow caption typography.
`style.scale`	`float`	`1.0`	Uniform scale on `.diagram` , `.before` , and `.after` .
`style.diff-colors`	`bool`	`true`	`false` keeps operation marks but uses normal fills.

Nested keys for `style.node-text`

Path	Type	Default	Description
`style.node-text.size`	`length`	inherits	Text size.
`style.node-text.color`	`color`	inherits	Text color.

`style.node-text.rotation`	`angle`	`0deg`	Text rotation applied by the diagram renderer.
`style.node-text.font`	`str / array`	inherits	Typst text font selection.
`style.node-text.weight`	`str / int`	inherits	Typst text weight.

Nested keys for `style.label-text`

Path	Type	Default	Description
`style.label-text.size`	`length`	85% of `style.node-text.size`	Label text size.
`style.label-text.color`	`color`	`rgb("#555555")`	Label text color.
`style.label-text.rotation`	`angle / str`	`0deg`	Label rotation. Tree and graph edge labels also accept `"edge"` to follow the edge angle.
`style.label-text.font`	`str / array`	inherits	Typst text font selection.
`style.label-text.weight`	`str / int`	inherits	Typst text weight.

Nested keys for diff-highlight dictionaries

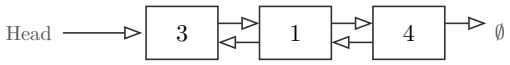
Path	Type	Default	Description
`style.new-style`	`color / dictionary`	`rgb("#C8E6C9")`	Node/cell added by an operation. A color is shorthand for `(fill: color)` .
`style.path-style`	`color / dictionary`	`rgb("#FFE9A8")`	Traversal or sift path.
`style.remove-style`	`color / dictionary`	`rgb("#FFCDD2")`	Node/cell removed by an operation.
`style.rotate-style`	`color / dictionary`	`rgb("#BBDEFB")`	AVL rotation nodes and visible subtree roots.
`style.*-style.fill`	`color`	mark default	Fill for the marked node/cell.
`style.*-style.shape`	`str`	`style.node-shape`	Marked tree/heap node shape: `"circle"` , `"square"` , `"rounded"` , `"capsule"` , `"diamond"` , or `"hexagon"` .
`style.*-style.stroke`	`stroke`	normal stroke	Marked node/cell outline.
`style.*-style.node-radius`	`float`	`style.node-radius`	Marked tree/heap node radius.
`style.*-style.text`	`dictionary`	`style.node-text`	Marked node/cell text overrides. Accepts `size` , `color` , `font` , and `weight` .

Field	Call	Effect
`.diagram`	n/a	The rendered list.
`.prepend`	`(obj.prepend)(v)`	Insert `v` at the head. Returns a step.
`.insert`	`(obj.insert)(v, index: none)`	Append by default, or insert `v` at an index.
`.delete`	`(obj.delete)(v)`	Remove the first node equal to `v` . Returns a step.

`.delete-at`	`(obj.delete-at)(index)`	Remove the node at `index` . Returns a step.
`.search`	`(obj.search)(v)`	Highlight the traversed prefix and expose `.found` / `.index` .

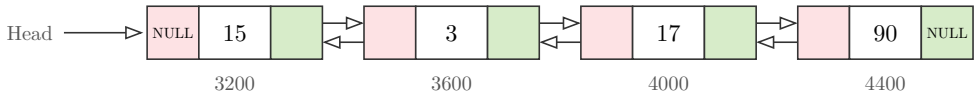
```
#doubly-linked-list(3, 1, 4,
head: true).diagram
```

Typst




```
#doubly-linked-list(15, 3, 17, 90, pointer: true, head: true,
  addresses: ("3200", "3600", "4000", "4400"),
  style: (
    prev-ptr-fill: rgb("#FDE2E4"),
    next-ptr-fill: rgb("#D7ECC9"),
  )).diagram
```

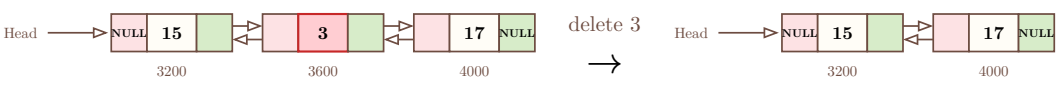
Typst



Argument coverage: doubly linked pointer cells have separate previous and next pointer fills, plus the same address, head, label, and mark styling as `linked-list` .

```
#let xs = doubly-linked-list(
  15, 3, 17,
  pointer: true,
  head: true,
  addresses: ("3200", "3600", "4000"),
  style: (
    box-w: 0.82,
    box-h: 0.62,
    box-gap: 0.5,
    box-fill: rgb("#FFFDF7"),
    box-stroke: 0.8pt + rgb("#6D4C41"),
    prev-ptr-fill: rgb("#FDE2E4"),
    next-ptr-fill: rgb("#D7ECC9"),
    node-text: (size: 8.5pt, weight: "bold"),
    label-text: (size: 7pt, color: rgb("#6D4C41")),
    path-style: (fill: rgb("#FFE9A8"), stroke: 1pt + rgb("#F9A825")),
    remove-style: (fill: rgb("#FFCDD2"), stroke: 1pt + rgb("#C62828")),
    scale : 0.8,
  ),
)
#(xs.delete)(3).diagram
```

Typst



3.5 `stack()`

Full signature:

```typst
#stack(..vals, style: (:), top-label: auto, language: "en", messages: (:))
```

Argument	Type	Default	Description
`..vals`	`int` / `content`	(required)	Values in the stack; the **first** argument is the top.
`style`	`dictionary`	`(:)`	Per-call style override merged over the defaults. See Section 8.
`top-label`	`content` / `auto`	`auto`	Label beside the top cell. `auto` uses the localized `stack.top` default (Section 7).

Nested keys for `style`

Path	Type	Default	Description
`style.box-w`	`float`	`0.95`	Cell width.
`style.box-h`	`float`	`0.7`	Cell height.
`style.box-shape`	`str`	`"square"`	`"square"` , `"rounded"` , or `"capsule"` .
`style.box-gap`	`float`	`0`	Gap between cells or list nodes.
`style.box-stroke`	`stroke`	`0.6pt + rgb("#333333")`	Cell outline.
`style.box-fill`	`color`	`white`	Default cell fill.
`style.ptr-fill`	`color`	`rgb("#D7ECC9")`	Next-pointer cell fill for `linked-list(pointer: true)` .
`style.prev-ptr-fill`	`color`	`rgb("#D7ECC9")`	Previous-pointer cell fill for `doubly-linked-list(pointer: true)` .
`style.next-ptr-fill`	`color`	`rgb("#D7ECC9")`	Next-pointer cell fill for `doubly-linked-list(pointer: true)` .
`style.node-text`	`dictionary`	`(size: 9pt)`	Cell text dictionary. See nested keys below.
`style.value-text`	`dictionary`	inherits `node-text`	Value text overrides.
`style.label-text`	`dictionary`	`(fill: rgb("#555555"))`	Head, address, front, and rear label text dictionary. See nested keys below.
`style.pointer-text`	`dictionary`	inherits `label-text`	Pointer, head, address, front, and rear typography.
`style.operation-text`	`dictionary`	`(size: 8pt)`	Operation-arrow caption typography.
`style.scale`	`float`	`1.0`	Uniform scale on `.diagram` , `.before` , and `.after` .
`style.diff-colors`	`bool`	`true`	`false` keeps operation marks but uses normal fills.

Nested keys for `style.node-text`

Path	Type	Default	Description
style.node-text.size	length	inherits	Text size.
style.node-text.color	color	inherits	Text color.
style.node-text.rotation	angle	0deg	Text rotation applied by the diagram renderer.
style.node-text.font	str / array	inherits	Typst text font selection.
style.node-text.weight	str / int	inherits	Typst text weight.

Nested keys for `style.label-text`

Path	Type	Default	Description
style.label-text.size	length	85% of style.node-text.size	Label text size.
style.label-text.color	color	rgb("#555555")	Label text color.
style.label-text.rotation	angle / str	0deg	Label rotation. Tree and graph edge labels also accept "edge" to follow the edge angle.
style.label-text.font	str / array	inherits	Typst text font selection.
style.label-text.weight	str / int	inherits	Typst text weight.

Nested keys for diff-highlight dictionaries

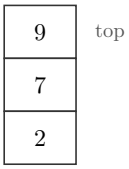
Path	Type	Default	Description
style.new-style	color / dictionary	rgb("#C8E6C9")	Node/cell added by an operation. A color is shorthand for (fill: color) .
style.path-style	color / dictionary	rgb("#FFE9A8")	Traversal or sift path.
style.remove-style	color / dictionary	rgb("#FFCDD2")	Node/cell removed by an operation.
style.rotate-style	color / dictionary	rgb("#BBDEFB")	AVL rotation nodes and visible subtree roots.
style.*-style.fill	color	mark default	Fill for the marked node/cell.
style.*-style.shape	str	style.node-shape	Marked tree/heap node shape: "circle" , "square" , "rounded" , "capsule" , "diamond" , or "hexagon" .
style.*-style.stroke	stroke	normal stroke	Marked node/cell outline.
style.*-style.node-radius	float	style.node-radius	Marked tree/heap node radius.
style.*-style.text	dictionary	style.node-text	Marked node/cell text overrides. Accepts size , color , font , and weight .

Field	Call	Effect
.diagram	n/a	The rendered stack.

`.push`	`(obj.push)(v)`	Push `v` onto the top. Returns a step.
`.pop`	`(obj.pop)()`	Pop the top value (no argument). Returns a step.

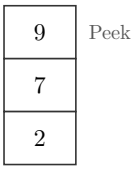
```
#stack(9, 7, 2).diagram
```

Typst



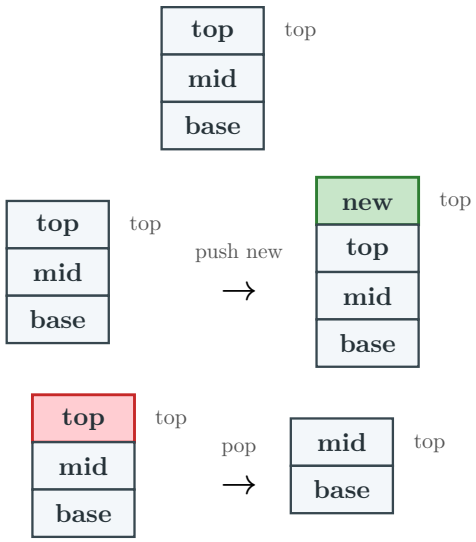

```
#stack(9, 7, 2, top-label:
[Peek]).diagram
```

Typst



Argument coverage: stacks use the linear `style` keys and operation marks for `.push` / `.pop` .

```typst
#let s = stack(
  [top], [mid], [base],
  style: (
    box-w: 1.25,
    box-h: 0.58,
    box-gap: 0,
    box-fill: rgb("#F3F7FA"),
    box-stroke: 0.8pt + rgb("#37474F"),
    node-text: (size: 9pt, color: rgb("#263238"), weight: "bold"),
    scale: 1.08,
    new-style: (fill: rgb("#C8E6C9"), stroke: 1pt + rgb("#2E7D32")),
    remove-style: (fill: rgb("#FFCDD2"), stroke: 1pt + rgb("#C62828")),
  ),
)
#std.stack(spacing: 1em, s.diagram, (s.push)([new]).diagram, (s.pop)().diagram)
```



3.6 `queue()`

Full signature:

```typst
#queue(
  ..vals,
  style: (:),
  enqueue: none,
  dequeue: none,
  front-label: auto,
  rear-label: auto,
  language: "en",
  messages: (:),
)
```

Argument	Type	Default	Description
`..vals`	`int` / `content`	(required)	Values in the queue; the **first** argument is the front.
`style`	`dictionary`	`(:)`	Per-call style override merged over the defaults. See Section 8.
`enqueue`	`none` / `int` / `content`	`none`	Draw one extra element entering at the rear, with an arrow, in the same frame.
`dequeue`	`none` / `int` / `content`	`none`	Draw the front element leaving to the left, with an arrow, in the same frame.
`front-label`	`content` / `auto`	`auto`	Label over the front cell; `auto` uses the localized `queue.front` default (Section 7). Combined with `rear-label` for a one-cell queue.
`rear-label`	`content` / `auto`	`auto`	Label over the rear cell; `auto` uses the localized `queue.rear` default (Section 7). Combined with `front-label` for a one-cell queue.

Nested keys for `style`

Path	Type	Default	Description
`style.box-w`	`float`	`0.95`	Cell width.
`style.box-h`	`float`	`0.7`	Cell height.
`style.box-shape`	`str`	`"square"`	`"square"` , `"rounded"` , or `"capsule"` .
`style.box-gap`	`float`	`0.55`	Gap between cells or list nodes.
`style.box-stroke`	`stroke`	`0.6pt + rgb("#333333")`	Cell outline.
`style.box-fill`	`color`	`white`	Default cell fill.
`style.ptr-fill`	`color`	`rgb("#D7ECC9")`	Next-pointer cell fill for `linked-list(pointer: true)` .
`style.prev-ptr-fill`	`color`	`rgb("#D7ECC9")`	Previous-pointer cell fill for `doubly-linked-list(pointer: true)` .
`style.next-ptr-fill`	`color`	`rgb("#D7ECC9")`	Next-pointer cell fill for `doubly-linked-list(pointer: true)` .
`style.node-text`	`dictionary`	`(size: 9pt)`	Cell text dictionary. See nested keys below.
`style.value-text`	`dictionary`	inherits `node-text`	Value text overrides.
`style.label-text`	`dictionary`	`(fill: rgb("#555555"))`	Head, address, front, and rear label text dictionary. See nested keys below.
`style.pointer-text`	`dictionary`	inherits `label-text`	Pointer, head, address, front, and rear typography.
`style.operation-text`	`dictionary`	`(size: 8pt)`	Operation-arrow caption typography.

style.scale	float	1.0	Uniform scale on .diagram , .before , and .after .
style.diff-colors	bool	true	false keeps operation marks but uses normal fills.

Nested keys for style.node-text

Path	Type	Default	Description
style.node-text.size	length	inherits	Text size.
style.node-text.color	color	inherits	Text color.
style.node-text.rotation	angle	0deg	Text rotation applied by the diagram renderer.
style.node-text.font	str / array	inherits	Typst text font selection.
style.node-text.weight	str / int	inherits	Typst text weight.

Nested keys for style.label-text

Path	Type	Default	Description
style.label-text.size	length	85% of style.node-text.size	Label text size.
style.label-text.color	color	rgb("#555555")	Label text color.
style.label-text.rotation	angle / str	0deg	Label rotation. Tree and graph edge labels also accept "edge" to follow the edge angle.
style.label-text.font	str / array	inherits	Typst text font selection.
style.label-text.weight	str / int	inherits	Typst text weight.

Nested keys for diff-highlight dictionaries

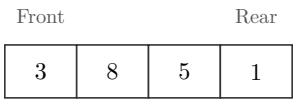
Path	Type	Default	Description
style.new-style	color / dictionary	rgb("#C8E6C9")	Node/cell added by an operation. A color is shorthand for (fill: color) .
style.path-style	color / dictionary	rgb("#FFE9A8")	Traversal or sift path.
style.remove-style	color / dictionary	rgb("#FFCDD2")	Node/cell removed by an operation.
style.rotate-style	color / dictionary	rgb("#BBDEFB")	AVL rotation nodes and visible subtree roots.
style.*-style.fill	color	mark default	Fill for the marked node/cell.
style.*-style.shape	str	style.node-shape	Marked tree/heap node shape: "circle" , "square" , "rounded" , "capsule" , "diamond" , or "hexagon" .
style.*-style.stroke	stroke	normal stroke	Marked node/cell outline.
style.*-style.node-radius	float	style.node-radius	Marked tree/heap node radius.

`style.*-style.text`	`dictionary`	`style.node-text`	Marked node/cell text overrides. Accepts `size` , `color` , `font` , and `weight` .

Field	Call	Effect
`.diagram`	n/a	The rendered queue.
`.enqueue`	`(obj.enqueue)(v)`	Enqueue `v` at the rear. Returns a step.
`.dequeue`	`(obj.dequeue)()`	Dequeue the front value (no argument). Returns a step.

```
1  #queue(3, 8, 5, 1).diagram
```

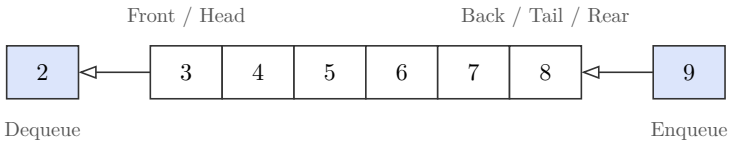
Typst



`enqueue` / `dequeue` draw a single-frame arrow, independent of the object's own `.enqueue` / `.dequeue` methods:

```
1  #queue(3, 4, 5, 6, 7, 8, enqueue: 9, dequeue: 2,
2    front-label: [Front / Head], rear-label: [Back / Tail / Rear]).diagram
```

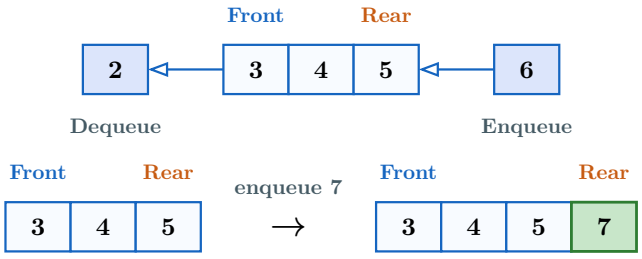
Typst



Argument coverage: queue examples can combine the single-frame `enqueue` / `dequeue` arrows, custom front/rear labels, label text styling, and operation mark styles.

Typst

```
#let q = queue(
  3, 4, 5,
  enqueue: 6,
  dequeue: 2,
  front-label: text(fill: rgb("#1565C0"))[*Front*],
  rear-label: text(fill: rgb("#C75B12"))[*Rear*],
  style: (
    box-w: 0.82,
    box-h: 0.62,
    box-gap: 0.35,
    box-fill: rgb("#F8FBFF"),
    box-stroke: 0.8pt + rgb("#1565C0"),
    node-text: (size: 9pt, weight: "bold"),
    label-text: (size: 7.5pt, color: rgb("#455A64"), weight: "bold"),
    scale: 1.05,
    new-style: (fill: rgb("#C8E6C9"), stroke: 1pt + rgb("#2E7D32")),
    remove-style: (fill: rgb("#FFCDD2"), stroke: 1pt + rgb("#C62828")),
  ),
)
#std.stack(spacing: 1em, q.diagram, (q.enqueue)(7).diagram)
```



3.7 skip-list()

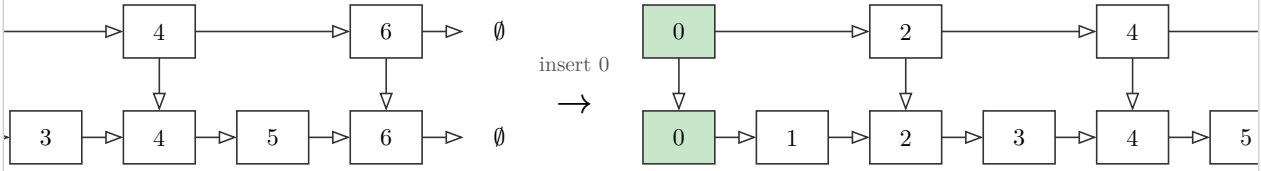
Full signature:

```typst
#skip-list(
  ..vals,
  style: (:),
  decision-fn: default-decision-fn,
  level-spacing: 1.4,
  max-level: 4,
  language: "en",
  messages: (:),
)
```

`..vals` must be strictly ascending, with no duplicate values; `skip-list` validates this but does not sort them for you. Values must be all numbers (integers and floats may be mixed) or all strings. Levels above the base row are express lanes: a node's height is decided once, when it is built or inserted, and never changes afterward, so later inserts/deletes elsewhere never reshuffle it.

The smallest value is always drawn spanning every level, exactly like the sentinel head node in a standard skip list — it's what search starts from at the top level, so it must always have a way in. Its own `decision-fn` -assigned height still exists underneath; it only takes over once `insert` gives it a smaller value to make way for.

```typst
#let l = skip-list(1, 2, 3, 4, 5, 6)
#(l.insert)(0).diagram
```



Argument	Type	Default	Description
`..vals`	`int` / `float` / `str`	(required)	Values in strictly ascending order, without duplicates. All values must be numbers (integers and floats may be mixed) or all strings. `insert` compares values to place them, so arbitrary `content` isn't supported here.
`style`	`dictionary`	`(:)`	Per-call style override merged over the defaults. See Section 8.
`decision-fn`	`function`	`default-decision-fn`	`(level, value) => bool` , called with `level` starting at `1` and increasing; a node keeps climbing while this returns `true` . Typst has no RNG, so the default hashes `value` deterministically instead of flipping a coin — same value, same height, every recompile.

`level-spacing`	`int` / `float`	`1.4`	Positive vertical distance between level rows, in canvas units.
`max-level`	`int`	`4`	Non-negative upper bound on the height `decision-fn` can assign automatically. `insert(..., level:)` can still exceed it when set explicitly.

Nested keys for `style`

Path	Type	Default	Description
`style.box-w`	`float`	`0.95`	Cell width.
`style.box-h`	`float`	`0.7`	Cell height.
`style.box-shape`	`str`	`"square"`	`"square"` , `"rounded"` , or `"capsule"` .
`style.box-gap`	`float`	`0.55`	Gap between cells or list nodes.
`style.box-stroke`	`stroke`	`0.6pt + rgb("#333333")`	Cell outline.
`style.box-fill`	`color`	`white`	Default cell fill.
`style.ptr-fill`	`color`	`rgb("#D7ECC9")`	Next-pointer cell fill for `linked-list(pointer: true)` .
`style.prev-ptr-fill`	`color`	`rgb("#D7ECC9")`	Previous-pointer cell fill for `doubly-linked-list(pointer: true)` .
`style.next-ptr-fill`	`color`	`rgb("#D7ECC9")`	Next-pointer cell fill for `doubly-linked-list(pointer: true)` .
`style.node-text`	`dictionary`	`(size: 9pt)`	Cell text dictionary. See nested keys below.
`style.value-text`	`dictionary`	inherits `node-text`	Value text overrides.
`style.label-text`	`dictionary`	`(fill: rgb("#555555"))`	Head, address, front, and rear label text dictionary. See nested keys below.
`style.pointer-text`	`dictionary`	inherits `label-text`	Pointer, head, address, front, and rear typography.
`style.operation-text`	`dictionary`	`(size: 8pt)`	Operation-arrow caption typography.
`style.scale`	`float`	`1.0`	Uniform scale on `.diagram` , `.before` , and `.after` .
`style.diff-colors`	`bool`	`true`	`false` keeps operation marks but uses normal fills.

Nested keys for `style.node-text`

Path	Type	Default	Description
`style.node-text.size`	`length`	inherits	Text size.
`style.node-text.color`	`color`	inherits	Text color.
`style.node-text.rotation`	`angle`	`0deg`	Text rotation applied by the diagram renderer.

style.node-text.font	str / array	inherits	Typst text font selection.
style.node-text.weight	str / int	inherits	Typst text weight.

Nested keys for style.label-text

Path	Type	Default	Description
style.label-text.size	length	85% of style.node-text.size	Label text size.
style.label-text.color	color	rgb("#555555")	Label text color.
style.label-text.rotation	angle / str	0deg	Label rotation. Tree and graph edge labels also accept "edge" to follow the edge angle.
style.label-text.font	str / array	inherits	Typst text font selection.
style.label-text.weight	str / int	inherits	Typst text weight.

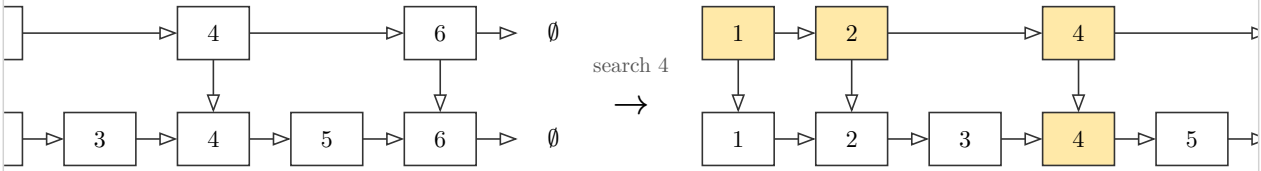
Nested keys for diff-highlight dictionaries

Path	Type	Default	Description
style.new-style	color / dictionary	rgb("#C8E6C9")	Node/cell added by an operation. A color is shorthand for (fill: color) .
style.path-style	color / dictionary	rgb("#FFE9A8")	Traversal or sift path.
style.remove-style	color / dictionary	rgb("#FFCDD2")	Node/cell removed by an operation.
style.rotate-style	color / dictionary	rgb("#BBDEFB")	AVL rotation nodes and visible subtree roots.
style.*-style.fill	color	mark default	Fill for the marked node/cell.
style.*-style.shape	str	style.node-shape	Marked tree/heap node shape: "circle" , "square" , "rounded" , "capsule" , "diamond" , or "hexagon" .
style.*-style.stroke	stroke	normal stroke	Marked node/cell outline.
style.*-style.node-radius	float	style.node-radius	Marked tree/heap node radius.
style.*-style.text	dictionary	style.node-text	Marked node/cell text overrides. Accepts size , color , font , and weight .

Field	Call	Effect
.diagram	n/a	The rendered skip list.
.search	(obj.search)(key)	Highlight the top-down search path to key , or its predecessor when absent. Returns a step with found and index (false / none for a miss).
.insert	(obj.insert)(value, level: auto)	Insert a new value in sorted position. The value must be compatible with the existing values and not already present. level: auto assigns height with decision-fn an

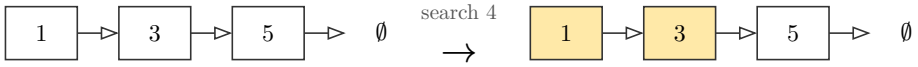
		explicit non-negative integer forces a specific tower height instead. Returns a step.
`.delete`	`(obj.delete)(value)`	Remove `value` from every level it spans at once. The value must be present. Returns a step.

```typst
#let l = skip-list(1, 2, 3, 4, 5, 6)
#(l.search)(4).diagram
```



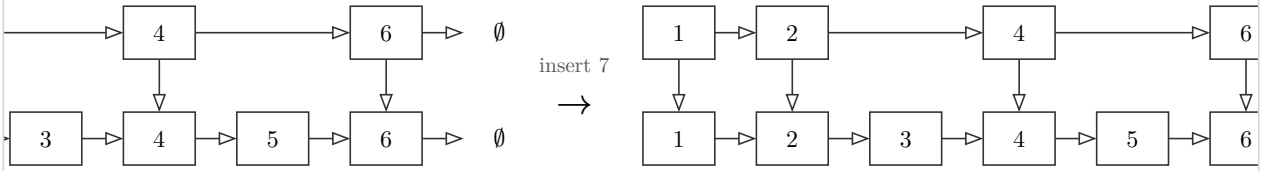
A search miss is still a valid operation: it follows the express lanes to the predecessor where the key would be inserted. The returned step exposes `found` and `index` so the miss can be handled programmatically.

```typst
#let miss = (skip-list(1, 3, 5).search)(4)
#assert(not miss.found)
#assert.eq(miss.index, none)
#miss.diagram
```



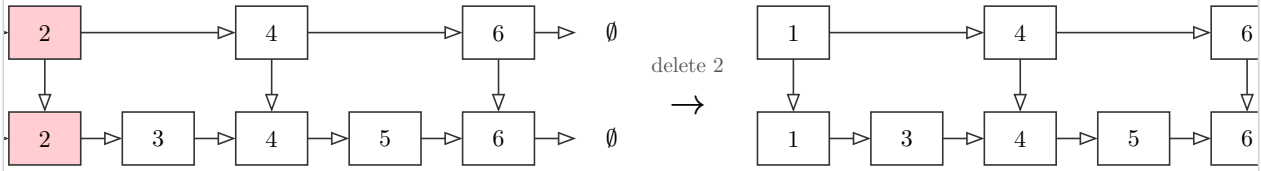
Insert only marks the node that was actually added (green); every other node's height stays exactly as it was, since `decision-fn` depends on the value alone, never on how many other values exist or where they sit.

```typst
#let l = skip-list(1, 2, 3, 4, 5, 6)
#(l.insert)(7).diagram
```



Delete highlights the target (red) across every level it appears at, then removes it from all of them in the same step — a node is one physical tower, not several independent copies, so leaving it linked at only some levels would corrupt the structure.

```typst
#let l = skip-list(1, 2, 3, 4, 5, 6)
#(l.delete)(2).diagram
```



3.8 `array-view()` and `matrix()`

Full signatures:

```typst
#array-view(..vals, style: (:), cell-customizations: (), pointers: ())
#matrix(rows, style: (:), cell-customizations: (), row-labels: none, column-labels: none)
```

These builders draw compact cell diagrams. They do not have operations yet; `cell-customizations` restyles individual array cells by index or matrix cells by `(row, column)` coordinate. `style.indices: (enabled: true)` draws zero-based array indices under the cells. `pointers` draws labelled arrows above array cells.

Argument	Type	Default	Description
`..vals`	content	(required for `array-view`)	Array cells, left to right.
`rows`	array	(required for `matrix`)	Rows of cells, written as nested arrays such as `((1, 2), (3, 4))`.
`cell-customizations`	array / dictionary	`()`	Cell overrides. For arrays, use `((index, options), ...)`. For matrices, use `(((row, col), options), ...)`.
`pointers`	array	`()`	`array-view` only. Labelled arrows above cells, each `(index: , label: , color: , text: )`. Markers sharing a cell spread across its width.
`row-labels`	none / array	none	Matrix only. Labels drawn to the left of rows.
`column-labels`	none / array	none	Matrix only. Labels drawn above columns.
`style`	dictionary	`(:)`	Cell style override. Uses `box-*`, `node-text`, `label-text`, and diff-highlight style keys.

Nested keys for `style`

Path	Type	Default	Description
`style.box-w`	float	0.95	Cell width.

style.box-h	float	0.7	Cell height.
style.box-shape	str	"square"	"square" , "rounded" , or "capsule" .
style.box-gap	float	0.55	Gap between cells or list nodes.
style.box-stroke	stroke	0.6pt + rgb("#333333")	Cell outline.
style.box-fill	color	white	Default cell fill.
style.ptr-fill	color	rgb("#D7ECC9")	Next-pointer cell fill for linked-list(pointer: true) .
style.prev-ptr-fill	color	rgb("#D7ECC9")	Previous-pointer cell fill for doubly-linked-list(pointer: true) .
style.next-ptr-fill	color	rgb("#D7ECC9")	Next-pointer cell fill for doubly-linked-list(pointer: true) .
style.node-text	dictionary	(size: 9pt)	Cell text dictionary. See nested keys below.
style.value-text	dictionary	inherits node-text	Value text overrides.
style.label-text	dictionary	(fill: rgb("#555555"))	Head, address, front, and rear label text dictionary. See nested keys below.
style.pointer-text	dictionary	inherits label-text	Pointer, head, address, front, and rear typography.
style.operation-text	dictionary	(size: 8pt)	Operation-arrow caption typography.
style.scale	float	1.0	Uniform scale on .diagram , .before , and .after .
style.diff-colors	bool	true	false keeps operation marks but uses normal fills.

Nested keys for `style.node-text`

Path	Type	Default	Description
style.node-text.size	length	inherits	Text size.
style.node-text.color	color	inherits	Text color.
style.node-text.rotation	angle	0deg	Text rotation applied by the diagram renderer.
style.node-text.font	str / array	inherits	Typst text font selection.
style.node-text.weight	str / int	inherits	Typst text weight.

Nested keys for `style.label-text`

Path	Type	Default	Description
style.label-text.size	length	85% of style.node-text.size	Label text size.
style.label-text.color	color	rgb("#555555")	Label text color.

`style.label-text.rotation`	`angle` / `str`	`0deg`	Label rotation. Tree and graph edge labels also accept `"edge"` to follow the edge angle.
`style.label-text.font`	`str` / `array`	inherits	Typst text font selection.
`style.label-text.weight`	`str` / `int`	inherits	Typst text weight.

Nested keys for diff-highlight dictionaries

Path	Type	Default	Description
`style.new-style`	`color` / `dictionary`	`rgb("#C8E6C9")`	Node/cell added by an operation. A color is shorthand for `(fill: color)` .
`style.path-style`	`color` / `dictionary`	`rgb("#FFE9A8")`	Traversal or sift path.
`style.remove-style`	`color` / `dictionary`	`rgb("#FFCDD2")`	Node/cell removed by an operation.
`style.rotate-style`	`color` / `dictionary`	`rgb("#BBDEFB")`	AVL rotation nodes and visible subtree roots.
`style.*-style.fill`	`color`	mark default	Fill for the marked node/cell.
`style.*-style.shape`	`str`	`style.node-shape`	Marked tree/heap node shape: `"circle"` , `"square"` , `"rounded"` , `"capsule"` , `"diamond"` , or `"hexagon"` .
`style.*-style.stroke`	`stroke`	normal stroke	Marked node/cell outline.
`style.*-style.node-radius`	`float`	`style.node-radius`	Marked tree/heap node radius.
`style.*-style.text`	`dictionary`	`style.node-text`	Marked node/cell text overrides. Accepts `size` , `color` , `font` , and `weight` .

Nested keys for `style.indices`

Path	Type	Default	Description
`style.indices.enabled`	`bool`	`false`	Draw index labels below array cells.
`style.indices.labels`	`auto` / `array`	`auto`	`auto` draws zero-based numeric indices; an array supplies custom labels. Merge sort's divide and merge trees address the array by each cell's position in the original array.
`style.indices.offset`	`tuple`	`(0, -0.28)`	Canvas offset from the bottom-center index position.
`style.indices.size`	`length`	`style.label-text.size`	Index text size.
`style.indices.color`	`color`	`style.label-text.color`	Index text color.
`style.indices.font`	`str` / `array`	inherits	Typst text font selection.

`style.indices.weight`	`str` / `int`	inherits	Typst text weight.

Nested keys for `cell-customizations`

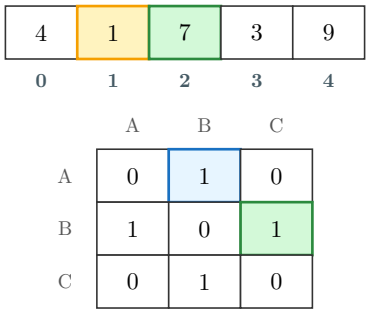
Path	Type	Default	Description
`cell-customizations[]`	`tuple`	n/a	One `(cell, options)` tuple.
`cell-customizations[].cell`	`content`	required	Array index, or matrix coordinate `(row, column)` .
`cell-customizations[].options`	`dictionary`	required	Per-cell drawing options.
`cell-customizations[].options.fill`	`color`	normal fill	Cell fill.
`cell-customizations[].options.stroke`	`stroke`	normal stroke	Cell outline.
`cell-customizations[].options.text`	`dictionary`	`style.node-text`	Text style merged into the cell text.

Field	Call	Effect
`.diagram`	n/a	The rendered array or matrix.

```typst
#array-view(
  4, 1, 7, 3, 9,
  style: (
    indices: (
      enabled: true,
      color: rgb("#455A64"),
      weight: "bold",
    ),
  ),
  cell-customizations: (
    (1, (fill: rgb("#FFF3BF"), stroke: 1pt + rgb("#F59F00"))),
    (2, (fill: rgb("#D3F9D8"), stroke: 1pt + rgb("#2B8A3E"))),
  ),
).diagram

#matrix(
  ((0, 1, 0), (1, 0, 1), (0, 1, 0)),
  row-labels: ([A], [B], [C]),
  column-labels: ([A], [B], [C]),
  cell-customizations: (
    ((0, 1), (fill: rgb("#E7F5FF"), stroke: 1pt + rgb("#1971C2"))),
    ((1, 2), (fill: rgb("#D3F9D8"), stroke: 1pt + rgb("#2B8A3E"))),
  ),
).diagram
```

Typst



3.9 `hash-table()`

`hash-table` supports separate chaining and linear probing while using the same cell/list styling and chainable operation steps as the other structures. Each chaining bucket points to its head entry or `none` with an arrow. Chain ends use the same compact `⌀` terminator as linked lists. Chained entries default to a wider `style.box-w` of `1.5` so key/value pairs fit; an explicit `style.box-w` overrides it.

```typst
#hash-table(..entries, size: 7, collision: "chaining", hash: auto, style: (:), language: "en", messages: (:))
```

Typst

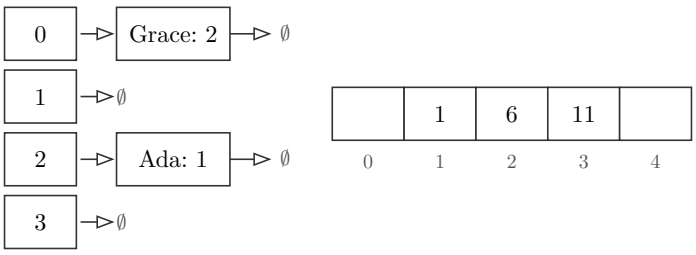
Argument	Type	Default	Description
`..entries`	`content` / `tuple`	`()`	Keys, or `(key, value)` pairs.

`size`	`int`	`7`	Positive bucket/slot count.
`collision`	`str`	`"chaining"`	`"chaining"` / `"chain"` or `"linear"` .
`hash`	`auto` / `function`	`auto`	Custom `key => int` function. The default hashes integers and the UTF-8 bytes of other values.
`style`	`dictionary`	`(:)`	Uses the existing box, value/index text, pointer text, scale, and mark-style keys.

Field	Call	Effect
`.diagram`	n/a	Rendered buckets or probe table.
`.insert`	`(obj.insert)(key, value: none)`	Insert or update a key. Returns a step.
`.delete`	`(obj.delete)(key)`	Delete a key; linear probing leaves a tombstone.
`.search`	`(obj.search)(key)`	Highlight traversed chain nodes or probe slots; exposes `.found` .

Typst

```
#std.stack(dir: ltr, spacing: 1.5em,
  hash-table(("Ada", 1), ("Grace", 2), size: 4).diagram,
  hash-table(1, 6, 11, size: 5, collision: "linear").diagram,
)
```



3.10 Sorting algorithms

Full signatures:

```typst
#merge-sort(array, order: "asc", labels: true, language: "en", messages: (:))
#merge-operation(left, right, order: "asc", pointers: true, labels: true, language: "en", messages: (:))
#partition-step(array, order: "asc", pivot: "middle", pointers: false, labels: true, language: "en", messages: (:))
#quick-sort(array, order: "asc", pivot: "last", labels: true, language: "en", messages: (:))
#bubble-sort(array, order: "asc", pointers: true, labels: true, compare: none, swap: none, language: "en", messages: (:))
#insertion-sort(array, order: "asc", pointers: true, labels: true, compare: none, swap: none, language: "en", messages: (:))
#selection-sort(array, order: "asc", pointers: true, labels: true,
  compare: none, current: none, minimum: none, swap: none, language: "en", messages: (:))
#sort-sequence(steps, columns: 3, gap: 1em, row-gap: 1em)
```

Typst

Sorting builders generate full traces from a single input array. Each returns `.diagram` , `.steps` , and `.result` . Use `.diagram` for the default wrapped trace, or pass `.steps` to `sort-sequence(...)` when you want different columns or spacing. Every trace records all comparison and swap steps. Each step keeps its label and array together when a page break is needed. Merge sort renders a connected tree: arrows lead from each array to its two divided arrays, then from each pair into its merged result. Curly braces identify the Divide, Merge, and Partition phases. Quicksort groups all active recursive partitions on the same row before expanding them together on the next row. Use `partition-step(..., pivot: "last")` for the detailed partition used by a last-pivot quicksort; `pointers: true` adds labelled cursor arrows, and each successful comparison includes the subsequent `i` advance as its own frame.

To style a trace, pass a styled `array-view(...)` in place of the bare array: its style (fills, text, and index configuration) is carried through every step of the trace. Custom `style.indices.labels` entries are addressed by each cell's position in the original array, so a labelled trace keeps the same indices in every divided and merged subtree. Bubble sort adds a pointer-free settled frame after each pass to highlight its green suffix, and selection sort adds one after each minimum swap to highlight its sorted prefix. Settled frames name the newly settled value, and selection scan labels list position, minimum, then item.

Merge operation, bubble, insertion, and selection sort show labelled arrows above their active positions by default. Pass `pointers: false` to hide them. Merge operation labels its left, right, and output cursors `i` , `j` , and `i+j` respectively. Each arrow sits over a marked cell, and several markers on one cell spread across its width. The arrows are additive: the role still fills its cells, so pointers layer on top of the fill. Each role (`compare` , `swap` , and, for selection sort, `current` and `minimum`) takes a `node-mark-style(...)` that restyles that role's fill, stroke, and text in either mode; the arrow colour comes from the mark's stroke, or its fill when no stroke is set. To keep a cell unfilled under a pointer, set the role's `fill` to `none` . Quick sort and merge sort render composite phase diagrams and do not take marker roles.

Pass `labels: false` to hide generated text captions while preserving the arrays, highlights, indices, and optional pointer arrows. This also hides merge's left/right/result headings, partition metadata, and composite phase labels, which lets you supply your own translated explanation alongside a trace.

Sorting traces work directly with Touying reveals: `.steps` is an ordered array of rendered frames, so it can be revealed one at a time.

```typst
#let trace = bubble-sort((5, 1, 4, 2))

#for (index, step) in trace.steps.enumerate() [
  #only(index + 1)[#step.diagram]
]
```

`only(index + 1)` shows exactly one step per reveal in the same spot. Using `#pause` instead accumulates the steps, stacking every frame on screen at once.

Argument	Type	Default	Description
`array`	`array` / `array-view`	(required)	Values to sort from left to right, as a plain array or a styled `array-view(...)` whose style is reused for every step. Values must be comparable with `<` / `>` . `merge-operation` takes two such arguments (`left` , `right`); every other builder takes one.
`order`	`"asc"` / `"desc"`	`"asc"`	Sort direction.
`pivot`	`"first"` / `"last"` / `int` / `"middle"`	`"last"` / `"middle"`	Quicksort selects an end of each subarray or a position index. `partition-step` accepts `middle` or `last` use `last` for the detailed quicksort partition trace.
`pointers`	`bool`	`true` / `false`	Merge operation, bubble, insertion, selection sort, and last-pivot `partition-step` only. Draw labelled index arrows above the marked cells, layered on top of their fills. Merge operation uses `i` , `j` , and `i+j` last-pivot partition uses `i` , `j` , and `pivot` .
`labels`	`bool`	`true`	All sorting builders. Set to `false` to hide generated step captions and structural text while preserving diagrams, highlights, indices, and pointer arrows.
`compare` , `swap` , `current` , `minimum`	`dictionary`	`none`	Bubble, insertion, and selection sort only. A `node-mark-style(...)` override for that role's marker. `current` and `minimum` apply to selection sort only.

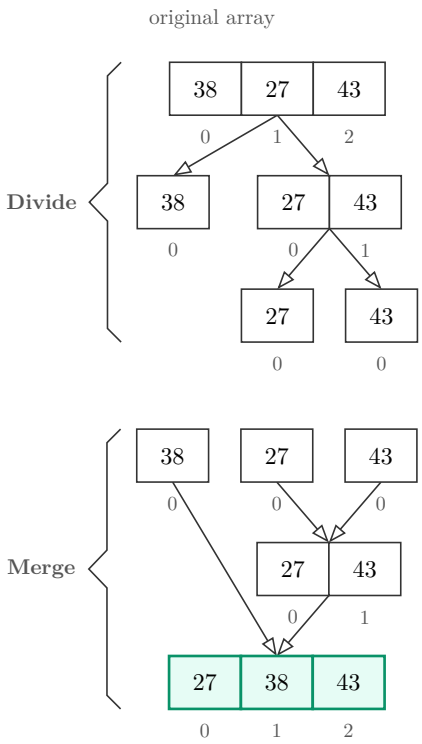
Field	Call	Effect
`.diagram`	n/a	Rendered full sorting trace.
`.steps`	`array`	Generated step dictionaries. Each step has `label` , `diagram` , and `values` .
`.result`	`array`	Sorted values.

Full traces

Long traces below flow one labelled step at a time, so page breaks cannot separate a label from its array or make neighbouring frames overlap.

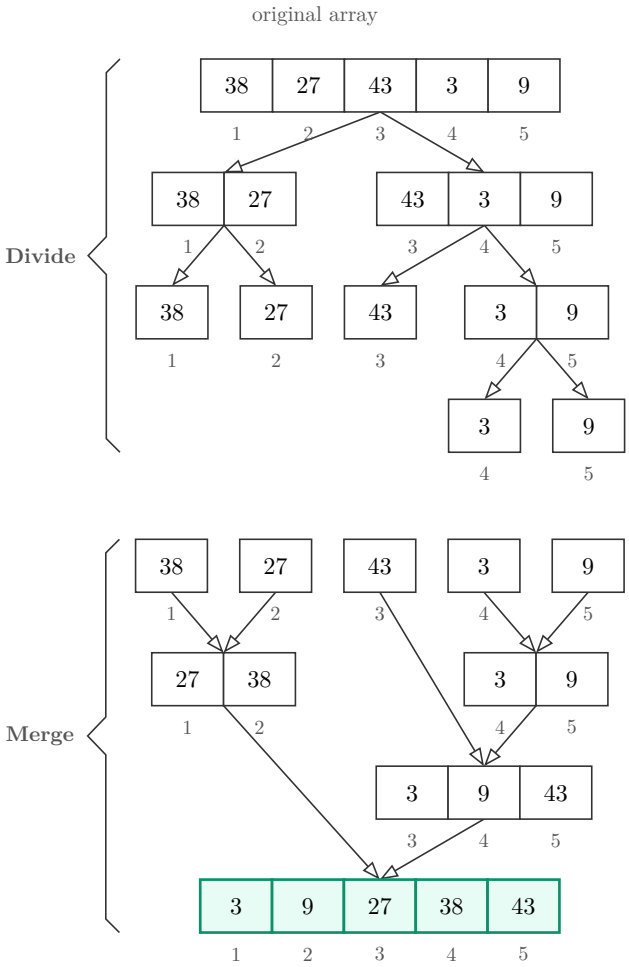
Merge sort

```typst
#let trace = merge-sort((38, 27, 43))
#trace.diagram
```



Merge sort with 1-based index labels

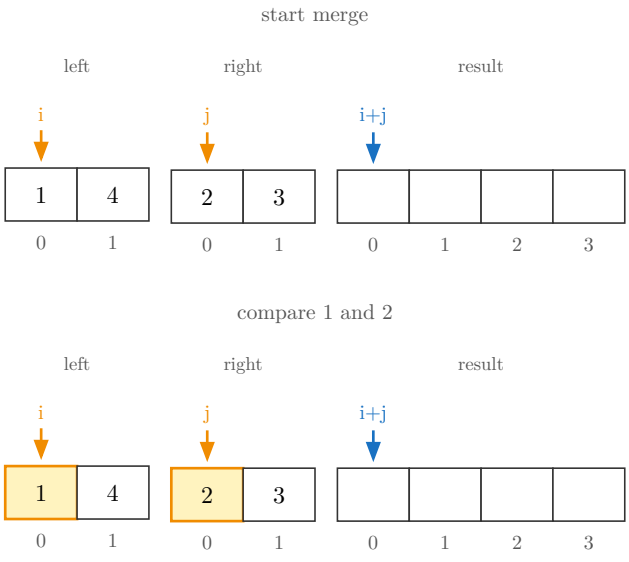
```typst
#let trace = merge-sort(array-view(
  38, 27, 43, 3, 9,
  style: (indices: (enabled: true, labels: ("1", "2", "3", "4", "5"))),
))
#trace.diagram
```



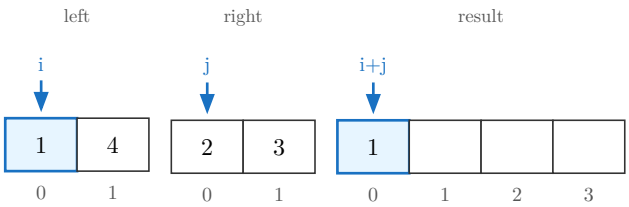
Merge operation

```
1  #let trace = merge-operation((1, 4), (2, 3))
2  #for step in trace.steps [#step.diagram]
```

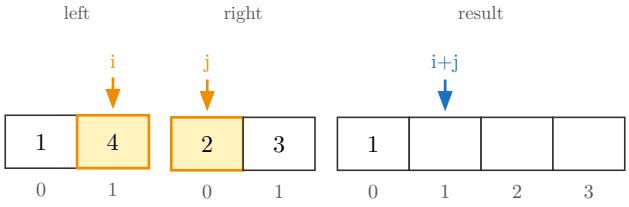
Typst



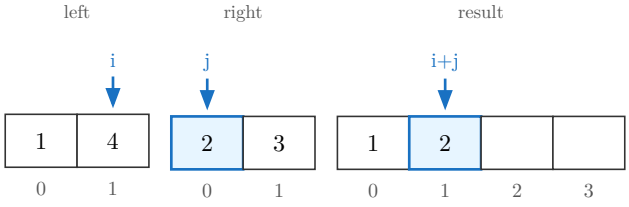
take 1



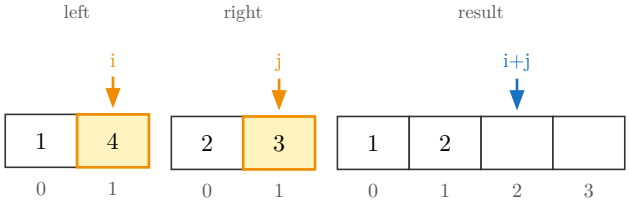
compare 4 and 2



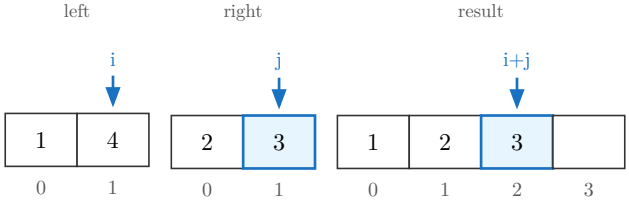
take 2



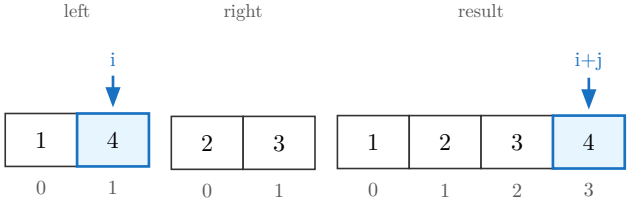
compare 4 and 3

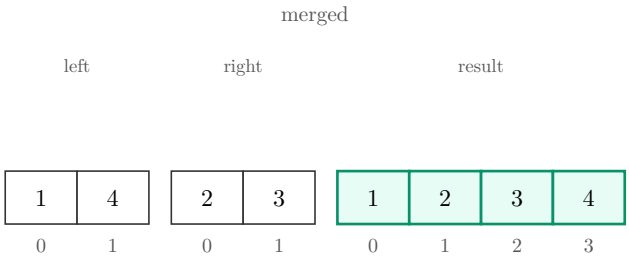


take 3



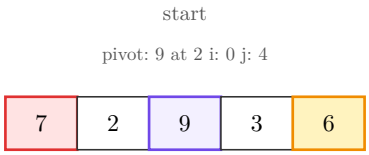
take remaining 4

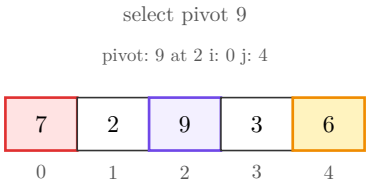


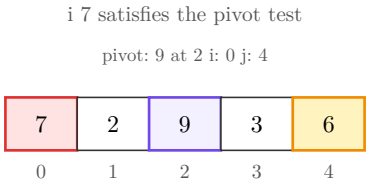


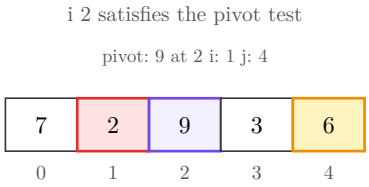
Partition around a middle pivot

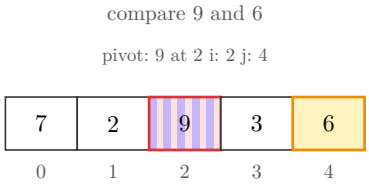
```typst
#let trace = partition-step((7, 2, 9, 3, 6))
#for step in trace.steps [#step.diagram]
```

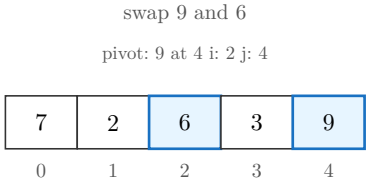


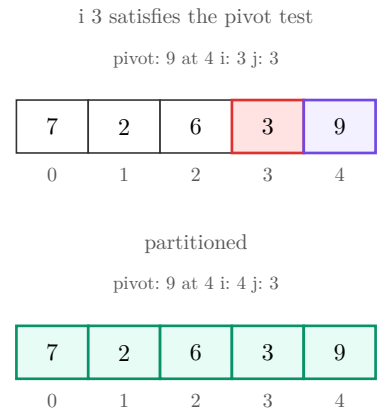






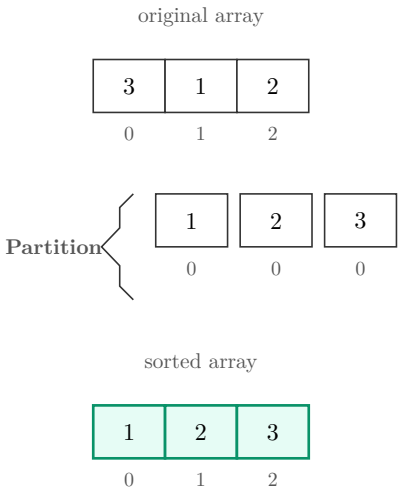






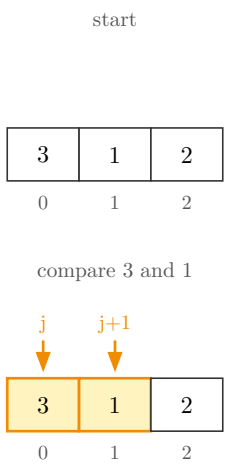
Quick sort

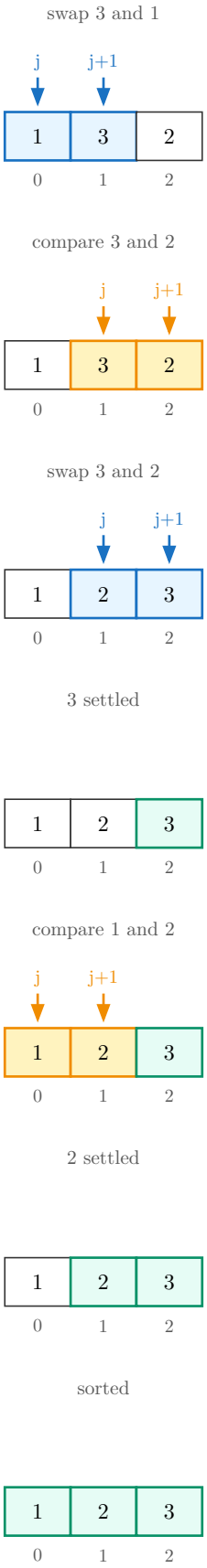
```typst
#let trace = quick-sort((3, 1, 2))
#trace.diagram
```



Bubble sort

```typst
#let trace = bubble-sort((3, 1, 2))
#for step in trace.steps [#step.diagram]
```

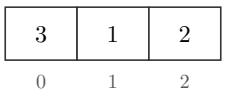




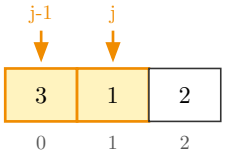
Insertion sort

```typst
#let trace = insertion-sort((3, 1, 2))
#for step in trace.steps [#step.diagram]
```

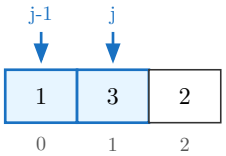
start



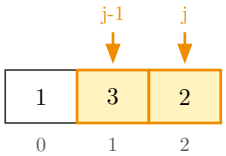
compare 3 and 1



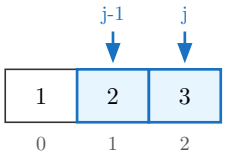
swap 3 and 1



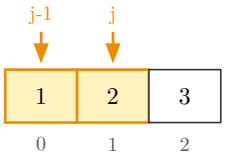
compare 3 and 2



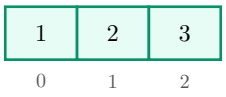
swap 3 and 2



compare 1 and 2



sorted



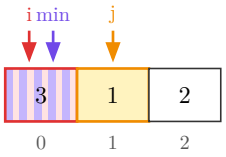
Selection sort

```typst
#let trace = selection-sort((3, 1, 2))
#for step in trace.steps [#step.diagram]
```

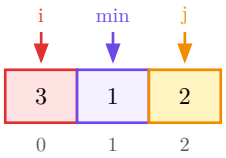
start



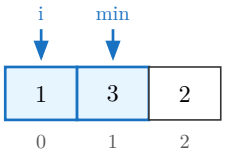
position 0, minimum 0, item 1



position 0, minimum 1, item 2



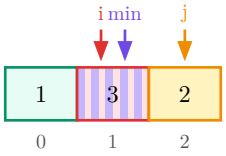
swap 3 and 1



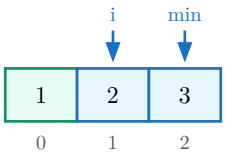
1 settled



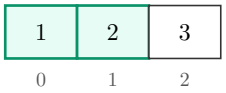
position 1, minimum 1, item 2



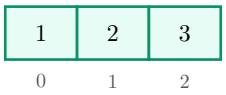
swap 3 and 2



2 settled



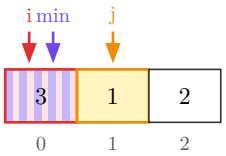
sorted



Hide generated captions

```
#let trace = selection-sort((3, 1, 2), labels: false)
#trace.steps.at(1).diagram
```

Typst



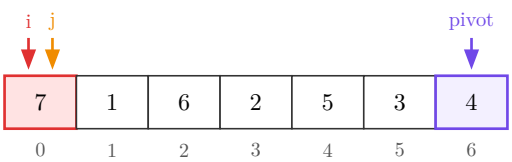
With `pivot: "last"` , placing the median at the end produces an even quicksort partition: three values on each side of the pivot. This trace renders every comparison, swap, and cursor position.

```
#let trace = partition-step(
  (7, 1, 6, 2, 5, 3, 4),
  pivot: "last",
  pointers: true,
)
#for step in trace.steps [#step.diagram]
```

Typst

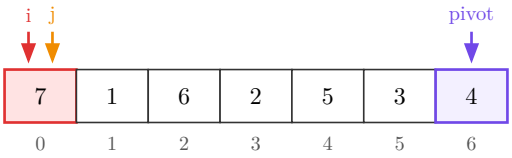
start

pivot: 4 at 6 i: 0 j: 0



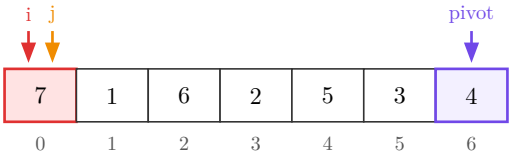
select last pivot 4

pivot: 4 at 6 i: 0 j: 0



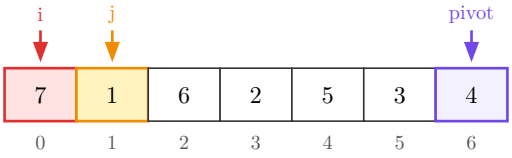
compare 7 and pivot 4

pivot: 4 at 6 i: 0 j: 0



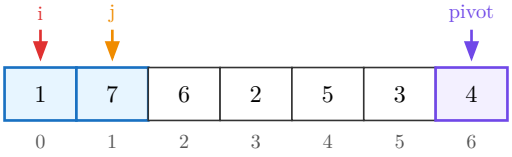
compare 1 and pivot 4

pivot: 4 at 6 i: 0 j: 1



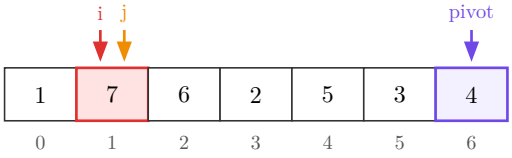
swap 7 and 1

pivot: 4 at 6 i: 0 j: 1



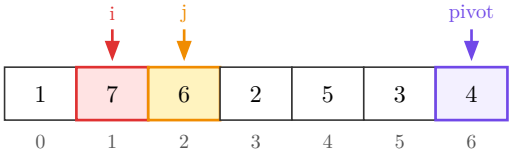
advance i to 1

pivot: 4 at 6 i: 1 j: 1



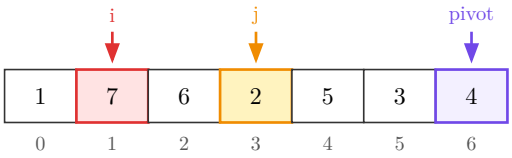
compare 6 and pivot 4

pivot: 4 at 6 i: 1 j: 2



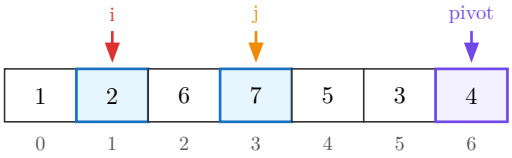
compare 2 and pivot 4

pivot: 4 at 6 i: 1 j: 3



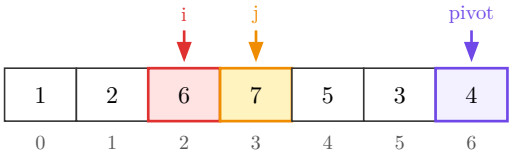
swap 7 and 2

pivot: 4 at 6 i: 1 j: 3



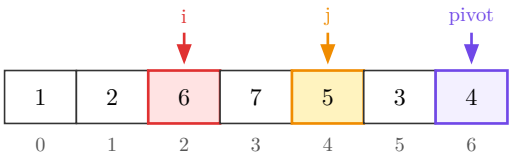
advance i to 2

pivot: 4 at 6 i: 2 j: 3



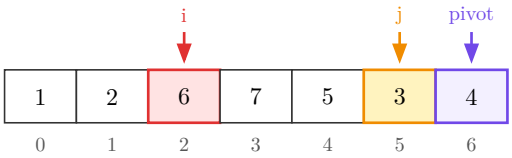
compare 5 and pivot 4

pivot: 4 at 6 i: 2 j: 4



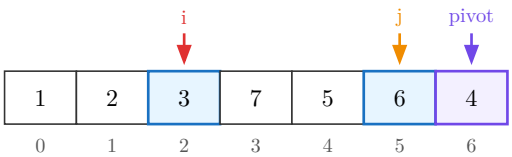
compare 3 and pivot 4

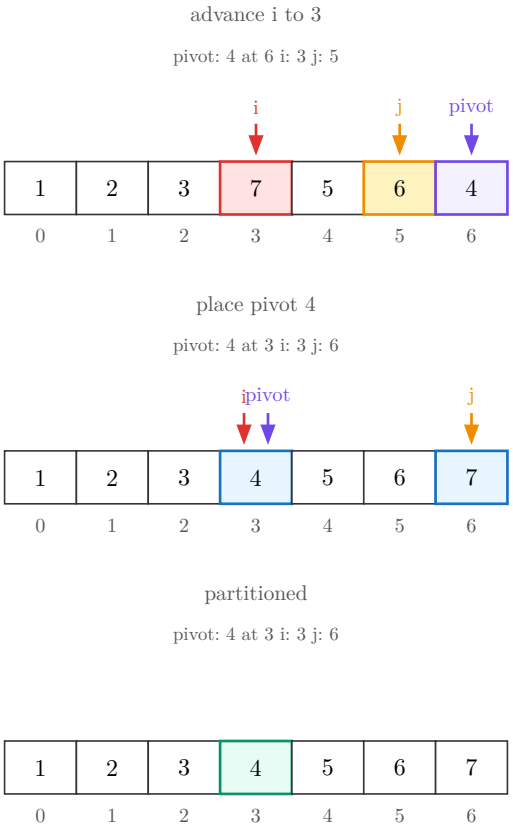
pivot: 4 at 6 i: 2 j: 5



swap 6 and 3

pivot: 4 at 6 i: 2 j: 5

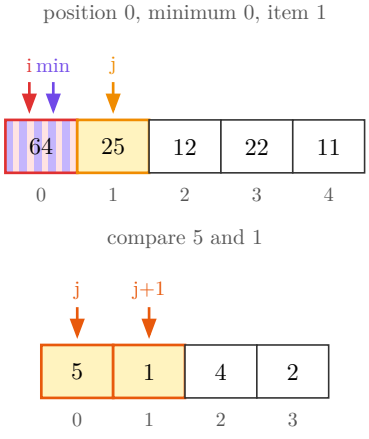




Pointer markers keep the cell fill clean and let each role carry its own colour. These focused frames show selection's `i` , `min` , and `j` markers together, plus customized bubble-sort marker strokes.

```typst
#let selection = selection-sort((64, 25, 12, 22, 11), pointers: true)
#selection.steps.at(1).diagram

#let bubble = bubble-sort(
  (5, 1, 4, 2),
  pointers: true,
  compare: node-mark-style(stroke: 1pt + rgb("#E8590C")),
  swap: node-mark-style(stroke: 1pt + rgb("#2F9E44")),
)
#bubble.steps.at(1).diagram
```



3.11 `graph()`

Full signature:

```typst
#graph(
  adjacency,
  directed: true,
  labels: (:),
  positions: (:),
  layout: "auto",
  layout-options: (:),
  radius: auto,
  gap: auto,
  edge-customizations: (),
  node-customizations: (),
  node-labels: (:),
  style: (:),
)
```

`adjacency` maps each node label to an array of that node's neighbor labels. To label an edge, use `(neighbor, label)` instead of just `neighbor` . A node with no outgoing edges still needs its key present, with an empty array; a label that only ever shows up as someone else's neighbor gets added on its own.

Argument	Type	Default	Description
`adjacency`	`dictionary`	(required)	Node label `(str)` to array of neighbor labels or `(neighbor, edge-label)` pairs.
`directed`	`bool`	`true`	Draw an arrowhead on every declared pair. `false` drops arrowheads and collapses a reciprocal pair into the one edge it represents.
`labels`	`dictionary`	`(:)`	Node label to drawn content: math, styled text, an image, anything. A label left out keeps the plain key as its own content.
`positions`	`dictionary`	`(:)`	Node label to absolute `(x, y)` or `(rel:, offset:)` placement.
`layout`	`str`	`"auto"`	`"auto"` uses a circle; `"linear"` uses a row; `"force"` clusters connections; `"layered"` ranks a directed DAG; `"manual"` requires every node in `positions` .
`layout-options`	`dictionary`	`(:)`	Options for `"force"` or `"layered"` . Non-empty options with another layout are an error.
`radius`	`auto` / `float`	`auto`	Circle radius for `layout: "auto"` . Passing a radius with another layout is an error.
`gap`	`auto` / `float`	`auto`	Spacing between nodes for `layout: "linear"` , in canvas units. `auto` is `1.5` .

			Passing a gap with another layout is an error.
`edge-customizations`	`array`	`()`	`(from, to, options)` tuples restyling one edge. See below.
`node-customizations`	`array`	`()`	`(node, options)` tuples restyling one node by graph identity.
`node-labels`	`array` / `dictionary`	`(:)`	Labels drawn outside individual graph nodes.
`style`	`dictionary`	`(:)`	Per-call style override merged over the defaults. See Section 8.

Nested keys for `adjacency`

Path	Type	Default	Description
`adjacency.<node>`	`array`	required	Outgoing neighbors for one node. Use an empty array to draw an isolated node.
`adjacency.<node>[]`	`content` / `tuple`	required	Either a neighbor label, or `(neighbor, edge-label)` to draw a label on that edge.
`adjacency.<node>[].neighbor`	`content`	required	Target node identity.
`adjacency.<node>[].edge-label`	`content`	none	Optional edge label, such as a weight.

Nested keys for `labels`

Path	Type	Default	Description
`labels.<node>`	`content`	node key	Drawn content for one node. Identity still comes from the plain adjacency/position label.

Nested keys for `positions`

Path	Type	Default	Description
`positions.<node>`	`point` / `dictionary`	auto layout position	Absolute `(x, y)` position or relative placement dictionary.
`positions.<node>.rel`	`content`	required for relative placement	Another node label to place from.
`positions.<node>.offset`	`point`	required for relative placement	`(dx, dy)` offset from `rel` .

`layout-options` for `layout: "force"`

Path	Type	Default	Description
`edge-length`	`float`	`1.8`	Preferred spring length. Must be positive.
`repulsion`	`float`	`1.0`	Strength of all-pairs electrical repulsion. Must be positive.

`attraction`	`float`	`1.0`	Strength of spring attraction along edges. Must be positive.
`node-edge-repulsion`	`float`	`1.0`	Strength that pushes non-endpoint nodes away from edges. Must be positive.
`node-edge-clearance`	`float`	`0.25`	Extra clearance beyond each node's shape boundary. Must be zero or positive.
`iterations`	`int`	`60`	Cooling iterations, from `1` through `500` .
`component-gap`	`float`	`2.5`	Predictable horizontal gap between disconnected components. Must be positive.

`layout-options` for `layout: "layered"`

Path	Type	Default	Description
`direction`	`str`	`"right"`	Flow direction: `"right"` , `"left"` , `"down"` , or `"up"` .
`layer-gap`	`float`	`2.2`	Gap between dependency ranks. Must be positive.
`node-gap`	`float`	`1.4`	Gap between nodes in one rank. Must be positive.
`crossing-sweeps`	`int`	`4`	Deterministic barycenter sweeps, from `0` through `20` . Use `0` to preserve declared ordering without crossing reduction.

Nested keys for `edge-customizations`

Path	Type	Default	Description
`edge-customizations[]`	`tuple`	n/a	One `(from, to, options)` tuple. For trees, `from` / `to` are parent and child values; for graphs, node labels.
`edge-customizations[].from`	`content`	required	Source node identity.
`edge-customizations[].to`	`content`	required	Target node identity.
`edge-customizations[].options`	`dictionary`	required	Per-edge options.
`edge-customizations[].options.stroke`	`stroke`	none	Full stroke override. Wins over `color` if both are set.
`edge-customizations[].options.color`	`color`	none	Paint only, at the default thickness.
`edge-customizations[].options.pattern`	`str`	`"normal"`	`"normal"` , `"dashed"` , `"dotted"` , or `"wavy"` .
`edge-customizations[].options.arrow`	`none` / `bool` / `str`	none	Per-edge arrowheads: `none` , `"end"` / `true` , `"start"` , or `"both"` .

edge-customizations[].options.bend	str / bool	false	"left" or "right" curves the edge relative to its travel direction. false keeps it straight.
edge-customizations[].options.angle	angle	25deg	How sharply a bent edge curves. Ignored when bend is false .
edge-customizations[].options.label	content / dictionary	none	Tree edge label content, or graph edge-label text overrides. A dictionary can include content for trees.

Nested keys for `edge-customizations[].options.label`

Path	Type	Default	Description
edge-customizations[].options.label.size	length	style.label-text.size	Edge-label size.
edge-customizations[].options.label.content	content	tree only	Tree edge label content. Graph edge labels come from adjacency entries such as ("B", [4]) .
edge-customizations[].options.label.color	color	style.label-text.color	Edge-label text color.
edge-customizations[].options.label.rotation	angle / str	style.label-text.rotation	An angle, or "edge" to follow the tree/graph edge angle.
edge-customizations[].options.label.font	str / array	style.label-text.font	Typst text font selection.
edge-customizations[].options.label.weight	str / int	style.label-text.weight	Typst text weight.

Nested keys for `style`

Path	Type	Default	Description
style.node-radius	float	0.34	Circle radius or base size for other node shapes.
style.node-shape	str	"circle"	"circle" , "square" , "rounded" , "capsule" , "diamond" , or "hexagon" .
style.node-stroke	stroke	0.6pt + rgb("#333333")	Node outline.
style.node-fill	color	white	Default node fill.
style.edge-stroke	stroke	0.6pt + rgb("#333333")	Default edge stroke.
style.edge-arrow	none / bool / str	none	Global arrowhead override: none / false , "end" or true , "start" , or "both" . directed: true uses "end" unless this overrides it.
style.edge-arrow-fill	none / color	none	none gives open arrowheads; a color fills arrowheads solid.
style.edge-pattern	str	"normal"	"normal" , "dashed" , "dotted" , or "wavy" .
style.edge-wave-amplitude	float	0.07	Wave height in canvas units.

style.edge-wave-step	float	0.14	Approximate wavelength in canvas units.
style.node-text	dictionary	(size: 9pt)	Node text dictionary. See nested keys below.
style.value-text	dictionary	inherits node-text	Node-value typography.
style.label-text	dictionary	(fill: rgb("#555555"))	Edge label text dictionary. See nested keys below.
style.edge-label-text	dictionary	inherits label-text	Edge-label typography.
style.algorithm-label-text	dictionary	(size: 8pt)	BFS/DFS/Dijkstra trace captions.
style.visited-style	dictionary	green	Visited-node fill/stroke/text/shape overrides.
style.current-style	dictionary	blue	Current-node overrides.
style.queued-style	dictionary	yellow/orange	Queued or stacked node overrides.
style.active-edge-style	dictionary	blue 2pt stroke	Currently inspected edge overrides.
style.scale	float	1.0	Uniform scale on the rendered graph.

Nested keys for `style.node-text`

Path	Type	Default	Description
style.node-text.size	length	inherits	Text size.
style.node-text.color	color	inherits	Text color.
style.node-text.rotation	angle	0deg	Text rotation applied by the diagram renderer.
style.node-text.font	str / array	inherits	Typst text font selection.
style.node-text.weight	str / int	inherits	Typst text weight.

Nested keys for `style.label-text`

Path	Type	Default	Description
style.label-text.size	length	85% of style.node-text.size	Label text size.
style.label-text.color	color	rgb("#555555")	Label text color.
style.label-text.rotation	angle / str	0deg	Label rotation. Tree and graph edge labels also accept "edge" to follow the edge angle.
style.label-text.font	str / array	inherits	Typst text font selection.
style.label-text.weight	str / int	inherits	Typst text weight.

Nested keys for `node-customizations`

Path	Type	Default	Description

`node-customizations[]`	`tuple`	n/a	One `(node, options)` tuple.
`node-customizations[].node`	`content`	required	Graph identity key, not the display label from `labels` .
`node-customizations[].options`	`dictionary`	required	Per-node drawing options.
`node-customizations[].options.fill`	`color`	normal fill	Node fill.
`node-customizations[].options.stroke`	`stroke`	normal stroke	Node outline.
`node-customizations[].options.shape`	`str`	`style.node-shape`	Trees/graphs only: `"circle"` , `"square"` , `"rounded"` , `"capsule"` , `"diamond"` , or `"hexagon"` .
`node-customizations[].options.node-radius`	`float`	`style.node-radius`	Trees/graphs only. Shape size.
`node-customizations[].options.text`	`dictionary`	`style.node-text`	Text style merged into the node text.

Nested keys for `node-labels`

Path	Type	Default	Description
`node-labels[]`	`tuple`	n/a	One `(node, label)` tuple.
`node-labels[].node`	`content`	required	Graph identity key, not the display label from `labels` .
`node-labels[].label`	`content` / `dictionary`	required	Bare content for quick labels, or a dictionary with `content` plus style and placement keys.
`node-labels[].label.content`	`content`	required for dict labels	Label body. `body` is also accepted.
`node-labels[].label.position`	`str` / `angle`	`"right"`	`"right"` , `"left"` , `"top"` , `"bottom"` , or an angle.
`node-labels[].label.offset`	`point`	`(0, 0)`	Extra `(dx, dy)` shift after the position is applied.
`node-labels[].label.size`	`length`	`style.label-text.size`	Node-label size.
`node-labels[].label.color`	`color`	`style.label-text.color`	Node-label text color.
`node-labels[].label.rotation`	`angle`	`style.label-text.rotation`	Rotation of the label text itself.
`node-labels[].label.font`	`str` / `array`	`style.label-text.font`	Typst text font selection.
`node-labels[].label.weight`	`str` / `int`	`style.label-text.weight`	Typst text weight.

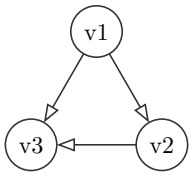
Nested keys for `style.node-labels`

Path	Type	Default	Description
`style.node-labels.position`	`str` / `angle`	`"right"`	Default position for all node labels in this diagram.

`style.node-labels.offset`	`point`	`(0, 0)`	Default extra `(dx, dy)` shift for all node labels.
`style.node-labels.gap`	`float`	`0.22`	Default distance from the node boundary in canvas units.
`style.node-labels.size`	`length`	`style.label-text.size`	Default node-label text size.
`style.node-labels.color`	`color`	`style.label-text.color`	Default node-label text color.
`style.node-labels.font`	`str / array`	inherits	Typst text font selection.
`style.node-labels.weight`	`str / int`	inherits	Typst text weight.
`style.node-labels.rotation`	`angle`	`style.label-text.rotation`	Default rotation of the label text itself.

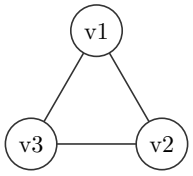
Field	Call	Effect
`.diagram`	n/a	The rendered graph.

```typst
#graph(("v1": ("v2", "v3"), "v2": ("v3",), "v3": ())).diagram
```



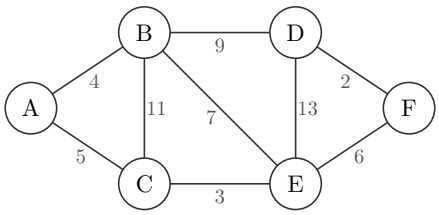
Declaring both directions of an edge still draws it once.

```typst
#graph(
  ("v1": ("v2", "v3"), "v2": ("v1", "v3"),
  "v3": ()),
  directed: false,
).diagram
```



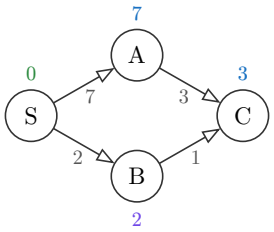
Use `(neighbor, label)` entries to draw edge labels, such as weights.

```typst
#graph(
  (
    "A": (("B", [4]), ("C", [5])),
    "B": (("C", [11]), ("D", [9]), ("E", [7])),
    "C": (("E", [3]),),
    "D": (("E", [13]), ("F", [2])),
    "E": (("F", [6]),),
    "F": (),
  ),
  directed: false,
  layout: "manual",
  positions: (
    "A": (0, 0),
    "B": (rel: "A", offset: (1.5, 1.0)),
    "C": (rel: "A", offset: (1.5, -1.0)),
    "D": (rel: "B", offset: (2.0, 0.0)),
    "E": (rel: "C", offset: (2.0, 0.0)),
    "F": (rel: "D", offset: (1.5, -1.0)),
  ),
).diagram
```



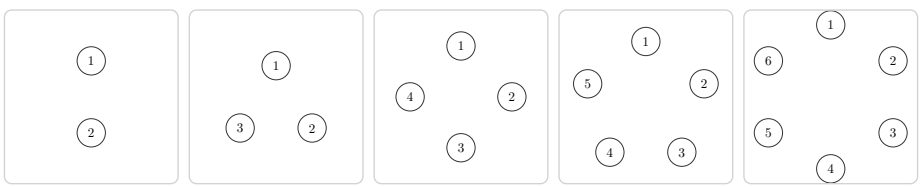
Use `node-labels` for annotations such as tentative distances, predecessor tags, ranks, or algorithm state. The node's identity and its drawn inside label stay unchanged.

```typst
#graph(
  ("S": (("A", [7]), ("B", [2])), "A": (("C", [3]),), "B": (("C", [1]),), "C": ()),
  layout: "manual",
  positions: ("S": (0, 0), "A": (1.4, 0.8), "B": (1.4, -0.8), "C": (2.8, 0)),
  node-labels: (
    ("S", (content: [$0$], position: "top", color: rgb("#2B8A3E"))),
    ("A", [$7$]),
    ("B", (content: [$2$], position: "bottom", color: rgb("#7048E8"))),
    ("C", [$3$]),
  ),
  style: (
    edge-arrow: "end",
    node-labels: (position: "top", color: rgb("#1971C2")),
  ),
).diagram
```



`layout: "auto"` is the default. Nodes are placed around a circle; the radius grows as the node count increases. Pass `radius:` to set the circle radius yourself.

```typst
#let panel(body) = block(width: 6.2em, height: 6.2em, inset: 0.2em,
  stroke: 0.5pt + luma(210), radius: 3pt, clip: true,
  align(center + horizon, scale(55%, reflow: true, body)))
#std.stack(dir: ltr, spacing: 0.4em,
  panel(graph(("1": (), "2": ())).diagram),
  panel(graph(("1": (), "2": (), "3": ())).diagram),
  panel(graph(("1": (), "2": (), "3": (), "4": ())).diagram),
  panel(graph(("1": (), "2": (), "3": (), "4": (), "5": ())).diagram),
  panel(graph(("1": (), "2": (), "3": (), "4": (), "5": (), "6": ())).diagram),
)
```



`radius` only applies to `layout: "auto"` . Combining it with `"linear"` , `"force"` , `"layered"` , or `"manual"` raises an error.

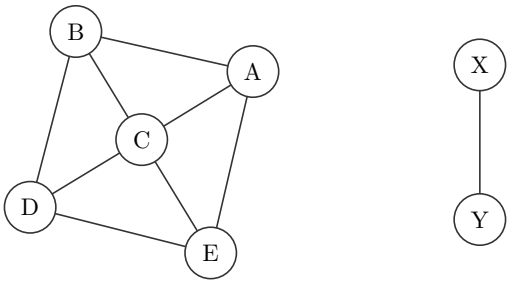
`layout: "linear"` places nodes left to right in a single row, useful for topological orderings. `gap:` sets the spacing between nodes; it only applies to this layout.

```typst
#graph(
  ("A": ("B",), "B": ("C",), "C": ("D",), "D": ()),
  layout: "linear",
  gap: 2.5,
).diagram
```



`layout: "force"` applies deterministic spring attraction along edges, electrical repulsion between every pair of nodes, and repulsion between each edge and every node that is not its endpoint. Node-edge clearance uses the resolved node shape and radius, including node customizations, followed by a bounded separation pass and uniform component expansion when needed. Three deterministic starting arrangements are scored by remaining node-edge overlap, proper edge crossings, and spring length; the best arrangement is selected. Directed edges act as ordinary physical connections; their direction affects arrowheads, not the simulation. Disconnected components are simulated independently and packed left to right. The same graph and options always produce the same coordinates, with no randomness.

```typst
#graph(
  (
    "A": ("B", "C"),
    "B": ("C", "D"),
    "C": ("D", "E"),
    "D": ("E",),
    "E": ("A",),
    "X": ("Y",),
    "Y": (),
  ),
  directed: false,
  layout: "force",
  layout-options: (edge-length: 2.0, component-gap: 3.0),
).diagram
```

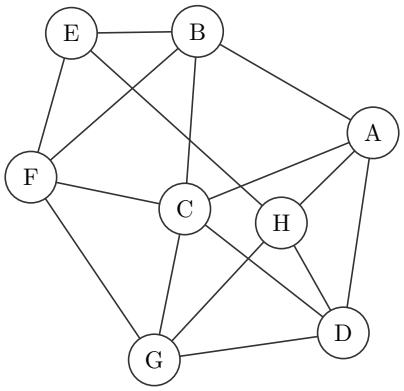


> **Note.** Force layout is designed for small educational graphs. It evaluates three deterministic candidate starts. Each iteration compares every pair of nodes and every non-endpoint node-edge pair within a component, so its dominant work is `O(V² + VE)` per candidate iteration and becomes cubic for dense graphs. Lower `iterations` for faster drafts; raise it, up to `500` , only when a graph benefits visibly. Explicit `positions:` overrides are applied after automatic layout and can intentionally reintroduce node-edge overlaps.

A denser mesh exercises node-edge clearance when many straight segments share the same region. Increasing `edge-length` gives the optimizer more room; `node-edge-clearance` controls the extra gap beyond node boundaries.

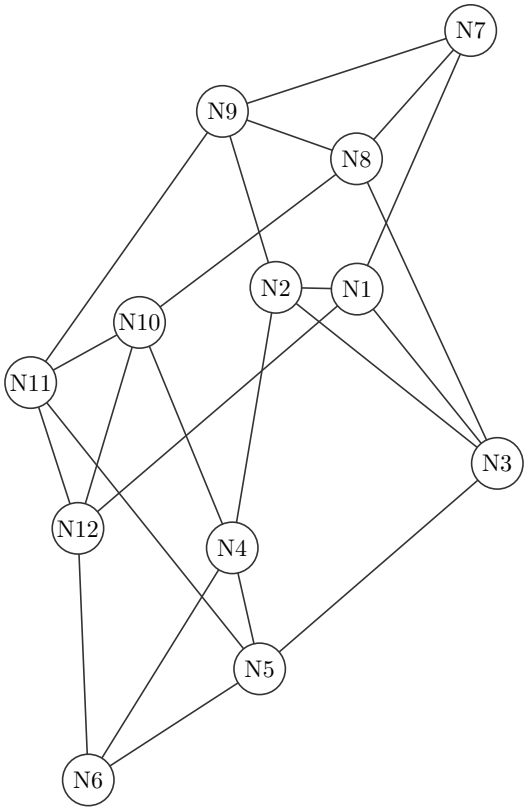
Typst

```
#graph(
  (
    "A": ("B", "C", "D"),
    "B": ("C", "E", "F"),
    "C": ("D", "F", "G"),
    "D": ("G", "H"),
    "E": ("F", "H"),
    "F": ("G",),
    "G": ("H",),
    "H": ("A",),
  ),
  directed: false,
  layout: "force",
  layout-options: (edge-length: 2.1, node-edge-clearance: 0.3),
).diagram
```



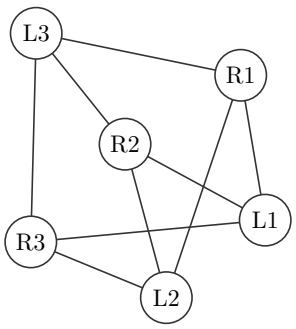
This larger example adds long-range connections among twelve nodes. The force model tests each such edge against all ten nodes that are not its endpoints.

```typst
#graph(
  (
    "N1": ("N2", "N3", "N7"),
    "N2": ("N3", "N4", "N9"),
    "N3": ("N5", "N8"),
    "N4": ("N5", "N6", "N10"),
    "N5": ("N6", "N11"),
    "N6": ("N12",),
    "N7": ("N8", "N9"),
    "N8": ("N9", "N10"),
    "N9": ("N11",),
    "N10": ("N11", "N12"),
    "N11": ("N12",),
    "N12": ("N1",),
  ),
  directed: false,
  layout: "force",
  layout-options: (edge-length: 2.2, node-edge-clearance: 0.3),
).diagram
```



$K_{3,3}$ is nonplanar, so at least one proper edge crossing is unavoidable with straight edges. The layout still keeps those edges outside every node that is not an endpoint.

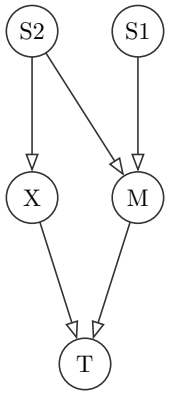
```typst
#graph(
  (
    "L1": ("R1", "R2", "R3"),
    "L2": ("R1", "R2", "R3"),
    "L3": ("R1", "R2", "R3"),
    "R1": (),
    "R2": (),
    "R3": (),
  ),
  directed: false,
  layout: "force",
  layout-options: (edge-length: 2.1, node-edge-clearance: 0.3),
).diagram
```



`layout: "layered"` requires `directed: true` and a directed acyclic graph. Sources occupy the first rank; longest-path ranking places every node after all of its predecessors. A deterministic barycenter pass reduces crossings, and `direction` transforms the canonical rightward layout. A directed cycle produces a diagnostic suggesting `"force"` or `"manual"` .

Typst

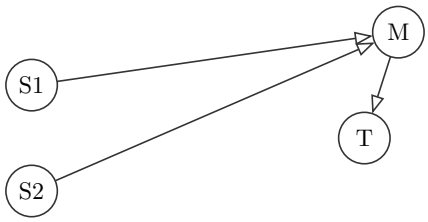
```
#graph(
  (
    "S1": ("M",),
    "S2": ("M", "X"),
    "M": ("T",),
    "X": ("T",),
    "T": (),
  ),
  layout: "layered",
  layout-options: (direction: "down"),
).diagram
```



Explicit `positions` retain their usual override semantics with both new layouts. Omitted nodes keep their automatic coordinates; absolute and relative overrides are resolved on top.

Typst

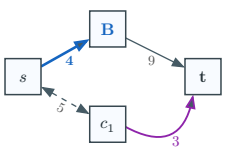
```
#graph(
  ("S1": ("M",), "S2": ("M",), "M": ("T",), "T": ()),
  layout: "layered",
  positions: ("M": (3.2, 1.4)),
).diagram
```



Argument coverage: this graph combines adjacency edge labels, node display labels, manual and relative positions, graph-wide style, per-edge options, and nested edge-label overrides.

Typst

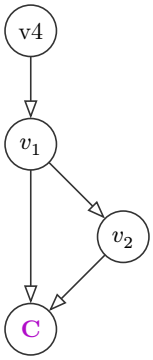
```
#graph(
  (
    "A": (("B", [4]), ("C", [5])),
    "B": (("D", [9]),),
    "C": (("D", [3]),),
    "D": (),
  ),
  directed: true,
  labels: (
    "A": $s$,
    "B": text(fill: rgb("#1565C0"))[*B*],
    "C": $c_1$,
    "D": text(weight: "bold")[t],
  ),
  layout: "manual",
  positions: (
    "A": (0, 0),
    "B": (rel: "A", offset: (1.6, 0.9)),
    "C": (rel: "A", offset: (1.6, -0.9)),
    "D": (rel: "B", offset: (1.8, -0.9)),
  ),
  edge-customizations: (
    ("A", "B", (stroke: 1.4pt + rgb("#1565C0"), label: (color: rgb("#1565C0"), weight: "bold"))),
    ("A", "C", (pattern: "dashed", arrow: "both", label: (rotation: "edge"))),
    ("C", "D", (bend: "right", angle: 35deg, color: rgb("#8E24AA"), label: (size: 8pt, color: rgb("#8E24AA")))),
  ),
  style: (
    node-shape: "square",
    node-radius: 0.34,
    node-fill: rgb("#F8FBFF"),
    node-stroke: 0.8pt + rgb("#37474F"),
    node-text: (size: 9pt, color: rgb("#263238")),
    edge-stroke: 0.7pt + rgb("#455A64"),
    edge-arrow: "end",
    edge-arrow-fill: rgb("#455A64"),
    edge-pattern: "normal",
    label-text: (size: 7.5pt, color: rgb("#555555")),
    scale: 0.7,
  ),
).diagram
```



3.12 `labels` and `positions`

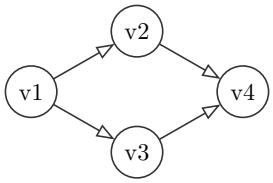
`labels` swaps what's drawn inside a node for any content, keyed by the plain-string label that `adjacency` / `positions` / `edge-customizations` still use for identity. With any automatic layout, `positions` places a node at an absolute `(x, y)` point or at `(rel: other-label, offset: (dx, dy))` on top of the calculated layout.

```typst
#graph(
  ("v1": ("v2", "v3"), "v2": ("v3",), "v3": (), "v4": ("v1",)),
  labels: ("v1": $v_1$, "v2": $v_2$, "v3": text(fill: purple)[*C*]),
  positions: ("v4": (rel: "v1", offset: (0, 1.5))),
).diagram
```



Set `layout: "manual"` to skip automatic placement. Every node, including a neighbor-only node, must have a `positions` entry. At least one absolute `(x, y)` entry should act as an anchor for relative placements.

```typst
#graph(
  ("v1": ("v2", "v3"), "v2": ("v4",), "v3": ("v4",), "v4": ()),
  layout: "manual",
  positions: (
    "v1": (0, 0),
    "v2": (rel: "v1", offset: (1.4, 0.8)),
    "v3": (rel: "v1", offset: (1.4, -0.8)),
    "v4": (rel: "v2", offset: (1.4, -0.8)),
  ),
).diagram
```



3.13 edge-customizations

An array of `(from, to, options)` tuples, each restyling one edge. For `graph` , `from` and `to` are node labels. For generated trees, they are parent and child values. `options` is a dictionary.

Nested keys for `edge-customizations`

Path	Type	Default	Description
`edge-customizations[]`	`tuple`	n/a	One `(from, to, options)` tuple. For trees, `from` / `to` are parent and child values; for graphs, node labels.
`edge-customizations[].from`	`content`	required	Source node identity.
`edge-customizations[].to`	`content`	required	Target node identity.
`edge-customizations[].options`	`dictionary`	required	Per-edge options.
`edge-customizations[].options.stroke`	`stroke`	none	Full stroke override. Wins over `color` if both are set.
`edge-customizations[].options.color`	`color`	none	Paint only, at the default thickness.
`edge-customizations[].options.pattern`	`str`	`"normal"`	`"normal"` , `"dashed"` , `"dotted"` , or `"wavy"` .
`edge-customizations[].options.arrow`	`none` / `bool` / `str`	none	Per-edge arrowheads: `none` , `"end"` / `true` , `"start"` , or `"both"` .
`edge-customizations[].options.bend`	`str` / `bool`	`false`	`"left"` or `"right"` curves the edge relative to its travel direction. `false` keeps it straight.
`edge-customizations[].options.angle`	`angle`	`25deg`	How sharply a bent edge curves. Ignored when `bend` is `false` .
`edge-customizations[].options.label`	`content` / `dictionary`	none	Tree edge label content, or graph edge-label text overrides. A dictionary can include `content` for trees.

Nested keys for `edge-customizations[].options.label`

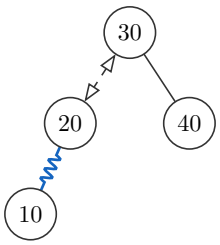
Path	Type	Default	Description
`edge-customizations[].options.label.size`	`length`	`style.label-text.size`	Edge-label size.
`edge-customizations[].options.label.content`	`content`	tree only	Tree edge label content. Graph edge labels come from adjacency entries such as `("B", [4])` .
`edge-customizations[].options.label.color`	`color`	`style.label-text.color`	Edge-label text color.
`edge-customizations[].options.label.rotation`	`angle` / `str`	`style.label-text.rotation`	An angle, or `"edge"` to follow the tree/graph edge angle.
`edge-customizations[].options.label.font`	`str` / `array`	`style.label-text.font`	Typst text font selection.
`edge-customizations[].options.label.weight`	`str` / `int`	`style.label-text.weight`	Typst text weight.

Typst

```
#graph(
  ("v1": ("v2", "v3"), "v2": ("v3",), "v3": ()),
  edge-customizations: (
    ("v1", "v2", (stroke: 2pt + red)),
    ("v2", "v3", (color: rgb("#2E7D32"))),
```

Tree edge customizations use the values in the parent and child nodes:

```typst
#bst(
  30, 20, 40, 10,
  edge-customizations: (
    (30, 20, (pattern: "dashed", arrow: "both")),
    (20, 10, (pattern: "wavy", color: rgb("#1565C0"))),
  ),
).diagram
```



Note. A mutual pair bent the same way on both directions (`("v1", "v2", (bend: "left"))` and `("v2", "v1", (bend: "left"))`) curves to opposite sides, since `"left"` is relative to each edge's own direction of travel. The two arrows read as two edges instead of one double-headed line.

Note. Nodes default to a circle, in the order their labels first show up, with radius growing by node count. Choose deterministic `"force"` placement when connectivity should shape the drawing, or `"layered"` for a directed DAG. `graph` has no add-node, add-edge, or tree-style mutation transition yet. See Section 10.

3.14 Graph algorithm traces

`bfs` , `dfs` , and `dijkstra` render algorithm steps directly on the graph. Green nodes are visited, blue is current, yellow/orange nodes are queued or stacked, and the blue edge is the neighbor currently being inspected. When `dijkstra` reaches a target, its final step also highlights every edge in the shortest path. `source` is required; `target` is optional so a trace can either stop at a goal or traverse every reachable node.

```typst
#bfs(adjacency, source, target: none, directed: true, goal-test: "discovery", columns: 1, captions: true, language: "en", messages: (:), ..graph-options)
#dfs(adjacency, source, target: none, directed: true, columns: 1, captions: true, language: "en", messages: (:), ..graph-options)
#dijkstra(adjacency, source, target: none, directed: true, columns: 1, captions: true, language: "en", messages: (:), ..graph-options)
```

Argument	Type	Default	Description
`adjacency`	`dictionary`	(required)	Same graph adjacency dictionary accepted by `graph` .
`source`	`str`	(required)	Starting node identity.
`target`	`none` / `str`	`none`	Optional early-stop node.
`goal-test`	`"discovery"` / `"expansion"`	`"discovery"`	BFS only. Stop when the target is first enqueued, or wait until it is dequeued.

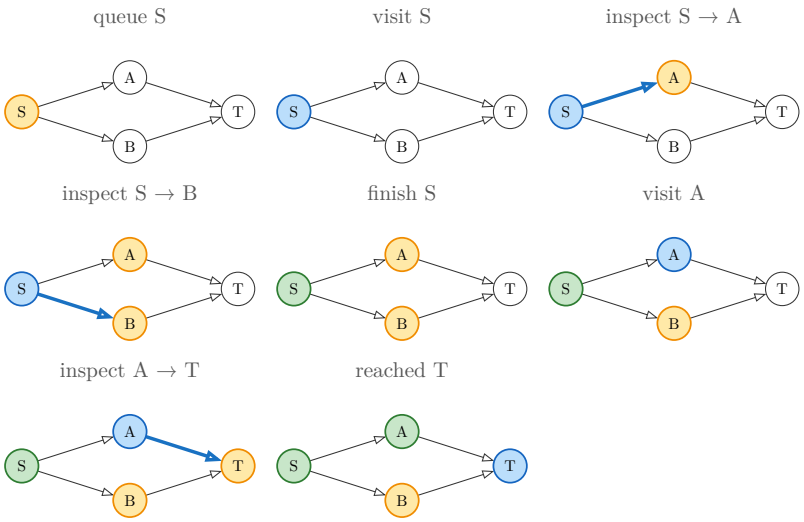
`directed`	`bool`	`true`	Whether adjacency pairs are directed.
`columns`	`int`	`1`	Trace grid columns.
`row-gap`	`length`	`0.8em`	Gap between trace cells.
`captions`	`bool`	`true`	Show generated step captions.
`labels` / `positions` / `layout` / `layout-options` / `radius` / `gap`	same as `graph`	same	Node display and placement options. One layout is resolved per call and reused unchanged by every trace panel.
`edge-customizations` / `node-customizations` / `node-labels`	same as `graph`	same	Base customizations merged with algorithm states.
`style`	`dictionary`	`(:)`	Graph style plus `visited-style` , `current-style` , `queued-style` , `active-edge-style` , and `algorithm-label-text` .

Note. For a targeted unweighted BFS, the default `goal-test: "discovery"` stops as soon as an unvisited target is found while expanding a node: that first discovery already gives a shortest path. Choose `goal-test: "expansion"` to keep the traditional dequeue-and-test trace, which may show work on other queued nodes before the target is removed.

Note. `dijkstra` treats an unlabeled edge as weight `1` . Weighted entries must use non-negative numeric values such as `("B", 4)` . It draws a distinct purple `d = ...` label beside every node on every step; the default label gap is `0.5` canvas units. `style.node-labels.gap` or per-node `node-labels` options can override it.

Note. Each trace step exposes `node-positions` , the resolved coordinate dictionary used for that panel. Every step in one trace shares the same value, including force layouts, so state highlighting never makes nodes drift.

```typst
#bfs(
  ("S": ("A", "B"), "A": ("T",), "B": ("T",), "T": ()),
  "S", target: "T", columns: 3,
  layout: "layered",
  style: graph-style(scale: 0.65),
).diagram
```



4 Hand-composed trees

For abstract diagrams, build a tree by hand with `node(...)` and `subtree(...)` instead of inserting real keys. Draw a node over two elided subtrees, the way you'd draw an AVL height invariant.

```
1 #tree(root, style: (:), edge-customizations: (), node-customizations: (),
  node-labels: (:))
2 #node(label, left: none, right: none, children: none, fill: none)
3 #subtree(label, fill: none, height: none, scale: 1)
```

Typst

Argument	Type	Default	Description
`root`	result of `node()`	(required)	The tree to render.
`style`	`dictionary`	`(:)`	Per-call style override merged over the defaults. See Section 8.
`edge-customizations`	`array`	`()`	`(parent, child, options)` tuples restyling individual tree edges by node label. See Section 3.13.
`node-customizations`	`array`	`()`	`(node, options)` tuples restyling individual nodes by label.
`node-labels`	`array` / `dictionary`	`(:)`	Labels drawn outside individual nodes.

Nested keys for `style`

Path	Type	Default	Description
`style.node-radius`	`float`	`0.34`	Circle radius, or base size for other node shapes, in canvas units.
`style.node-shape`	`str`	`"circle"`	`"circle"` , `"square"` , `"rounded"` , `"capsule"` , `"diamond"` , or `"hexagon"` .
`style.x-gap`	`float`	`1.05`	Horizontal spacing between in-order columns.
`style.y-gap`	`float`	`1.2`	Vertical spacing between depths.
`style.tree-layout`	`str`	`"columns"`	`"columns"` gives every leaf its own column. `"tidy"` packs sibling subtrees against their contours, keeping parents centered and subtree triangles clear. See Section 3.1.1.
`style.node-stroke`	`stroke`	`0.6pt + rgb("#333333")`	Node outline.
`style.node-fill`	`color`	`white`	Default node fill.
`style.edge-stroke`	`stroke`	`0.6pt + rgb("#333333")`	Parent-child edge stroke.
`style.edge-arrow`	`none` / `bool` / `str`	`none`	Edge arrowheads: `none` / `false` , `"end"` or `true` , `"start"` , or `"both"` .
`style.edge-pattern`	`str`	`"normal"`	`"normal"` , `"dashed"` , `"dotted"` , or `"wavy"` .

`style.edge-wave-amplitude`	`float`	`0.07`	Wave height in canvas units.
`style.edge-wave-step`	`float`	`0.14`	Approximate wavelength in canvas units.
`style.node-text`	`dictionary`	`(size: 9pt)`	Node text dictionary. See nested keys below.
`style.value-text`	`dictionary`	inherits `node-text`	Value text overrides.
`style.label-text`	`dictionary`	`(fill: rgb("#555555"))`	Annotation text dictionary. See nested keys below.
`style.edge-label-text`	`dictionary`	inherits `label-text`	Edge-label typography.
`style.operation-text`	`dictionary`	`(size: 8pt)`	Operation-arrow caption typography.
`style.scale`	`float`	`1.0`	Uniform scale on `.diagram` , `.before` , and `.after` .
`style.diff-colors`	`bool`	`true`	`false` keeps operation marks but uses normal fills.

Nested keys for `style.node-text`

Path	Type	Default	Description
`style.node-text.size`	`length`	inherits	Text size.
`style.node-text.color`	`color`	inherits	Text color.
`style.node-text.rotation`	`angle`	`0deg`	Text rotation applied by the diagram renderer.
`style.node-text.font`	`str` / `array`	inherits	Typst text font selection.
`style.node-text.weight`	`str` / `int`	inherits	Typst text weight.

Nested keys for `style.label-text`

Path	Type	Default	Description
`style.label-text.size`	`length`	85% of `style.node-text.size`	Label text size.
`style.label-text.color`	`color`	`rgb("#555555")`	Label text color.
`style.label-text.rotation`	`angle` / `str`	`0deg`	Label rotation. Tree and graph edge labels also accept `"edge"` to follow the edge angle.
`style.label-text.font`	`str` / `array`	inherits	Typst text font selection.
`style.label-text.weight`	`str` / `int`	inherits	Typst text weight.

Nested subtree style keys

Path	Type	Default	Description
`style.tri-w`	`float`	`1.2`	Triangle base width before `subtree(scale:)` .

style.tri-h	float	1.4	Triangle height before subtree(scale:) .

Nested keys for diff-highlight dictionaries

Path	Type	Default	Description
style.new-style	color / dictionary	rgb("#C8E6C9")	Node/cell added by an operation. A color is shorthand for (fill: color) .
style.path-style	color / dictionary	rgb("#FFE9A8")	Traversal or sift path.
style.remove-style	color / dictionary	rgb("#FFCDD2")	Node/cell removed by an operation.
style.rotate-style	color / dictionary	rgb("#BBDEFB")	AVL rotation nodes and visible subtree roots.
style.*-style.fill	color	mark default	Fill for the marked node/cell.
style.*-style.shape	str	style.node-shape	Marked tree/heap node shape: "circle" , "square" , "rounded" , "capsule" , "diamond" , or "hexagon" .
style.*-style.stroke	stroke	normal stroke	Marked node/cell outline.
style.*-style.node-radius	float	style.node-radius	Marked tree/heap node radius.
style.*-style.text	dictionary	style.node-text	Marked node/cell text overrides. Accepts size , color , font , and weight .

Nested keys for edge-customizations

Path	Type	Default	Description
edge-customizations[]	tuple	n/a	One (from, to, options) tuple. For trees, from / to are parent and child values; for graphs, node labels.
edge-customizations[].from	content	required	Source node identity.
edge-customizations[].to	content	required	Target node identity.
edge-customizations[].options	dictionary	required	Per-edge options.
edge-customizations[].options.stroke	stroke	none	Full stroke override. Wins over color if both are set.
edge-customizations[].options.color	color	none	Paint only, at the default thickness.
edge-customizations[].options.pattern	str	"normal"	"normal" , "dashed" , "dotted" , or "wavy" .
edge-customizations[].options.arrow	none / bool / str	none	Per-edge arrowheads: none , "end" / true , "start" , or "both" .

edge-customizations[].options.bend	str / bool	false	"left" or "right" curves the edge relative to its travel direction. false keeps it straight.
edge-customizations[].options.angle	angle	25deg	How sharply a bent edge curves. Ignored when bend is false .
edge-customizations[].options.label	content / dictionary	none	Tree edge label content, or graph edge-label text overrides. A dictionary can include content for trees.

Nested keys for edge-customizations[].options.label

Path	Type	Default	Description
edge-customizations[].options.label.size	length	style.label-text.size	Edge-label size.
edge-customizations[].options.label.content	content	tree only	Tree edge label content. Graph edge labels come from adjacency entries such as ("B", [4]) .
edge-customizations[].options.label.color	color	style.label-text.color	Edge-label text color.
edge-customizations[].options.label.rotation	angle / str	style.label-text.rotation	An angle, or "edge" to follow the tree/graph edge angle.
edge-customizations[].options.label.font	str / array	style.label-text.font	Typst text font selection.
edge-customizations[].options.label.weight	str / int	style.label-text.weight	Typst text weight.

Nested keys for node-customizations

Path	Type	Default	Description
node-customizations[]	tuple	n/a	One (node, options) tuple.
node-customizations[].node	content	required	Hand-composed node label.
node-customizations[].options	dictionary	required	Per-node drawing options.
node-customizations[].options.fill	color	normal fill	Node fill.
node-customizations[].options.stroke	stroke	normal stroke	Node outline.
node-customizations[].options.shape	str	style.node-shape	Trees/graphs only: "circle" , "square" , "rounded" , "capsule" , "diamond" , or "hexagon" .
node-customizations[].options.node-radius	float	style.node-radius	Trees/graphs only. Shape size.
node-customizations[].options.text	dictionary	style.node-text	Text style merged into the node text.

Nested keys for node-labels

Path	Type	Default	Description
node-labels[]	tuple	n/a	One (node, label) tuple.
node-labels[].node	content	required	Hand-composed node label.

`node-labels[].label`	`content` / `dictionary`	required	Bare content for quick labels, or a dictionary with `content` plus style and placement keys.
`node-labels[].label.content`	`content`	required for dict labels	Label body. `body` is also accepted.
`node-labels[].label.position`	`str` / `angle`	`"right"`	`"right"` , `"left"` , `"top"` , `"bottom"` , or an angle.
`node-labels[].label.offset`	`point`	`(0, 0)`	Extra `(dx, dy)` shift after the position is applied.
`node-labels[].label.size`	`length`	`style.label-text.size`	Node-label size.
`node-labels[].label.color`	`color`	`style.label-text.color`	Node-label text color.
`node-labels[].label.rotation`	`angle`	`style.label-text.rotation`	Rotation of the label text itself.
`node-labels[].label.font`	`str` / `array`	`style.label-text.font`	Typst text font selection.
`node-labels[].label.weight`	`str` / `int`	`style.label-text.weight`	Typst text weight.

Nested keys for `style.node-labels`

Path	Type	Default	Description
`style.node-labels.position`	`str` / `angle`	`"right"`	Default position for all node labels in this diagram.
`style.node-labels.offset`	`point`	`(0, 0)`	Default extra `(dx, dy)` shift for all node labels.
`style.node-labels.gap`	`float`	`0.22`	Default distance from the node boundary in canvas units.
`style.node-labels.size`	`length`	`style.label-text.size`	Default node-label text size.
`style.node-labels.color`	`color`	`style.label-text.color`	Default node-label text color.
`style.node-labels.font`	`str` / `array`	inherits	Typst text font selection.
`style.node-labels.weight`	`str` / `int`	inherits	Typst text weight.
`style.node-labels.rotation`	`angle`	`style.label-text.rotation`	Default rotation of the label text itself.

`node(label, left:, right:, children:, fill:)` : a node with any content label. Use `left` / `right` for binary trees, or `children` for a multiway tree such as a B-tree schematic.

Argument	Type	Default	Description
`label`	`content`	(required)	The node's label; any content, not just a key.
`left`	`none` / `node()` / `subtree()`	`none`	Left child.
`right`	`none` / `node()` / `subtree()`	`none`	Right child.
`children`	`none` / `array`	`none`	Multiway children. When set, this replaces `left` / `right` for layout and rendering.
`fill`	`none` / `color`	`none`	Fill tint for this node.

Nested keys for `node()` children

Path	Type	Default	Description
`left.label`	`content`	required when `left` is `node()` or `subtree()`	Label drawn inside the left child.
`left.left`	`none` / `node()` / `subtree()`	`none`	Nested left child.
`left.right`	`none` / `node()` / `subtree()`	`none`	Nested right child.
`right.label`	`content`	required when `right` is `node()` or `subtree()`	Label drawn inside the right child.
`right.left`	`none` / `node()` / `subtree()`	`none`	Nested left child.
`right.right`	`none` / `node()` / `subtree()`	`none`	Nested right child.
`children[]`	`node()` / `subtree()`	n/a	One child in a multiway tree, laid out left to right.

`subtree(label, fill:, height:, scale:)` : a triangle leaf standing in for an elided subtree.

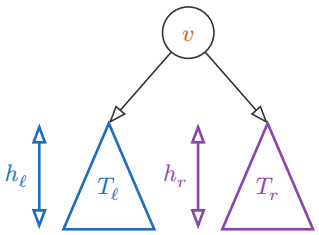
Argument	Type	Default	Description
`label`	`content`	(required)	Label drawn inside the triangle.
`fill`	`none` / `color`	`none`	Outline and label tint.
`height`	`none` / `content`	`none`	Optional side bracket labeling the subtree's height.
`scale`	`float`	`1`	Resizes the triangle relative to the theme's `tri-w` / `tri-h` .

Style keys used by `subtree()`

Path	Type	Default	Description
`style.tri-w`	`float`	`1.2`	Triangle base width before `subtree(scale:)` .
`style.tri-h`	`float`	`1.4`	Triangle height before `subtree(scale:)` .
`style.node-text`	`dictionary`	`(size: 9pt)`	Triangle label and height-bracket text styling.

Typst

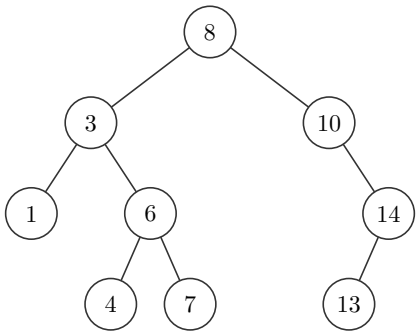
```
#tree(
  node(
    text(fill: rgb("#C75B12"))[$v$],
    left: subtree($T_ell$, fill: rgb("#1F6FBF"), height: $h_ell$),
    right: subtree($T_r$, fill: rgb("#8E44AD"), height: $h_r$),
  ),
  style: (edge-arrow: true),
)
```



Nest `node(...)` calls to draw a full manual tree. The indentation below mirrors the tree shape: each child is written inside its parent's `left:` or `right:` argument.

Typst

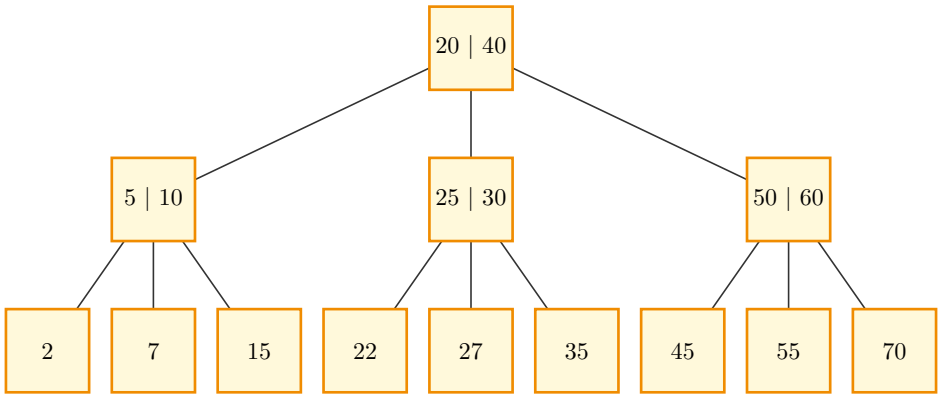
```
#tree(
  node(
    [8],
    left: node(
      [3],
      left: node([1]),
      right: node(
        [6],
        left: node([4]),
        right: node([7]),
      ),
    ),
    right: node(
      [10],
      right: node([14], left: node([13])),
    ),
  ),
)
```



Use `children: (...)` for B-tree-style or n-ary schematics.

Typst

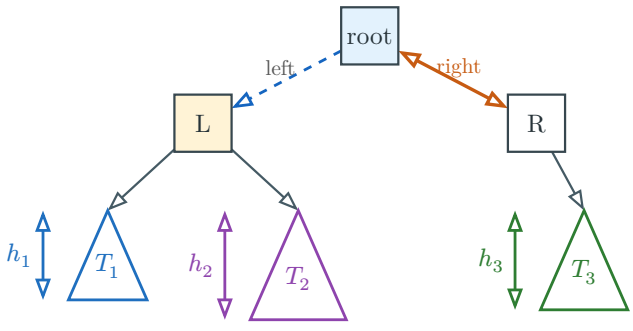
```
#tree(
  node([20 | 40], children: (
    node([5 | 10], children: (
      node([2]),
      node([7]),
      node([15]),
    )),
    node([25 | 30], children: (
      node([22]),
      node([27]),
      node([35]),
    )),
    node([50 | 60], children: (
      node([45]),
      node([55]),
      node([70]),
    )),
  )),
  style: (
    node-shape: "square",
    node-radius: 0.55,
    node-fill: rgb("#FFF9DB"),
    node-stroke: 1pt + rgb("#F08C00"),
    x-gap: 1.4,
    y-gap: 2.0,
  ),
)
```



Argument coverage: hand-composed trees combine nested `node()` calls, `subtree()` leaves, per-node fills, subtree height labels, tree style, and edge customizations keyed by the visible node labels.

Typst

```
#tree(
  node(
    [root],
    fill: rgb("#E3F2FD"),
    left: node(
      [L],
      fill: rgb("#FFF3D6"),
      left: subtree($T_1$, fill: rgb("#1F6FBF"), height: $h_1$, scale: 0.9),
      right: subtree($T_2$, fill: rgb("#8E44AD"), height: $h_2$, scale: 1.1),
    ),
    right: node(
      [R],
      right: subtree($T_3$, fill: rgb("#2E7D32"), height: $h_3$),
    ),
  ),
  style: (
    node-shape: "square",
    node-radius: 0.36,
    x-gap: 1.2,
    y-gap: 1.1,
    tri-w: 1.1,
    tri-h: 1.25,
    node-stroke: 0.8pt + rgb("#37474F"),
    node-text: (size: 9pt, color: rgb("#263238")),
    edge-stroke: 0.8pt + rgb("#455A64"),
    edge-arrow: "end",
    scale: 1.05,
  ),
  edge-customizations: (
    ([root], [L], (pattern: "dashed", color: rgb("#1565C0"), label: [left])),
    ([root], [R], (stroke: 1.2pt + rgb("#C75B12"), arrow: "both", label: (content:
    [right], color: rgb("#C75B12")))),
  ),
)
```



5 Operation transitions

`transition` is one-shot: build the structure from `keys`, apply the operation, and render the before state, an arrow, and the derived after state with the diff highlighted, in one call, no variable required. To keep operating on a structure across several calls, use the object model in Section 6 instead.

```
#transition(variant, keys, op, style: (:), edge-customizations: (), node-customizations: (), node-labels: (:))
```

Typst

Argument	Type	Default	Description
`variant`	`str`	(required)	`"bst"` , `"avl"` , `"min-heap"` , or `"max-heap"` .
`keys`	`array`	(required)	Keys the structure is built from, inserted in order.
`op`	operation	(required)	`tree-insert(key)` , `tree-delete(key)` , or `tree-search(key)` for trees. `heap-insert(key)` or `heap-extract` for heaps.
`style`	`dictionary`	`(:)`	Per-call style override merged over the defaults. See Section 8.
`edge-customizations`	`array`	`()`	Tree transition only. `(parent, child, options)` tuples restyling individual tree edges by value.
`node-customizations`	`array`	`()`	Tree transition only. `(node, options)` tuples restyling individual tree nodes by value.
`node-labels`	`array` / `dictionary`	`(:)`	Tree transition only. Labels drawn outside individual tree nodes.

Nested keys for `style`

Path	Type	Default	Description
`style.node-radius`	`float`	`0.34`	Circle radius, or base size for other node shapes, in canvas units.
`style.node-shape`	`str`	`"circle"`	`"circle"` , `"square"` , `"rounded"` , `"capsule"` , `"diamond"` , or `"hexagon"` .
`style.x-gap`	`float`	`1.05`	Horizontal spacing between in-order columns.
`style.y-gap`	`float`	`1.2`	Vertical spacing between depths.
`style.tree-layout`	`str`	`"columns"`	`"columns"` gives every leaf its own column. `"tidy"` packs sibling subtrees against their contours, keeping parents centered and subtree triangles clear. See Section 3.1.1.
`style.node-stroke`	`stroke`	`0.6pt + rgb("#333333")`	Node outline.
`style.node-fill`	`color`	`white`	Default node fill.
`style.edge-stroke`	`stroke`	`0.6pt + rgb("#333333")`	Parent-child edge stroke.

`style.edge-arrow`	`none` / `bool` / `str`	`none`	Edge arrowheads: `none` / `false` , `"end"` or `true` , `"start"` , or `"both"` .
`style.edge-pattern`	`str`	`"normal"`	`"normal"` , `"dashed"` , `"dotted"` , or `"wavy"` .
`style.edge-wave-amplitude`	`float`	`0.07`	Wave height in canvas units.
`style.edge-wave-step`	`float`	`0.14`	Approximate wavelength in canvas units.
`style.node-text`	`dictionary`	`(size: 9pt)`	Node text dictionary. See nested keys below.
`style.value-text`	`dictionary`	inherits `node-text`	Value text overrides.
`style.label-text`	`dictionary`	`(fill: rgb("#555555"))`	Annotation text dictionary. See nested keys below.
`style.edge-label-text`	`dictionary`	inherits `label-text`	Edge-label typography.
`style.operation-text`	`dictionary`	`(size: 8pt)`	Operation-arrow caption typography.
`style.scale`	`float`	`1.0`	Uniform scale on `.diagram` , `.before` , and `.after` .
`style.diff-colors`	`bool`	`true`	`false` keeps operation marks but uses normal fills.

Nested keys for `style.node-text`

Path	Type	Default	Description
`style.node-text.size`	`length`	inherits	Text size.
`style.node-text.color`	`color`	inherits	Text color.
`style.node-text.rotation`	`angle`	`0deg`	Text rotation applied by the diagram renderer.
`style.node-text.font`	`str` / `array`	inherits	Typst text font selection.
`style.node-text.weight`	`str` / `int`	inherits	Typst text weight.

Nested keys for `style.label-text`

Path	Type	Default	Description
`style.label-text.size`	`length`	85% of `style.node-text.size`	Label text size.
`style.label-text.color`	`color`	`rgb("#555555")`	Label text color.
`style.label-text.rotation`	`angle` / `str`	`0deg`	Label rotation. Tree and graph edge labels also accept `"edge"` to follow the edge angle.
`style.label-text.font`	`str` / `array`	inherits	Typst text font selection.
`style.label-text.weight`	`str` / `int`	inherits	Typst text weight.

Nested keys for diff-highlight dictionaries

Path	Type	Default	Description
`style.new-style`	`color` / `dictionary`	`rgb("#C8E6C9")`	Node/cell added by an operation. A color is shorthand for `(fill: color)` .
`style.path-style`	`color` / `dictionary`	`rgb("#FFE9A8")`	Traversal or sift path.
`style.remove-style`	`color` / `dictionary`	`rgb("#FFCDD2")`	Node/cell removed by an operation.
`style.rotate-style`	`color` / `dictionary`	`rgb("#BBDEFB")`	AVL rotation nodes and visible subtree roots.
`style.*-style.fill`	`color`	mark default	Fill for the marked node/cell.
`style.*-style.shape`	`str`	`style.node-shape`	Marked tree/heap node shape: `"circle"` , `"square"` , `"rounded"` , `"capsule"` , `"diamond"` , or `"hexagon"` .
`style.*-style.stroke`	`stroke`	normal stroke	Marked node/cell outline.
`style.*-style.node-radius`	`float`	`style.node-radius`	Marked tree/heap node radius.
`style.*-style.text`	`dictionary`	`style.node-text`	Marked node/cell text overrides. Accepts `size` , `color` , `font` , and `weight` .

Nested keys for `edge-customizations`

Path	Type	Default	Description
`edge-customizations[]`	`tuple`	n/a	One `(from, to, options)` tuple. For trees, `from` / `to` are parent and child values; for graphs, node labels.
`edge-customizations[].from`	`content`	required	Source node identity.
`edge-customizations[].to`	`content`	required	Target node identity.
`edge-customizations[].options`	`dictionary`	required	Per-edge options.
`edge-customizations[].options.stroke`	`stroke`	none	Full stroke override. Wins over `color` if both are set.
`edge-customizations[].options.color`	`color`	none	Paint only, at the default thickness.
`edge-customizations[].options.pattern`	`str`	`"normal"`	`"normal"` , `"dashed"` , `"dotted"` , or `"wavy"` .
`edge-customizations[].options.arrow`	`none` / `bool` / `str`	none	Per-edge arrowheads: `none` , `"end"` / `true` , `"start"` , or `"both"` .
`edge-customizations[].options.bend`	`str` / `bool`	`false`	`"left"` or `"right"` curves the edge relative to its travel direction. `false` keeps it straight.
`edge-customizations[].options.angle`	`angle`	`25deg`	How sharply a bent edge curves. Ignored when `bend` is `false` .

edge-customizations[].options.label	content / dictionary	none	Tree edge label content, or graph edge-label text overrides. A dictionary can include content for trees.

Nested keys for edge-customizations[].options.label

Path	Type	Default	Description
edge-customizations[].options.label.size	length	style.label-text.size	Edge-label size.
edge-customizations[].options.label.content	content	tree only	Tree edge label content. Graph edge labels come from adjacency entries such as ("B", [4]) .
edge-customizations[].options.label.color	color	style.label-text.color	Edge-label text color.
edge-customizations[].options.label.rotation	angle / str	style.label-text.rotation	An angle, or "edge" to follow the tree/graph edge angle.
edge-customizations[].options.label.font	str / array	style.label-text.font	Typst text font selection.
edge-customizations[].options.label.weight	str / int	style.label-text.weight	Typst text weight.

Nested keys for node-customizations

Path	Type	Default	Description
node-customizations[]	tuple	n/a	One (node, options) tuple.
node-customizations[].node	content	required	Tree node value.
node-customizations[].options	dictionary	required	Per-node drawing options.
node-customizations[].options.fill	color	normal fill	Node fill.
node-customizations[].options.stroke	stroke	normal stroke	Node outline.
node-customizations[].options.shape	str	style.node-shape	Trees/graphs only: "circle" , "square" , "rounded" , "capsule" , "diamond" , or "hexagon" .
node-customizations[].options.node-radius	float	style.node-radius	Trees/graphs only. Shape size.
node-customizations[].options.text	dictionary	style.node-text	Text style merged into the node text.

Nested keys for node-labels

Path	Type	Default	Description
node-labels[]	tuple	n/a	One (node, label) tuple.
node-labels[].node	content	required	Tree node value.
node-labels[].label	content / dictionary	required	Bare content for quick labels, or a dictionary with content plus style and placement keys.
node-labels[].label.content	content	required for dict labels	Label body. body is also accepted.

node-labels[].label.position	str / angle	"right"	"right" , "left" , "top" , "bottom" , or an angle.
node-labels[].label.offset	point	(0, 0)	Extra (dx, dy) shift after the position is applied.
node-labels[].label.size	length	style.label-text.size	Node-label size.
node-labels[].label.color	color	style.label-text.color	Node-label text color.
node-labels[].label.rotation	angle	style.label-text.rotation	Rotation of the label text itself.
node-labels[].label.font	str / array	style.label-text.font	Typst text font selection.
node-labels[].label.weight	str / int	style.label-text.weight	Typst text weight.

Nested keys for `style.node-labels`

Path	Type	Default	Description
style.node-labels.position	str / angle	"right"	Default position for all node labels in this diagram.
style.node-labels.offset	point	(0, 0)	Default extra (dx, dy) shift for all node labels.
style.node-labels.gap	float	0.22	Default distance from the node boundary in canvas units.
style.node-labels.size	length	style.label-text.size	Default node-label text size.
style.node-labels.color	color	style.label-text.color	Default node-label text color.
style.node-labels.font	str / array	inherits	Typst text font selection.
style.node-labels.weight	str / int	inherits	Typst text weight.
style.node-labels.rotation	angle	style.label-text.rotation	Default rotation of the label text itself.

The highlight colors are: yellow for the traversal path, green for an added node, red for a removed node, and blue for the nodes and visible subtree roots in an AVL rotation schematic.

5.1 Tree operations: `tree-insert()`, `tree-delete()`, `tree-search()`

Each is a constructor. Call it with a key and it returns the closure `transition` applies. These aren't object methods. Section 6 covers the `.insert` / `.delete` / `.search` fields on a `bst` / `avl` object, which wrap these same operations.

Argument	Type	Default	Description
`key`	`int` / `content`	(required)	The key to insert, delete, or search for.
`rebalance`	`dictionary`	`(:)`	`tree-insert` only. `(enabled:, all-steps:)`, both `false` by default. See below.
`step-label`	`content`	auto	Overrides the arrow label for this operation step.

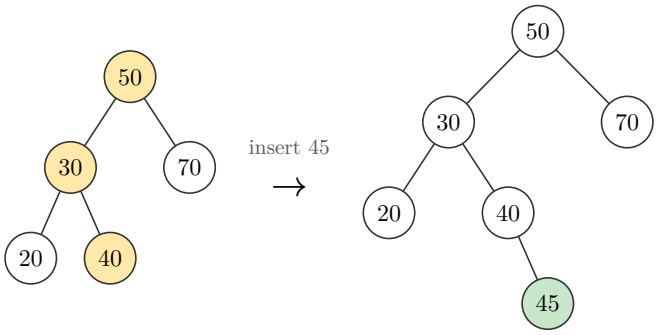
Nested keys for `rebalance`

Path	Type	Default	Description
`rebalance.enabled`	`bool`	`false`	Show the unrotated AVL tree before a rotation.
`rebalance.all-steps`	`bool`	`false`	For double rotations, split the inner and outer rotations into separate panels.

A BST insert highlights the search path on the before state and the new node on the after state.

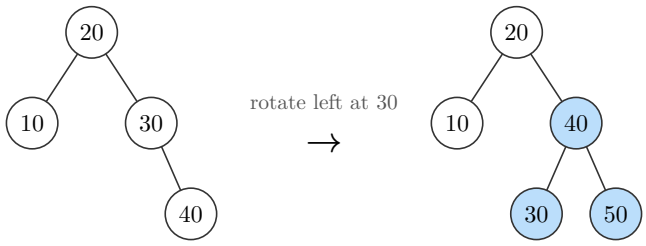
```
1  #transition("bst", (50, 30, 70, 20, 40), tree-insert(45))
```

Typst



An AVL insert that unbalances the tree triggers a rotation. Every node that changed parent turns blue: the pivot **and** the child promoted above it, plus any subtree that swapped from one to the other. The new node is green.

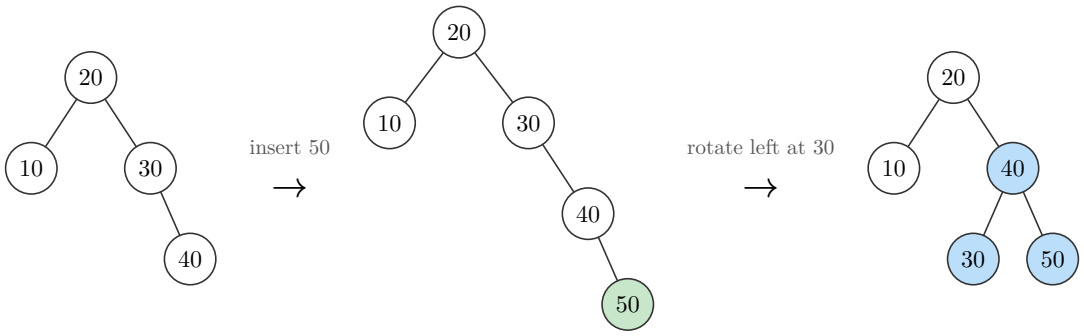
```typst
#transition("avl", (20, 10, 30, 40), tree-insert(50))
```



Argument	Type	Default	Description
`enabled`	`bool`	`false`	Adds a panel for the tree right after the plain insert, before any rotation straightens it out, when the AVL fixup actually rotates. A BST insert, or an AVL insert that never unbalances the tree, ignores this and stays 2-panel.
`all-steps`	`bool`	`false`	A **double** rotation (left-right or right-left) normally collapses straight from the unbalanced tree to the final one. `true` splits it into its own inner-then-outer panels instead. No effect on a single rotation, since there's only one step either way.

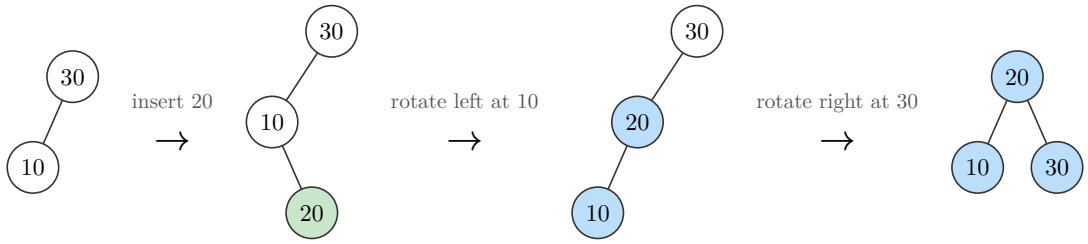
`rebalance: (enabled: true)` splits that same rotation into three panels: the tree right after the plain insert (unbalanced, new node green), then the rotation that fixes it (every reparented node blue).

```typst
#transition("avl", (20, 10, 30, 40), tree-insert(50, rebalance: (enabled: true)))
```



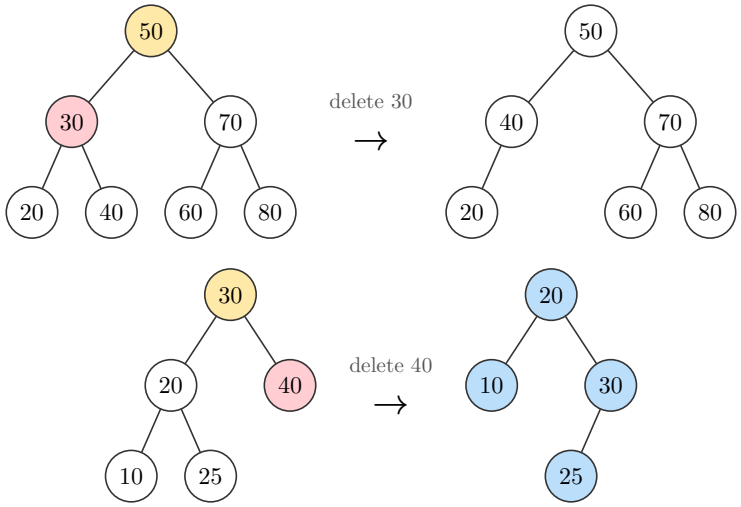
A double rotation, left-right or right-left, needs two rotations to fix. By default `enabled: true` still collapses it to 3 panels, marking every node either rotation touched; `all-steps: true` shows the inner rotation as its own step, for 4 panels total.

```typst
#transition("avl", (30, 10), tree-insert(20, rebalance: (enabled: true, all-steps: true)))
```



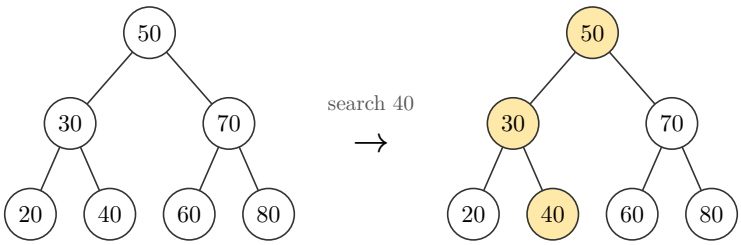
A delete marks the search path on the before state and gives the removed node the remove style. Two-child nodes are replaced by their in-order successor; AVL deletes also rebalance when removal makes a subtree too heavy, marking rotation nodes blue.

```typst
#transition("bst", (50, 30, 70, 20, 40, 60, 80), tree-delete(30))
#transition("avl", (30, 20, 40, 10, 25), tree-delete(40))
```



A search leaves the before state plain and highlights the traversal path on the after state.

```typst
#transition("bst", (50, 30, 70, 20, 40, 60, 80), tree-search(40))
```



5.2 Heap operations: `heap-insert()` and `heap-extract`

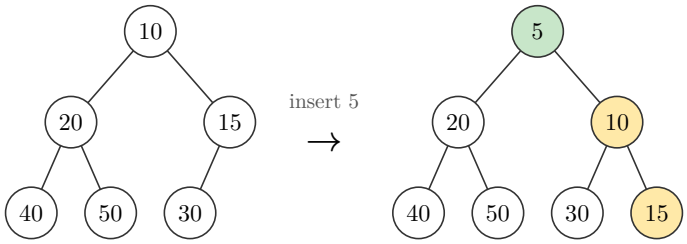
`heap-insert` is a constructor, like `tree-insert` . `heap-extract` is different: it takes no key (a heap can only pull its root), so it **is** the operation directly, passed without calling it. Object methods such as `(heap.extract)(step-label: [...])` can still override a step label.

Argument	Type	Default	Description
`key`	`int` / `content`	(required)	The key to insert, `heap-insert` only. `heap-extract` takes nothing.
`step-label`	`content`	auto	Overrides the arrow label. For `heap-extract` , use it on the object method: `(h.extract)(step-label: [...])` .

`heap-insert` appends the key, marks the inserted value green, and highlights the existing sift-up path in yellow:

```
1  #transition("min-heap", (10, 20, 15, 40, 50, 30), heap-insert(5))
```

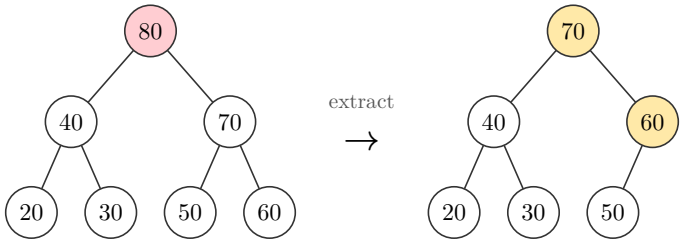
Typst



`heap-extract` removes the root (marked red), moves the last leaf into its place, and highlights the sift-down path in yellow:

```
1  #transition("max-heap", (50, 30, 70, 20, 40, 60, 80), heap-extract)
```

Typst



Note. Heap operation marks are keyed by array position while rendering, so inserting a value already present in the heap can still highlight only the new node. Tree operation marks remain value-keyed; see Section 10.

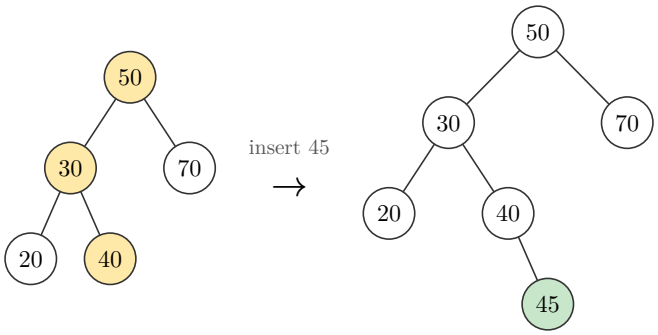
6 Objects and operations

6.1 The object model

Every builder in Section 3 returns a dictionary that is both the drawing and the structure you operate on. `.diagram` holds the rendering. An operation field derives the next state: `.insert` / `.delete` / `.search` on trees, `.insert` / `.extract` on heaps, `.push` / `.pop` on stacks, `.enqueue` / `.dequeue` on queues, `.insert` / `.delete` on linked lists and doubly linked lists.

Typst requires parentheses around a callable dictionary field, so calling an operation looks like `(obj.insert)(45)` , not `obj.insert(45)` .

```typst
#let b = bst(50, 30, 70, 20, 40)
#(b.insert)(45).diagram
```



Calling an operation field returns a **step**: a dictionary with five fields.

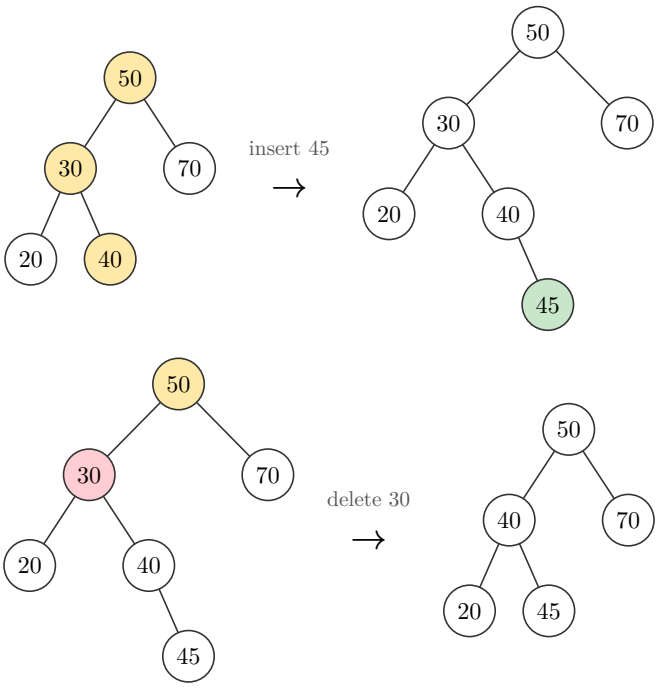
Field	Description
`.label`	The arrow's text, e.g. `"insert 45"` or `"extract"` .
`.before`	Rendered picture of the state **before** the operation, with that side's highlight marks.
`.after`	Rendered picture of the state **after** the operation, with that side's highlight marks. A picture only: no `.insert` / `.push` /etc. of its own.
`.diagram`	The packaged `.before` → arrow → `.after` row, what you show by default.
`.result`	The **next live object**, same shape as what the builder returned: `.diagram` plus operation fields, ready for the next call.

Note. `.result` is fully displayable via `result.diagram` , but plain, with no highlights. Once you have `.result` , it is a fresh object built from the post-operation state, exactly as if you had called `bst(...)` (or `min-heap(...)` , etc.) on those exact keys directly. The diff-marking machinery belongs to the one step that produced it. It doesn't travel with the resulting object, and it doesn't accumulate across a chain of `.result` calls.

6.2 Chaining operations

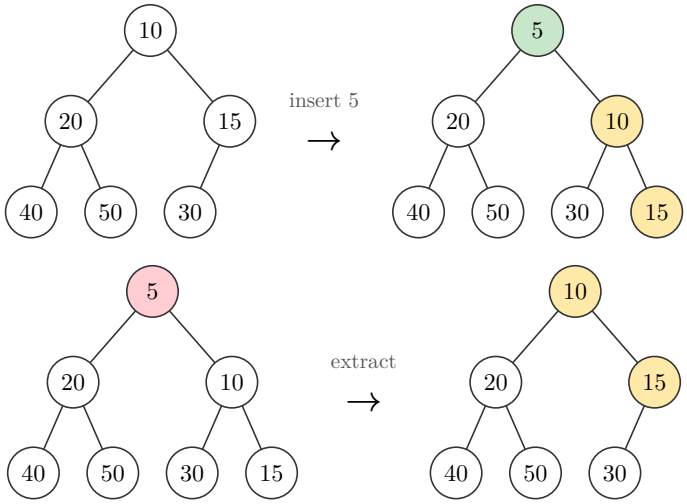
Keep calling operations on `.result` to chain an arbitrary sequence, each with its own fresh highlight:

```typst
#let b = bst(50, 30, 70, 20, 40)
#let step = (b.insert)(45)
#std.stack(spacing: 1em,
  step.diagram,
  ((step.result).delete)(30).diagram,
)
```



Every structure's natural operations chain the same way: heaps (`.insert` / `.extract`), stacks (`.push` / `.pop`), queues (`.enqueue` / `.dequeue`), linked lists (`.insert` / `.delete`):

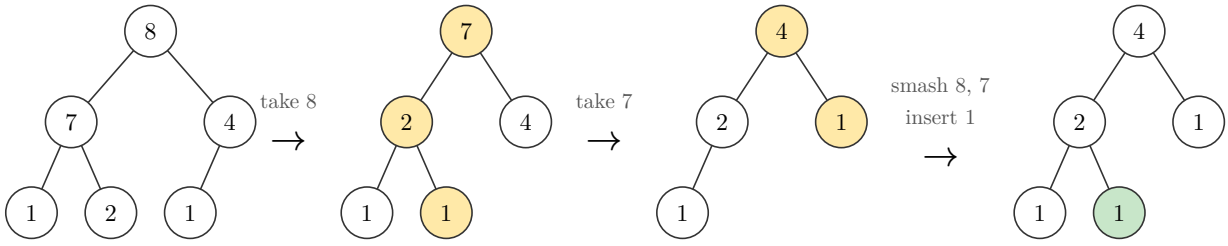
```typst
#let h = min-heap(10, 20, 15, 40, 50, 30)
#let step = (h.insert)(5)
#std.stack(spacing: 1em,
  step.diagram,
  ((step.result).extract)().diagram,
)
```



Pass `step-label:` to an operation method when a sequence needs a problem-specific arrow label:

```typst
#let h = max-heap(2, 7, 4, 1, 8, 1)
#let s1 = (h.extract)(step-label: [take 8])
#let s2 = ((s1.result).extract)(step-label: [take 7])
#let s3 = ((s2.result).insert)(1, step-label: [smash 8, 7 \ insert 1])

#sequence(h, s1, s2, s3, mode: "after", columns: 7)
```



6.3 Wrapped operation sequences

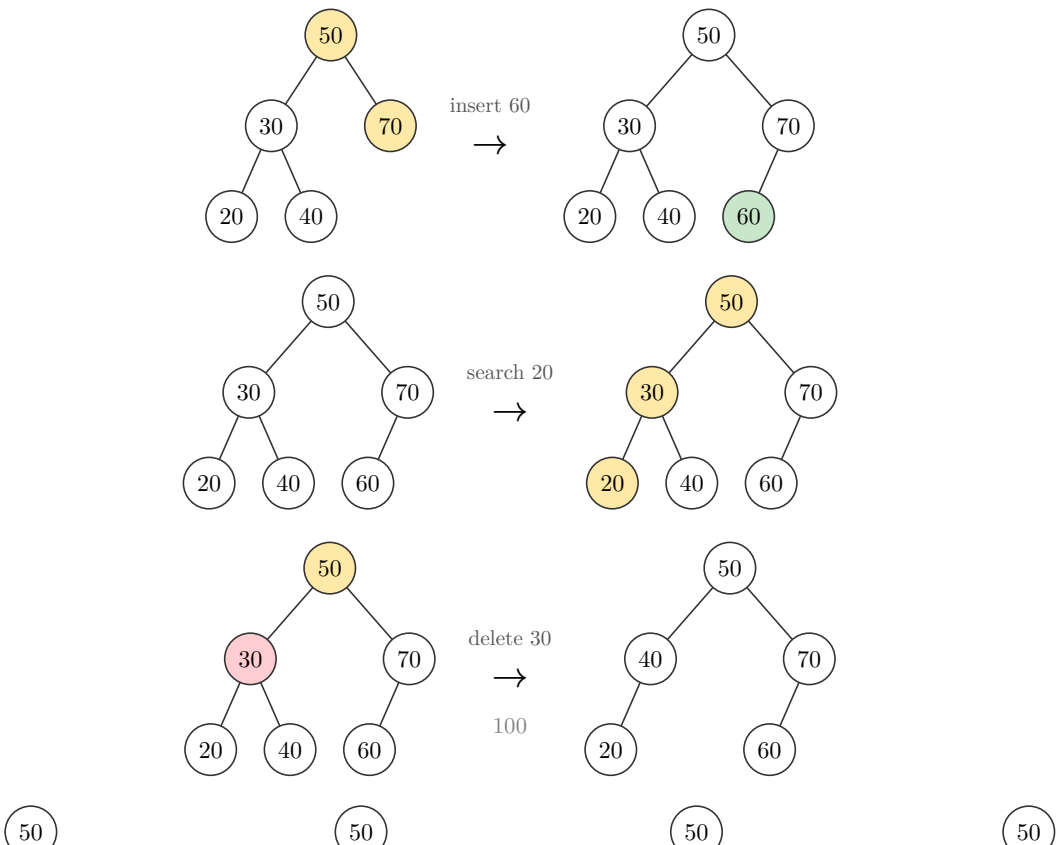
`sequence(..., columns:, mode:)` lays out step objects in a grid. It accepts operation steps directly, so you do not need to write `.diagram` for each one. Use `columns: 1` for a vertical trace, or a larger number to wrap across rows. The default `mode: "all"` shows each full transition row. Compact modes keep operation arrows between states: `mode: "after"` shows highlighted after panels, while `mode: "result"` shows the plain live object after each step, without operation highlights. Put the starting object first when you want the trace to begin with the original structure.

Argument	Type	Default	Description
`..steps`	step / content	(required)	Operation steps, live objects, or already-rendered content.
`columns`	`int`	`1`	Number of grid columns before wrapping. Compact modes count arrows as columns too.
`gap`	`length`	`1em`	Horizontal gap between columns.
`row-gap`	`length`	`1em`	Vertical gap between rows.
`mode`	`str`	`"all"`	`"all"` / `"diagram"` renders `.diagram` compact modes `"before"`, `"after"`, and `"result"` render selected states with labeled arrows between operations.
`style`	`dictionary`	`(:)`	Uses `style.operation-text` for compact-mode arrow captions.

Typst

```
#let b = bst(50, 30, 70, 20, 40)
#let s1 = (b.insert)(60)
#let s2 = ((s1.result).search)(20)
#let s3 = ((s2.result).delete)(30)

#sequence(s1, s2, s3, columns: 1, row-gap: 1.2em)
#sequence(b, s1, s2, s3, mode: "result", columns: 7)
```

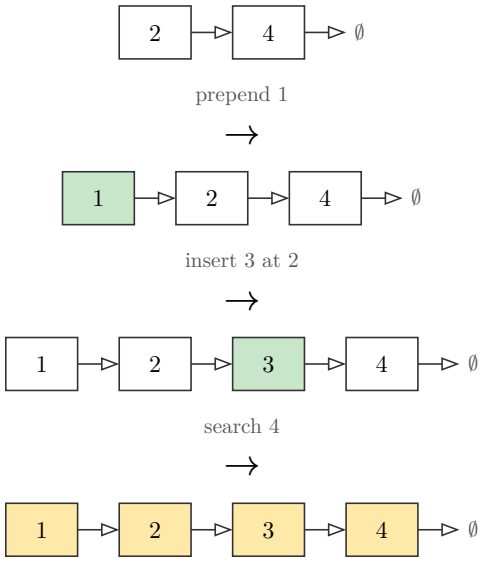


`operation-sequence` computes the chain as well as laying it out. Each operation closure receives the current live object and returns one ordinary step. The result exposes `.steps` , the final live `.result` , and `.diagram` .

Argument	Type	Default	Description
`initial`	live object	(required)	Starting tree, heap, list, stack, queue, or hash table.
`..operations`	`function`	(required)	Closures of the form `obj => (obj.operation)(..args)` .
`columns` / `gap` / `row-gap` / `mode`	same as `sequence`	`mode: "after"`	Layout settings passed to `sequence` .
`style`	`dictionary`	`(:)`	Typography for operation-arrow captions.

Typst

```
#operation-sequence(
  linked-list(2, 4),
  list => (list.prepend)(1),
  list => (list.insert)(3, index: 2),
  list => (list.search)(4),
  columns: 1,
).diagram
```

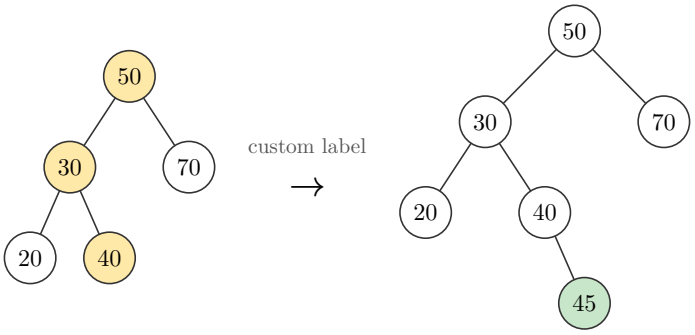


6.4 Custom arrows with `op-arrow()`

To compose the arrow yourself instead of using `.diagram` , combine the step's `.before` / `.after` with `op-arrow(label, symbol:, style:)` .

Argument	Type	Default	Description
`label`	`content`	(required)	Text shown above the arrow.
`symbol`	`content`	`$arrow.r$`	The arrow glyph itself.
`style`	`dictionary`	`(:)`	Uses `style.operation-text` for the caption.

```typst
#let step = (bst(50, 30, 70, 20, 40).insert)(45)
#std.stack(dir: ltr, spacing: 1.2em,
  align(horizon, step.before),
  op-arrow[custom label],
  align(horizon, step.after),
)
```



7 Localization

Every caption a diagram generates — the operation wording above a transition arrow, the labels in a BFS/DFS/Dijkstra trace, the phase names in a sorting tree, the `Head` / `Front` / `Rear` / `top` annotations — is drawn from a central message catalog. You can switch that catalog to another language with `language:` , override individual phrases with `messages:` , and still override any single operation call with `step-label:` . Nothing about the geometry or styling changes; only the words do.

A document that passes none of these arguments renders exactly as before, in English.

7.1 The three resolution layers

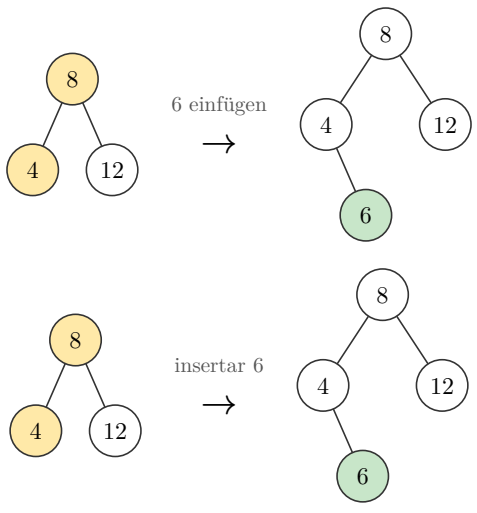
A caption is resolved by walking four sources, each one winning over the ones before it:

Layer	What it is
English / default catalog	The built-in source of truth. Contains every message key.
Selected language catalog	Chosen with `language:` . A complete translation that replaces the English strings.
`messages:` overrides	A partial dictionary layered over the selected language, per builder.
`step-label:`	The highest-priority override, for one operation call only.

7.2 `language:`

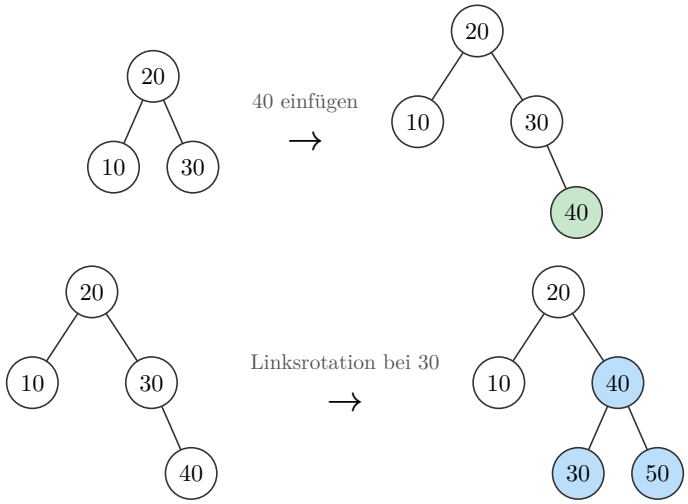
Every builder that can generate a caption accepts `language:` . The supported codes are `"en"` (default), `"de"` , and `"es"` . Any other value fails immediately with a clear assertion message, so a typo never silently falls back to English.

```typst
#std.stack(spacing: 1em,
  (bst(8, 4, 12, language: "de").insert)(6).diagram,
  (bst(8, 4, 12, language: "es").insert)(6).diagram,
)
```



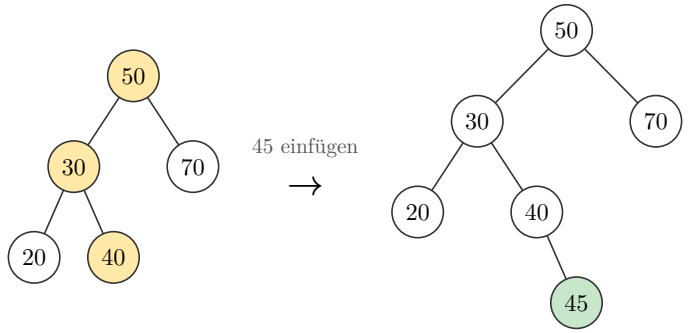
The chosen language is stored on the returned object and travels with it. Every operation you call, and every `.result` you chain from, keeps the same language without repeating the argument.

```typst
#let tree = avl(20, 10, 30, language: "de")
#let step = (tree.insert)(40, rebalance: (enabled: true))
#let next = (step.result.insert)(50)
#std.stack(spacing: 1em, step.diagram, next.diagram)
```



The standalone operation functions (`tree-insert` , `heap-insert` , and the graph and sorting builders) also take `language:` directly, so `transition(...)` and one-off traces localize the same way:

```typst
#transition("bst", (50, 30, 70, 20, 40), tree-insert(45, language: "de"))
```



7.3 `messages:` and the `messages(...)` helper

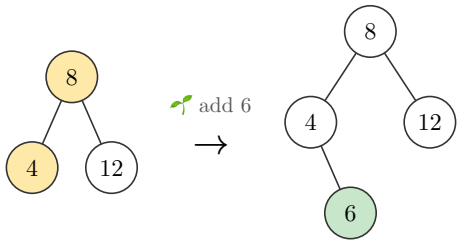
`messages:` layers a partial dictionary of overrides over whatever `language:` selected. Use it to change specific wording — house style, a different verb, a discipline's notation — without shipping a whole catalog, or to add a language the package doesn't include yet by overriding the English base.

Build the dictionary with the `messages(...)` helper, which takes one named argument per structure group and returns a flat, reusable dictionary you can share across many builders:

```typst
#let spanish-tweaks = messages(
  tree: (
    insert: key => [insertar #key],
    delete: key => [eliminar #key],
  ),
  graph: (
    visit: node => [visitar #node],
  ),
)
```

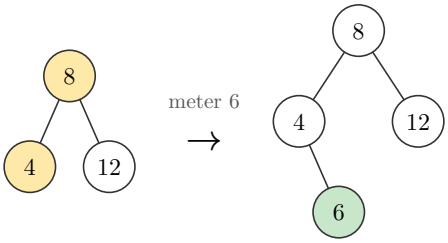
Each value is either static `content` or a callback returning `content` . Interpolating captions use a callback; fixed captions can be plain content. Callbacks receive whatever Typst value the caller passes — integers, strings, math, arrows, arbitrary content — so you are free to build a caption out of anything, not just a formatted number.

```typst
#let mine = messages(
  tree: (insert: k => [🌱 add #k]),
)
#(bst(8, 4, 12, messages: mine).insert)(6).diagram
```



Overrides compose with a language. Here the Spanish catalog supplies every caption except `tree.insert` , which the override replaces:

```typst
#let mine = messages(tree: (insert: k => [meter #k]))
#(bst(8, 4, 12, language: "es", messages: mine).insert)(6).diagram
```



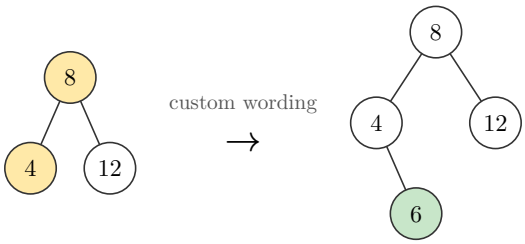
An override key that does not name a real message fails with an error naming the offending key, so a misspelled group or key is caught at compile time instead of being silently ignored. (`messages:` also accepts a plain grouped dictionary like `(tree: (insert: ...))` directly, but `messages(...)` is the recommended, validated form.)

Note. Localization is deliberately **not** part of `style:` . Styling controls how a diagram looks (color, shape, size); `messages:` controls the words. They are independent, and a caption can contain math, arrows, or styled content, so wording is always `content` , never a plain string to be concatenated.

7.4 `step-label:` has the last word

The existing `step-label:` argument on each operation is unchanged and still wins over everything: language, messages, and the default. It sets both the rendered caption and the step's `.label` field for one call only.

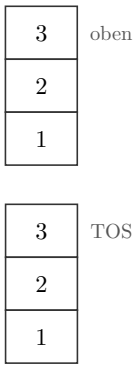
```typst
#let tree = bst(8, 4, 12, language: "de")
#(tree.insert)(6, step-label: [custom wording]).diagram
```



7.5 Structural default labels

`stack` 's `top-label` , and `queue` 's `front-label` and `rear-label` , default to `auto` , which means "use the localized default". Passing explicit content still overrides the localized word for that one builder, so an explicit label always wins over the language:

```typst
#std.stack(spacing: 1.4em,
  stack(3, 2, 1, language: "de").diagram,
  stack(3, 2, 1, top-label: [TOS]).diagram,
)
```



7.6 Message-key reference

Every key below exists in all three catalogs. In `messages(...)` , the part before the dot is the group name and the part after is the key. The **Arguments** column lists what a callback receives; keys with no arguments are static content.

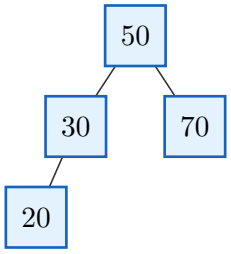
Key	Arguments	English default
`tree.insert`	`key`	insert `key`
`tree.delete`	`key`	delete `key`
`tree.search`	`key`	search `key`
`tree.rotate-left`	`pivot`	rotate left at `pivot`
`tree.rotate-right`	`pivot`	rotate right at `pivot`
`heap.insert`	`key`	insert `key`
`heap.extract`	—	extract
`list.insert`	`value`	insert `value`
`list.insert-at`	`value, index`	insert `value` at `index`
`list.prepend`	`value`	prepend `value`
`list.delete`	`value`	delete `value`
`list.delete-at`	`index`	delete index `index`
`list.search`	`value`	search `value`
`list.head`	—	Head
`stack.push`	`value`	push `value`
`stack.pop`	—	pop
`stack.top`	—	top
`queue.enqueue`	`value`	enqueue `value`
`queue.dequeue`	—	dequeue
`queue.enqueue-label`	—	Enqueue (single-frame builder annotation)
`queue.dequeue-label`	—	Dequeue (single-frame builder annotation)
`queue.front`	—	Front
`queue.rear`	—	Rear
`skip.search`	`key`	search `key`
`skip.insert`	`value`	insert `value`
`skip.delete`	`value`	delete `value`
`hash.insert`	`key`	insert `key`
`hash.delete`	`key`	delete `key`
`hash.search`	`key`	search `key`
`graph.queue`	`node`	queue `node` (BFS start)
`graph.stack`	`node`	stack `node` (DFS start)
`graph.visit`	`node`	visit `node`
`graph.inspect`	`from, to`	inspect `from` → `to`
`graph.finish`	`node`	finish `node`
`graph.reached`	`node`	reached `node`
`graph.settle`	`node`	settle `node` (Dijkstra)
`graph.relax`	`from, to`	relax `from` → `to`

graph.distance-init	node	distance(node) = 0
graph.shortest-path	node	shortest path to node found
graph.distance	value	d = value (node distance label)
sort.original	—	original array
sort.sorted-array	—	sorted array
sort.divide	—	divide (merge-sort step)
sort.divide-phase	—	Divide (merge-sort phase brace)
sort.merge	—	merge (merge-sort step)
sort.merge-phase	—	Merge (merge-sort phase brace)
sort.partition-phase	—	Partition (quick-sort phase brace)
sort.left	—	left (merge-operation column)
sort.right	—	right (merge-operation column)
sort.result	—	result (merge-operation column)
sort.start	—	start
sort.sorted	—	sorted
sort.compare	a, b	compare a and b
sort.swap	a, b	swap a and b
sort.settled	value	value settled
sort.start-merge	—	start merge
sort.merged	—	merged
sort.take	value	take value
sort.take-remaining	value	take remaining value
sort.compare-pivot	a, pivot	compare a and pivot pivot
sort.select-pivot	value	select pivot value
sort.select-last-pivot	value	select last pivot value
sort.place-pivot	value	place pivot value
sort.advance-i	i	advance i to i
sort.partitioned	—	partitioned
sort.partition-pivot	value	partition pivot value
sort.partition-around	value	partition around pivot value
sort.pivot-info	value, index, i, j	pivot: value at index i: i j: j
sort.i-satisfies	value	i value satisfies the pivot test
sort.j-satisfies	value	j value satisfies the pivot test
sort.selection-status	pos, min, item	position pos , minimum min , item item

8 Styling

Every builder takes a `style:` dictionary that is merged over the package defaults, so colors, size, and shape can be overridden per call. `theme` is the default dictionary itself and `resolve(style)` is the merge helper (`theme + style`); both are exported if you want to build your own styling utilities on top. Raw dictionaries remain supported.

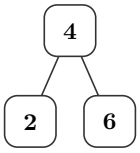
```typst
#bst(50, 30, 70, 20, style: tree-style(
  node-shape: "square",
  node-radius: 0.4,
  node-fill: rgb("#E3F2FD"),
  node-stroke: 1pt + rgb("#1565C0"),
  node-text: text-style(size: 11pt),
)).diagram
```



8.1 Named themes and typography roles

`theme-preset(name)` returns a sparse style dictionary. Available names are `"default"` , `"dark"` , `"print"` , `"colorblind"` , and `"chalkboard"` . Add a structure-specific style dictionary to override part of a preset.

```typst
#bst(
  4, 2, 6,
  style: theme-preset("colorblind") + tree-style(
    node-shape: "rounded",
    value-text: text-style(weight: "bold"),
  ),
).diagram
```



Argument	Type	Default	Description
`value-text`	dictionary	inherits `node-text`	Values inside nodes and cells.
`index-text`	dictionary	inherits `label-text`	Array indices and matrix row/column labels.
`pointer-text`	dictionary	inherits `label-text`	Pointers and linear-structure annotations.
`operation-text`	dictionary	`(size: 8pt)`	Transition and sequence arrow captions.
`edge-label-text`	dictionary	inherits `label-text`	Tree and graph edge labels.
`algorithm-label-text`	dictionary	`(size: 8pt)`	Sorting and graph-algorithm captions.

Each typography role accepts `size` , `color` / `fill` , `font` , `weight` , and `rotation` , the same nested keys produced by `text-style(...)` .

Note. `node-text` and `label-text` remain the broad compatibility defaults. The specialized text dictionaries inherit from them, so an existing style keeps applying until a more specific role overrides it.

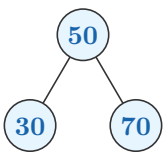
Note. To set a document-wide default instead of repeating `style:` at every call, rebind the builder once: `#let bst = bst.with(style: tree-style(..))` .

Note. Editor tip: Typst dictionaries do not expose a schema to autocomplete. Use the style-builder functions when you want argument suggestions while writing `style:` . Raw dictionaries still work when you prefer them.

8.2 Autocomplete-friendly style builders

Structure-specific style builders return sparse dictionaries. Because their keys are named function arguments, Typst editors suggest only relevant keys while you type `style: tree-style(...)` , `style: stack-style(...)` , and so on. Omitted arguments do not overwrite package or structure-specific defaults. Nested builders provide the same completion for dictionary-valued settings.

```typst
#bst(
  50, 30, 70,
  style: tree-style(
    x-gap: 1.4,
    node-fill: rgb("#E7F5FF"),
    node-text: text-style(
      size: 10pt,
      color: rgb("#1864AB"),
      weight: "bold",
    ),
    new-style: node-mark-style(
      fill: rgb("#D3F9D8"),
      stroke: 1pt + rgb("#2B8A3E"),
    ),
  ),
).diagram
```



Argument	Type	Default	Description
`tree-style(...)`	`function`	n/a	Styles BSTs, AVL trees, manual trees, and tree transitions.
`heap-style(...)`	`function`	n/a	Styles min/max heaps and heap transitions.

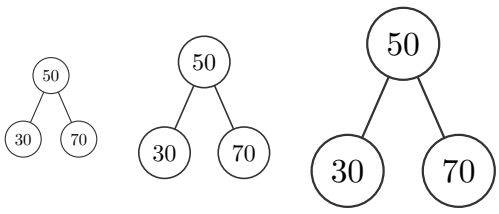
`graph-style(...)`	function	n/a	Styles graphs.
`list-style(...)`	function	n/a	Styles linked and doubly linked lists.
`stack-style(...)`	function	n/a	Styles stacks.
`queue-style(...)`	function	n/a	Styles queues.
`array-style(...)`	function	n/a	Styles arrays, including `indices:` .
`matrix-style(...)`	function	n/a	Styles matrices.
`text-style(size:, color:, fill:, font:, weight:, rotation:)`	function	n/a	Builds `style.node-text` or mark text overrides.
`label-style(size:, color:, fill:, font:, weight:, rotation:)`	function	n/a	Builds `style.label-text` .
`node-mark-style(fill:, shape:, stroke:, node-radius:, text:)`	function	n/a	Builds tree/heap diff styles.
`cell-mark-style(fill:, stroke:, text:)`	function	n/a	Builds linear-structure diff styles without unsupported node geometry.
`node-label-style(position:, offset:, gap:, size:, color:, font:, weight:, rotation:)`	function	n/a	Builds `style.node-labels` .
`indices-style(enabled:, labels:, offset:, size:, color:, font:, weight:)`	function	n/a	Builds `style.indices` . Pass `labels: auto` for zero-based indices.

8.3 Where keys are documented

The detailed key lists now live where the argument appears, including nested dictionaries such as `style.node-text` , `style.label-text` , and `edge-customizations[].options.label` .

Area	Detailed reference
Trees	`bst` , `avl` , `tree` , and `transition` list their full `style.*` , `edge-customizations.*` , `node-customizations.*` , and `node-labels.*` keys in their argument reference.
Heaps	`min-heap` , `max-heap` , and heap transitions list the tree-renderer `style.*` keys they use.
Graphs	`graph` lists `adjacency.*` , `labels.*` , `positions.*` , `edge-customizations.*` , `node-customizations.*` , `node-labels.*` , `edge-customizations[].options.label.*` , and `style.*` .
Linear structures	`linked-list` , `doubly-linked-list` , `stack` , and `queue` list their cell, pointer, label, scale, and diff-highlight `style.*` keys.

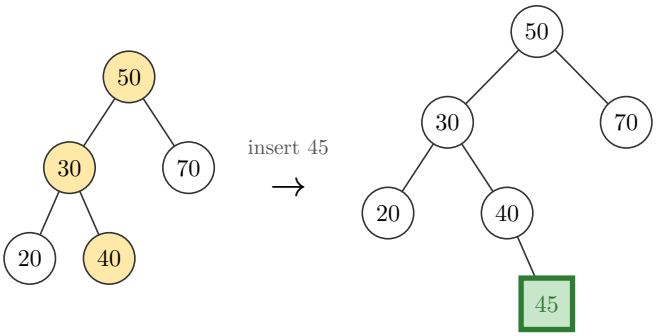
```typst
#std.stack(dir: ltr, spacing: 1.5em,
  bst(50, 30, 70, style: (scale: 0.7)).diagram,
  bst(50, 30, 70).diagram,
  bst(50, 30, 70, style: (scale: 1.4)).diagram,
)
```



8.4 Diff-highlight styling

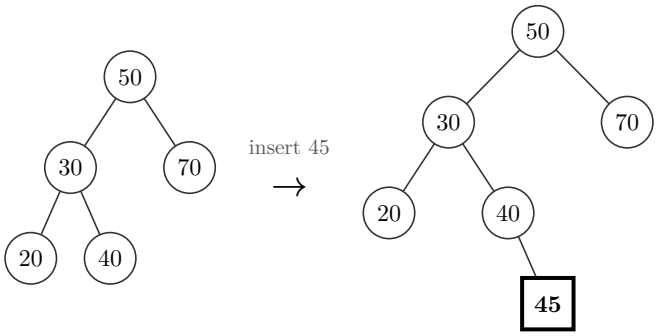
A plain color is shorthand for `(fill: color)` every example so far used that shorthand. Pass a dictionary instead to also override `shape` , `stroke` , `node-radius` , or `text` on just the marked node/cell. Fields the dictionary leaves out fall back to the surrounding `node-shape` / `node-stroke` / `node-radius` , **except** `fill` , which falls back to the key's own default highlight color, not `node-fill` . `text` merges over `node-text` , so each mark kind can have its own label color, size, font, or weight.

```typst
#transition("bst", (50, 30, 70, 20, 40), tree-insert(45), style: (
  // square instead of circle, thicker stroke; fill stays the default green
  new-style: (shape: "square", stroke: 2pt + rgb("#2E7D32"), text: (color: rgb("#2E7D32"))),
))
```



Set `diff-colors: false` to remove highlight fill colors while keeping operation marks. Shape, stroke, and radius overrides still apply.

```typst
#transition("bst", (50, 30, 70, 20, 40), tree-insert(45), style: (
  diff-colors: false,
  new-style: (shape: "square", stroke: 1.5pt + black, text: (weight: "bold")),
))
```



Note. This applies to every structure that highlights marks: `bst` / `avl` (node `shape` / `stroke` / `node-radius`), `min-heap` / `max-heap` (same, since they reuse the tree renderer), and `stack` / `queue` / `linked-list` / `doubly-linked-list` (cell `fill` / `stroke` cell marks keep the surrounding `box-shape` and do not use `node-radius`).

9 Worked example: Last Stone Weight

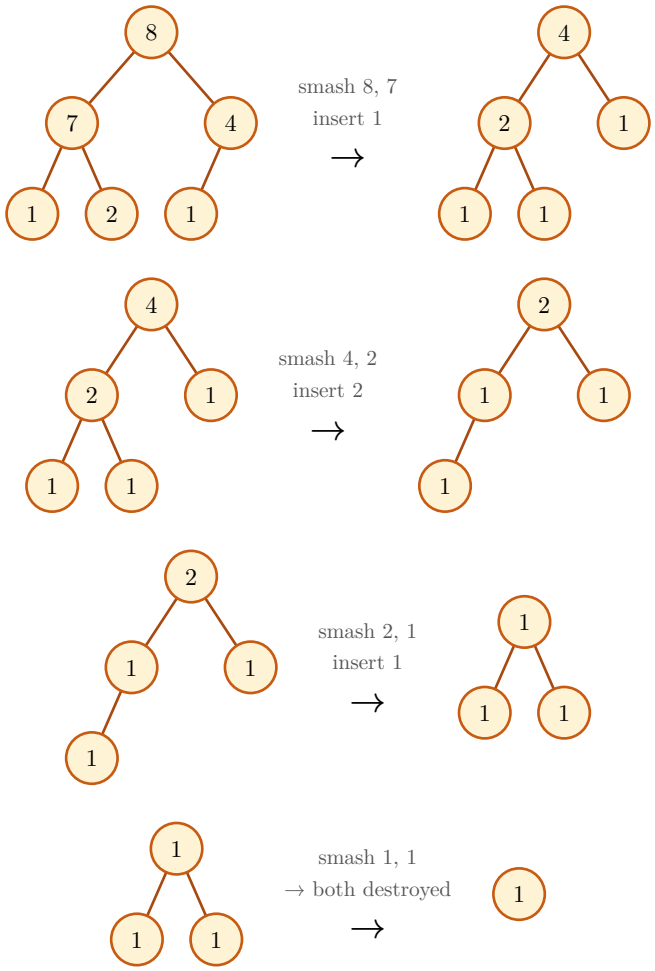
A complete example pairing a real solution with the diagram it produces: LeetCode 1046, Last Stone Weight (Easy). Repeatedly smash the two heaviest stones together; if they're equal both are destroyed, otherwise the difference goes back in. A max-heap hands you the two heaviest in $O(\log n)$ each round instead of a full rescan.

Python

```python
import heapq

def lastStoneWeight(stones):
    heap = [-s for s in stones]
    heapq.heapify(heap)
    while len(heap) > 1:
        y, x = -heapq.heappop(heap), -heapq.heappop(heap)
        if y != x:
            heapq.heappush(heap, -(y - x))
    return -heap[0] if heap else 0
```

The diagram below runs the **same** algorithm through `max-heap` , chained with `.extract()` / `.insert()` and a custom `op-arrow` label per round. The diagram comes from running the algorithm, not from hand-drawing pictures to match the code:



One stone remains, weight **1**, matching `lastStoneWeight([2, 7, 4, 1, 8, 1]) == 1` .

10 Limitations

- Tree operation diff marks are keyed by node **value**, not position, so a tree with duplicate keys can highlight more nodes than actually moved. Heap operation marks are position-keyed. For static tree/graph diagrams, use `node-customizations` for explicit per-node control. For arrays and matrices, use `cell-customizations` .
- Force layout is deterministic but heuristic. Automatic positions keep straight edges outside non-endpoint node boundaries and score candidate layouts to reduce proper edge crossings, but crossings cannot always be eliminated, especially in nonplanar graphs. Explicit position overrides can reintroduce overlaps. The layout is tuned for small teaching diagrams rather than large-scale or scientific layout.
- Layered layout requires a directed acyclic graph. Use force or manual layout for cyclic dependency diagrams.
- `graph` has no operations yet: no add-node, add-edge, or a transition to match trees and heaps.
- Graph algorithm traces show frontier state on the graph itself; they do not draw a separate queue, stack, or priority queue panel.
- The default hash function is intended for teaching examples, not cryptographic or adversarial input. Pass `hash: key => int` when bucket placement must follow a course-specific function.

11 Quick reference

11.1 Structure builders

Call	Result
`bst(..keys, style:, edge-customizations:, node-customizations:, node-labels:)`	Binary search tree built by inserting keys in order; ops `insert` , `delete` , `search`
`avl(..keys, style:, edge-customizations:, node-customizations:, node-labels:)`	AVL tree, rebalanced on every insertion and deletion; same ops as `bst`
`min-heap(..keys, style:)`	Array-backed complete binary tree, smallest key at the root; ops `insert` , `extract`
`max-heap(..keys, style:)`	Array-backed complete binary tree, largest key at the root; ops `insert` , `extract`
`linked-list(..vals, style:, pointer:, addresses:, head:)`	Linked list; ops `prepend` , `insert` , `delete` , `delete-at` , and `search`
`doubly-linked-list(..vals, style:, pointer:, addresses:, head:)`	Doubly linked list; same positional and value operations as `linked-list`
`stack(..vals, style:, top-label:)`	Stack, first argument is the top; ops `push` , `pop`
`queue(..vals, style:, enqueue:, dequeue:, front-label:, rear-label:)`	Queue, first argument is the front; ops `enqueue` , `dequeue`
`skip-list(..vals, style:, decision-fn:, level-spacing:, max-level:)`	Sorted skip list with express-lane levels; ops `search` , `insert` , `delete`
`array-view(..vals, style:, cell-customizations:, pointers:)`	Static array cells with optional `style.indices` , per-cell overrides, and labelled `pointers` arrows above cells
`matrix(rows, style:, cell-customizations:, row-labels:, column-labels:)`	Static matrix/grid cells with optional row/column labels and per-cell overrides
`graph(adjacency, directed:, labels:, positions:, layout:, layout-options:, radius:, gap:, edge-customizations:, node-customizations:, node-labels:, style:)`	Graph from an adjacency dict; circular, linear, force-directed, layered DAG, or manual layout
`hash-table(..entries, size:, collision:, hash:, style:)`	Separate-chaining or linear-probing table; ops `insert` , `delete` , and `search`

11.2 Graph algorithms

Call	Result
`bfs(adjacency, source, target:, ..graph-options)`	Breadth-first trace; result has `order` , `found` , and `path`
`dfs(adjacency, source, target:, ..graph-options)`	Depth-first trace with the same result fields
`dijkstra(adjacency, source, target:, ..graph-options)`	Non-negative shortest-path trace; result also has `distances` and `previous`

11.3 Sorting algorithms

Call	Result
`merge-sort(array, order:, labels:)`	One breadth-by-depth divide-and-merge tree from a single input array; returns `.diagram` , `.steps` , and `.result`
`merge-operation(left, right, order:, pointers:, labels:)`	Step-by-step merge of two sorted arrays, with default `i` , `j` , and `i+j` cursor pointers and a progressively filled result array
`partition-step(array, order:, pivot:, pointers:, labels:)`	Partition trace with a middle pivot, or a detailed last-pivot quicksort trace with `i` , `j` , and `pivot` markers
`quick-sort(array, order:, pivot:, labels:)`	Breadth-by-depth partition tree: every active subarray partitions on the same level; `pivot` is `"first"` , `"last"` , or an index
`bubble-sort(array, order:, pointers:, labels:, compare:, swap:)`	Comparison and swap trace; pointers mark positions with index arrows by default
`insertion-sort(array, order:, pointers:, labels:, compare:, swap:)`	Sorted-prefix insertion trace using adjacent swaps and default pointer markers
`selection-sort(array, order:, pointers:, labels:, compare:, current:, minimum:, swap:)`	Selection and swap trace with default pointer markers; red marks `i` , purple marks minimum index `m` , and yellow marks inspected index `j` . When `i = m` , the cell combines red with purple stripes
`sort-sequence(steps, columns:, gap:, row-gap:)`	Custom layout for generated sorting steps

11.4 Hand-composed trees

Call	Result
`tree(root, style:, edge-customizations:, node-customizations:, node-labels:)`	Render a tree built from `node` / `subtree`
`node(label, left:, right:, children:, fill:)`	A binary or multiway node with any content label
`subtree(label, fill:, height:, scale:)`	Triangle leaf with an optional height bracket

11.5 Transitions and operations

Call	Result
`transition(variant, keys, op, style:, edge-customizations:, node-customizations:, node-labels:)`	Before → arrow → derived after for `bst` / `avl` / `min-heap` / `max-heap` , diff highlighted
`tree-insert(key, rebalance:, step-label:)`	Operation: insert `key` . `rebalance: (enabled: true)` adds a panel for the unrotated tree, when AVL rotates; `all-steps: true` splits a double rotation into its own inner/outer steps
`tree-delete(key, step-label:)`	Operation: delete `key` AVL rebalances when needed
`tree-search(key, step-label:)`	Operation: highlight the path to `key`
`heap-insert(key, step-label:)`	Operation: insert `key` , marks the inserted value and the sift-up path

`heap-extract`	Operation: remove the root, marks it and the sift-down path (no key)
`sequence(..steps, columns:, gap:, row-gap:, mode:, style:)`	Wrap operation steps or diagrams into a grid; `mode` chooses full steps, before/after panels, or plain results
`operation-sequence(initial, ..operations, columns:, mode:)`	Apply operation closures in order and return `.steps` , final `.result` , and `.diagram`
`op-arrow(label, symbol:, style:)`	Reusable labeled arrow for custom operation layouts

11.6 Localization

Call / argument	Result
`language:`	Caption language on every generating builder: `"en"` (default), `"de"` , `"es"` . Persists through operations and `.result` . Invalid codes assert.
`messages:`	Partial overrides layered over the selected language. Build with `messages(...)` . Unknown keys assert.
`messages(..groups)`	Turn structure-grouped definitions (`tree: (insert: k => ...)`) into a flat, reusable overrides dictionary.
`step-label:`	Highest-priority caption override for one operation call; also sets the step's `.label` .
`supported-languages`	The array of accepted language codes, currently `("en", "de", "es")` .

See Section 7 for the full message-key catalog and the resolution order.

11.7 Styling keys

Key	Default	Effect
`node-radius`	`0.34`	Circle radius / half shape size.
`node-shape`	`"circle"`	`"circle"` , `"square"` , `"rounded"` , `"capsule"` , `"diamond"` , or `"hexagon"` .
`x-gap`	`1.05`	Horizontal spacing between columns.
`y-gap`	`1.2`	Vertical spacing between depths.
`tree-layout`	`"columns"`	`"columns"` or `"tidy"` (contour-packed subtrees).
`node-stroke`	`0.6pt + #333`	Node outline.
`node-fill`	`white`	Default node fill.
`edge-stroke`	`0.6pt + #333`	Edge stroke.
`edge-arrow`	`none`	Edge arrowheads: `none` , `"end"` / `true` , `"start"` , `"both"` .
`edge-arrow-fill`	`none`	`graph` arrowheads: `none` (open) or a color (solid).
`edge-pattern`	`"normal"`	Global tree/ `graph` edge pattern.
`edge-wave-amplitude`	`0.07`	Wave height in canvas units.
`edge-wave-step`	`0.14`	Approximate wavelength in canvas units.

`tri-w`	`1.2`	Subtree triangle base width.
`tri-h`	`1.4`	Subtree triangle height.
`box-w`	`0.95`	Linear-structure cell width.
`box-h`	`0.7`	Linear-structure cell height.
`box-shape`	`"square"`	`"square"` , `"rounded"` , or `"capsule"` cells.
`box-gap`	`0.55`	Gap between cells/rows.
`box-stroke`	`0.6pt + #333`	Cell outline.
`box-fill`	`white`	Default cell fill.
`ptr-fill`	`#D7ECC9`	Linked-list pointer-cell fill.
`prev-ptr-fill`	`#D7ECC9`	Doubly linked list previous-pointer fill.
`next-ptr-fill`	`#D7ECC9`	Doubly linked list next-pointer fill.
`node-text`	`(size: 9pt)`	Node/cell text styling.
`label-text`	`(fill: #555)`	Annotation/edge text styling.
`value-text`	`(:)`	Specialized node/cell value typography.
`index-text`	`(size: 7.5pt)`	Index and row/column typography.
`pointer-text`	`(:)`	Pointer/linear annotation typography.
`operation-text`	`(size: 8pt)`	Operation caption typography.
`edge-label-text`	`(:)`	Edge-label typography.
`algorithm-label-text`	`(size: 8pt)`	Algorithm trace caption typography.
`scale`	`1.0`	Uniform scale on the whole rendered diagram.
`diff-colors`	`true`	Set `false` to keep mark shapes/strokes but use normal fills.
`new-style`	`#C8E6C9`	Added node/cell; color or `(fill:, shape:, stroke:, node-radius:, text:)` .
`path-style`	`#FFE9A8`	Traversal / sift path; color or dict, as above.
`remove-style`	`#FFCDD2`	Removed node/cell; color or dict, as above.
`rotate-style`	`#BBDEFB`	AVL rotation nodes and visible subtree roots; color or dict, as above.
`visited-style` / `current-style` / `queued-style`	green / blue / yellow	Graph-algorithm node states.
`active-edge-style`	blue `2pt`	Graph-algorithm inspected edge.

11.8 Styling helpers

Call	Result
`tree-style(..tree keys)`	Sparse BST/AVL/manual-tree style dictionary
`heap-style(..heap keys)`	Sparse min/max-heap style dictionary

`graph-style(..graph keys)`	Sparse graph style dictionary
`list-style(..list keys)`	Sparse linked-list style dictionary
`stack-style(..stack keys)`	Sparse stack style dictionary
`queue-style(..queue keys)`	Sparse queue style dictionary
`array-style(..array keys)`	Sparse array style dictionary
`matrix-style(..matrix keys)`	Sparse matrix style dictionary
`theme-preset(name)`	Named `default` / `dark` / `print` / `colorblind` / `chalkboard` style dictionary
`text-style(size:, color:, fill:, font:, weight:, rotation:)`	Node/cell or mark text dictionary
`label-style(size:, color:, fill:, font:, weight:, rotation:)`	Annotation/edge text dictionary
`node-mark-style(fill:, shape:, stroke:, node-radius:, text:)`	Tree/heap diff-highlight dictionary
`cell-mark-style(fill:, stroke:, text:)`	Linear-cell diff-highlight dictionary
`node-label-style(position:, offset:, gap:, size:, color:, font:, weight:, rotation:)`	Default external node-label dictionary
`indices-style(enabled:, labels:, offset:, size:, color:, font:, weight:)`	Array-index dictionary